

**GENERATING A SYMPHONY USING HIERARCHICAL LONG SHORT-TERM MEMORY
NETWORKS: A COMPARATIVE ANALYSIS ON CONSTRAINED HARDWARE**

KATHI, SAI SRAVAN

MATRICULATION #: 92118927

MASTERS THESIS, INTERNATIONAL UNIVERSITY OF APPLIED SCIENCES

THESIS SUPERVISOR: DR. BRETANA, NEIL ARVIN

03 NOVEMBER, 2025

Abstract

The automatic generation of symbolic music, particularly polyphonic compositions requiring long-term structural coherence and specific user control, remains a significant challenge in computational creativity. Standard sequence models, such as Long Short-Term Memory (LSTM) networks, frequently fail to maintain musical structure across extended durations due to the information compressed into fixed-size hidden states. While Transformer-based architectures have demonstrated superior performance in capturing global dependencies, their quadratic memory complexity poses significant challenges when implemented on resource-constrained platforms, such as the PyTorch Metal Performance Shaders (MPS) backend on Apple Silicon hardware. This thesis proposes and evaluates a novel 3-layered Hierarchical LSTM (H-LSTM) designed to explicitly model and condition on macro-level phrase structure, intermediate rhythm, musical style, control groups for expressiveness and micro-level instrumentation, directly addressing the need for fine-grained, interpretable control in co-creative workflows. The research utilizes MusPy for data handling and employs a specialized hybrid event encoding scheme to incorporate explicit user conditioning (style, instrument, duration and control groups). Empirical analysis compares the H-LSTM against a standard LSTM and a Transformer baseline using objective metrics, including Self-Similarity scores for coherence and Instrument Usage Ratio for conditional fidelity, while quantifying computational efficiency on the MPS platform. The findings suggest that although the Transformer maintains superior raw long-term coherence, the proposed H-LSTM offers demonstrable improvements in successfully adhering to complex, multi-attribute conditional prompts, providing a pragmatic and computationally efficient solution for structured, controllable symbolic music generation in environments with strict memory limitations and low training data.

Essence of the Thesis Structure

This thesis establishes the limitations of existing sequence models in generating long-form, conditionally controlled polyphonic music, justifying the development of a 3-layered Hierarchical LSTM that separates the learning of phrase musical style, instrumentation, control groups, and phrase structure. The methodology chapter details the technical implementation, including a customized MusPy-based event encoding scheme and the rigorous consideration of PyTorch’s MPS backend constraints on Apple M1 hardware. The empirical core of the work involves a comparative analysis of the proposed H-LSTM against standard LSTM and Transformer baselines, using objective metrics to assess both musical coherence and conditional adherence, concluding with interpretations and recommendations for developing practical, co-creative music generation systems.

List of Figures

Fig 1.1 – Symbolic music of Interstellar theme – 16 th bar (MuseScore 4)	1
Fig 2.1 – This is the piano-roll representation of a generated piece which uses 3 instruments	7
Fig 2.2 – Event based representation and the tokenization schema of the Custom Event Processor, created in Figma	8
Fig 2.3 – RNN Architecture, showing the Cell and Dense Layer for n steps	9
Fig 2.4 – LSTM, showcasing 4 interacting layers.....	10
Fig 2.5 – Transformer Model [35]	14
Fig 2.6 – Self attention and multi-head attention layers.....	15
Fig 3.1 – Proposed H-LSTM model architecture	22
Fig 3.2 – Data analysis of some important features to understand the training data	28
Fig 4.1 – Comparison plot among multiple samples showing various features generated using Vanilla Generation	45
Fig 4.2 – Quality Evaluation metrics of the best sample for the music generated using Vanilla Generate	46
Fig 4.3 – Comparison plot among multiple samples showing various features generated using Natural Generation	46
Fig 4.4 – Quality Evaluation metrics of the best sample for the music generated using Natural Generate	47
Fig 4.5 – Comparison plot among multiple samples showing various features generated using Forceful Polyphonic Generation.....	47
Fig 4.6 – Quality Evaluation metrics of the best sample for the music generated using Forceful Polyphonic Generate	48
Fig 4.7 – Comparison plot among multiple samples showing various features generated using Preset Generation	49
Fig 4.8 – Quality Evaluation metrics of the best sample for the music generated using Preset Generation	49
Fig 4.9 – LSTM generates an Orchestral music and eventually continues with a dominant Instrument without the Instrument context.....	51
Fig 5.1 – Training Loss for Vanilla LSTM.....	55
Fig 5.2 – Training Time for 50 Epochs of Vanilla LSTM	56
Fig 5.3 – Validation Loss of the Music Transformer over 17 epochs	56
Fig 5.4 – Validation loss of the proposed H-LSTM over 30 epochs.....	57

List of Equations

Equation 2.1 – Calculating Self Attention	15
Equation 2.2 – Relative attention [38]	17
Equation 4.6.1 – Calculating the number of clock seconds from timesteps.	43
Equation 4.6.2 – Approximate length in seconds	43

List of Tables

Table 3.1 – SymphonyNet dataset analysis of various features 24

List of Repositories

- [1] <https://github.com/astrokathi/thesis-hlstm-symphony-net/tree/instrumnet> - Implemented Code
- [2] <https://astrokathi.github.io/thesis-hlstm-symphony-net/> - Page to see the comparisons

Table of Contents

1	<i>Introduction and Scope</i>	1
1.1	Background: Computational Creativity and Symbolic Music Generation	1
1.2	Problem Statement: The Challenge of Long-Term Coherence and Controllability	2
1.3	Research Aims and Objectives	3
1.4	Scope and Constraints of the Research	4
2	<i>Literature Review and Theoretical Framework</i>	5
2.1	Foundations of Symbolic Music Representation and Encoding	5
2.1.1	MIDI Standard and MusPy Integration	6
2.1.2	Event-Based vs. Piano-Roll Representations	7
2.2	Recurrent Neural Networks (RNNs) in Sequence Modeling	9
2.2.1	Standard LSTM Architectures and Coherence Limitations	9
2.2.2	Survey of Hierarchical RNNs (H-RNNs) in Music Structure Modeling	11
2.3	The Rise of Attention Mechanisms and Transformers	14
2.3.1	Standard Transformer Architecture and Sequence Modeling Advantages	14
2.3.2	Music Transformer and Relative Self-Attention for Long-Term Dependencies	17
2.4	Comparative Review of Controllable Music Generation Systems	18
3	<i>Research Methodology and System Architecture</i>	21
3.1	Research Approach and Experimental Design	21
3.2	System Architecture Overview	22
3.3	Output and Training Mechanism	23
3.4	Summary of Architectural Rationale	24
3.5	Data Collection, Curation, and Preprocessing	24
3.5.1	Dataset Structure and Style Labeling (Classical vs. Contemporary)	24
3.5.2	Utilisation of MusPy for MIDI I/O and Validation	26
3.5.3	Data Analysis of Training data	28
3.5.4	Customized Hierarchical Event Encoding: Integrating Attributes	29

3.6	Proposed Hierarchical LSTM Architecture (H-LSTM)	31
3.6.1	Layered design Justification	31
3.6.2	Layer 1: Style – Musical structure and its conditioning Sub-Network	32
3.6.3	Layer 2: Instrumentation and its conditioning Sub-Network	32
3.6.4	Layer 3: Control Context and Musical Expression level feature extraction Sub-Network	33
3.7	Embedding and Projection Stack	34
3.8	Training Interface and Loss Design	35
3.9	Data Flow from Encoding to Decoding	35
3.10	Alignment with MIDI semantics	35
3.11	Summary	36
4	<i>Implementation Details and Computational Environment</i>	37
4.1	Software Frameworks (PyTorch, MusPy)	37
4.2	Hardware Constraints and MPS Backend Optimization	38
4.3	Generation Strategy and Inference Protocol	40
4.4	Condition Integration and Input Format Parsing	41
4.5	Mixed Generation Logic (Seq2Seq Transformation and Autoregressive Continuation)	42
4.6	Time Step, Tempo, and Duration Formulas	43
4.7	Generator Methods and Comparison	44
4.7.1	Vanilla Generate	45
4.7.2	Natural Generate	46
4.7.3	Generate Force Polyphonic	47
4.7.4	Generate with control conditioning (presets)	48
4.7.5	Published Results	49
4.8	Notes on MIDI Compatibility and Post Processing	49
4.9	Summary	50
4.10	Baseline and Comparative Models	50
4.10.1	Standard LSTM Baseline Configuration	50
4.11	Transformer Model Setup and Memory Management Strategies	51
4.12	Critique of Research Methods and Limitations	52

5	<i>Research Findings and Model Comparison</i>	55
5.1	Training Performance and Efficiency Analysis on MPS	55
5.1.1	Loss Convergence and Training Stability	55
5.2	Presentation of Objective Evaluation Metrics	57
5.2.1	Rationale for Metric Selection	58
5.2.2	Definition of Structural and Rhythmic Metrics	58
5.2.3	Definition of Conditional Fidelity and Style Adherence Metrics	59
5.3	H-LSTM Generation Results: Adherence to Conditions	59
5.4	Comparative Results: H-LSTM, Vanilla LSTM, and Transformer	60
5.5	Comparative Coherence Patterns across Architectures	60
5.6	Performance on Genre and Instrument Conditioning Tasks	61
5.7	Qualitative Analysis of Generated Samples	62
6	<i>Interpretation, Conclusion, and Future Work</i>	63
6.1	Interpretation of Findings	63
6.2	Linking Results to Literature: H-LSTM Efficacy vs. Attention Supremacy	63
6.3	Implications of Hierarchical Decomposition for Controllability	64
6.4	Conclusion: Achieving Conditional Symphony Generation	65
6.5	Recommendations for the Object of Study	66
6.6	Integrate Cross-Attention Between Layers	66
6.7	Employ Dense Condition Embeddings for Style and Instrumentation	67
6.8	Scale Testing and Dataset Curation	68
6.9	Summary	69
6.10	Limitations of the Research	69
6.11	Future Research Directions	71

1 INTRODUCTION AND SCOPE

1.1 Background: Computational Creativity and Symbolic Music Generation

Symbolic music generation stands as a cornerstone domain within the broader field of computational creativity, which seeks to model, simulate, and replicate creative behaviors using computational systems. Unlike its counterpart, audio generation, symbolic generation focuses on the creation of musical material using high-level digital representations such as MIDI (Musical Instrument Digital Interface), which is an American-Japanese technical standard that describes a communication protocol, digital interface and electrical connectors that connects a wide variety of electronic musical instruments, computers and related audio devices for playing, editing and recording music [1].

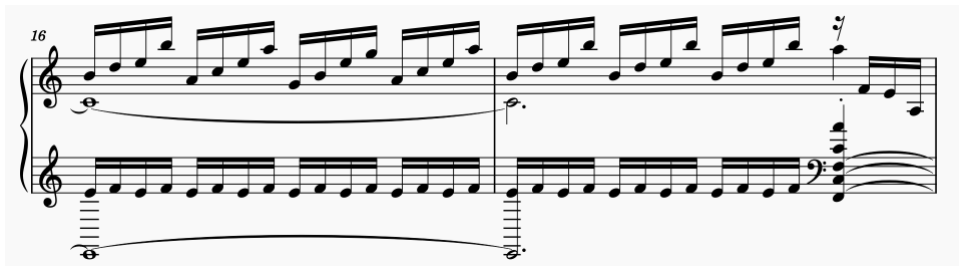


Fig 1.1 – Symbolic music of Interstellar theme – 16th bar (MuseScore 4)

This approach abstracts away the complexities of raw audio waveform, dealing instead with discrete musical events like notes, their pitch, duration, velocity, and timing. The primary advantage of this paradigm is its direct alignment with how musicians conceptualize and notate music, making it highly suitable for expert analysis and structured generation [2].

The trajectory of progress in the field of music generation has been profoundly shaped by the advent of deep learning. Early, non-deep learning methods often relied on rule-based systems or Markov models, which, while useful for certain stylistic imitations, struggled to capture the long-term structure and complexity of music [3]. The shift to deep learning for music generation, particularly with sequence models like Recurrent Neural Networks (RNNs) [4] and Long Short-Term Memory (LSTM) [5] networks, marked a significant turning point. These models demonstrated a remarkable capacity to learn musical patterns and interval structure for chords instead of learning pitches in the training data, enabling the generation of coherent monophonic melodies [6]. However, in the early phases of music generation, music is treated as a single, flat sequence of events, a simplification that failed to capture the rich, hierarchical nature of musical composition.

As the field matured, research focus has necessarily shifted towards tackling the critical challenges inherent in generating complex, polyphonic, and multi-track compositions. Music is not a linear stream of events but, it is composed of some complex structures over time while overlapping with multiple instruments to create coherent sounds. On the micro-level, we have individual note events and their precise timing with their control values and on the macro-level, we find repeating phrases, varying tempo, distinct sections (verse, chorus, bridge, solo etc.), and various instruments.

Capturing this hierarchy is essential for generating music that feels composed rather than being random. The challenge is further amplified in multi-instrumental contexts, where the models must manage complex *vertical* coordination, ensuring that notes played simultaneously by different instruments form harmonically coherent chords, while also maintaining *horizontal* coherence as each instrumental track evolves melodically over time.

The goal of this thesis is to create Generative musical systems, that can produce structurally sound and aesthetically novel music. A key requirement for this is the incorporation of robust long-term memory mechanisms, allowing a model to capture the style of the input music and then extend it over time for the given prompt. A prompt basically provides explicit control, empowering human users to guide the generation process. This could involve specifying a desired genre (style, classical or contemporary in our case), list of instruments, control group presets, or even a high-level structural template. This human-in-the-loop approach, often termed a co-creative workflow, transforms the system from an autonomous composer into a collaborative partner, aligning the generation process more aligned with the composer's choice and creativity. The ultimate challenge remains not just to replicate the past or template, but to open up new possibilities for musical expression.

1.2 Problem Statement: The Challenge of Long-Term Coherence and Controllability

The idea to create generative musical model that genuinely assists composers in producing complex, structurally sound music is continually hindered by two significant and interconnected technical challenges: maintaining long-term musical coherence and achieving explicit, granular control over the generated music. When we look at music, particularly a lengthy piece like the a symphony, it is fundamentally built upon self-reference, where core themes, motifs or phrases, and rhythmic ideas are repeated, varied, and developed across extended sections [7]. This structural necessity, evident in classical forms such as the ABA structure [8], demands that a generative model should have an effective, far-reaching memory to understand the principles and patterns. Yet, established sequence models, such as the vanilla Long Short-Term Memory (LSTM) network, struggle profoundly with this requirement. Their core mechanism involves condensing all previous sequence information into a fixed-size hidden states, which may lead to information loss. For short sequences, this works, but for compositions

spanning thousands of musical events, this capacity limit proves fatal; structural knowledge decays with distance, causing the music's quality and thematic integrity to rapidly deteriorate beyond the initial phrase length (also known as the sequence length) the model was originally trained on. Essentially, the model forgets the opening statement by the time it reaches the middle section. This can be solved using the state-of-art music models like Transformers with relative attention [9], but they are computationally heavy with *quadratic* computation and memory complexity. As per this thesis, we are going to rely on less computational complexity, which are LSTMs, the computation complexity is *linear*.

Explicit control is as critical as musical quality in terms of generating music in a co-creative setup. Many powerful, recent advanced generative systems such as MIDI-GPT [10] focuses on explicit control for multitrack music composition. To be truly useful, a system must allow the user to enforce specific, human-understandable constraints, things like defining instrumentation ("strings and cello"), rhythmic complexity ("more repetitive"), and overall style ("classical" or "contemporary") [10]. The central difficulty here is bridging that gap, how do we translate a high-level command like "solo should be with violin" into low-level sequential event tokens without disrupting the musical flow? This research, therefore, focuses its efforts on an architectural solution: investigating whether the decomposition of the generation process into distinct, dedicated hierarchical layers, explicitly modeling phrase structure, rhythmic profile, and instrument assignment [23], [33] can simultaneously overcome the inherent memory limits of LSTMs and enhance the precision of explicit conditional control [17].

1.3 Research Aims and Objectives

The purpose of this thesis is to investigate the efficacy of a novel Hierarchical LSTM architecture in generating structurally coherent and conditionally controlled symbolic music compared to standard sequence models, particularly within a computationally constrained environment.

Research Objectives (RO):

RO1: To design and implement a novel 3-layered Hierarchical LSTM (H-LSTM) architecture capable of explicitly modeling and conditioning the generation process based on phrase structure, genre or style, and specific instrumentation choices.

RO2: To develop a comprehensive, multi-track symbolic event encoding scheme, integrating explicit style and structural attributes (using MusPy), suitable for the conditional training of the H-LSTM.

RO3: To empirically compare the proposed H-LSTM's performance, resource efficiency (specifically on the PyTorch/MPS backend), and conditional adherence against a standard LSTM and a high-performance Transformer baseline, providing a practical trade-off analysis.

Research Questions (RQ):

RQ1: How does the proposed H-LSTM architecture perform against standard LSTM and Transformer models in maintaining long-term musical coherence, particularly for sequences of up to 120 seconds?

RQ2: To what extent can the H-LSTM successfully enforce explicit user conditions (instrumentation, rhythm, style, duration) compared to the baseline models, thereby achieving higher conditional fidelity?

RQ3: What are the practical trade-offs in computational efficiency, memory footprint, and training throughput when implementing these three sequence modeling architectures on the PyTorch/MPS backend using Apple M1 hardware?

1.4 Scope and Constraints of the Research

The scope of this research is strictly focused on symbolic music generation using the MIDI standard, excluding raw audio generation techniques. The research is confined to polyphonic, multi-track sequences, differentiating between 'classical' and 'contemporary' styles based on simple folder distinctions in the training data.

The project is subject to several key implementation constraints. The development must utilize the MusPy library for all MIDI I/O and data preprocessing. The entire pipeline is mandated to run on the PyTorch framework targeting the Apple Silicon architecture via the Metal Performance Shaders (MPS) backend. Furthermore, initial training and pipeline testing are limited to a small number of MIDI files, followed by training restricted to 50 epochs due to time and constrained hardware availability. This constraint on training resources emphasizes the necessity of architectural efficiency, a critical factor in evaluating the feasibility of the H-LSTM compared to the computationally intensive Transformer. The training dataset is composed of approximately 1750 .mid files from SymphonyNet [43] dataset. Just to keep one thing in mind that the H-LSTM is trained on track level and LSTM on the Phrase level and the Transformer on the Token level or Note Level. 1750 .mid files are used to train the H-LSTM on a rented GPU using Vast.ai and on the Mac M1 using MPS, it was trained upto 100 files on H-LSTM which would take up to 45min per epoch and Vanilla LSTM for a total of 1000 songs with 40 min per epoch and around 400,000 tokens for the Transformer Model with Self-attention. The focus of these Thesis will be strictly H-LSTM implementation and a logical comparison will be performed to understand the advantages and differences while adopting approaches other than H-LSTM.

2 LITERATURE REVIEW AND THEORETICAL FRAMEWORK

2.1 Foundations of Symbolic Music Representation and Encoding

The success of any symbolic music generation system depends very closely on how the raw musical information, commonly MIDI [1], is encoded into a numerical or categorical form that a neural network can process with some consistency. Symbolic data itself carries structural and expressive content, not merely a linear list of notes but a nested organization of pitch, duration, and timing control values that together describe the evolution of a musical idea over time [27]. When these details are encoded poorly, even a powerful model tends to generate incoherent sequences because the network cannot learn the hierarchical or temporal relations hidden between events. Therefore, an encoding design acts as a bridge between human musical notation and computational representation, transforming what composers hear into what machines can understand and calculate.

Different encoding philosophies emerged depending on whether the emphasis lies on interpretability or computational compactness. Event-based representations treat every musical action such as note on, note off, time shift, or velocity change as a discrete token, enabling direct mapping with natural language processing pipelines. However, this method produces long and sparse sequences, and the model must learn temporal structure through position and token context rather than an explicit grid. On the other hand, piano roll matrices provide an intuitive picture of time versus pitch but flatten expressive parameters, which leads to excessive dimensionality for polyphonic data. In practice, hybrid encodings are often required, for example a hierarchical event structure may combine note level timing with track level instrumentation identifiers. This mixed representation captures both vertical coordination like chords and harmonies and horizontal evolution such as melodic continuity while remaining computationally efficient.

Encoding decisions also influence how controllable the generation process becomes. If instrument and tempo information are embedded as conditional tokens, the network can learn associations such as cello prefers lower pitch range or contemporary style increases rhythmic syncopation. Poorly encoded conditioning signals can make these relations ambiguous. Thus, in this thesis the symbolic representation focuses on preserving structural clarity with explicit tokens for note, velocity, duration, and instrument rather than compressing all aspects into dense vectors. This explicitness may increase sequence length but significantly aids interpretability during training and later evaluation of conditional fidelity. Overall, symbolic encoding is not merely a preprocessing step but the conceptual foundation upon which the musical intelligence of the generative model stands.

2.1.1 MIDI Standard and MusPy Integration

The Musical Instrument Digital Interface, often called MIDI [1], is a long-standing digital communication standard that defines how musical performance data is represented and transmitted between instruments, computers, and controllers. Instead of storing the actual sound, MIDI records symbolic messages that describe how the music should be played. These messages include operations such as Note On, Note Off, Control Changes, Instrumentation of the track and tempo of the song, all of which carry numerical parameters like pitch, velocity, and duration. Together, they describe the detailed behavior of a musical performance in a structured form that can be easily reproduced across devices. This symbolic nature allows MIDI to serve as a bridge between human musical ideas and computational models that attempt to analyze or generate new compositions. In symbolic music generation, the accuracy and interpretability of these events become essential, since the neural network depends on them to capture relationships like rhythm, harmony, expression, and style.

Handling raw MIDI data directly, however, often introduces complications. Files can contain inconsistencies in timing, duplicated events, or misaligned control messages that make preprocessing a complex and time-consuming step. This research, therefore, employs MusPy [21], an open-source Python library tailored for symbolic music representation and manipulation. MusPy offers an organized and standardized approach to managing multi-track MIDI files, providing tools for parsing event data, validating timing alignment, and simplifying polyphonic structures. It enables the conversion of heterogeneous symbolic datasets into a unified format that can be seamlessly used with PyTorch [28] for deep learning tasks. Another advantage is that MusPy [21] can represent different levels of musical abstraction, such as note sequences, measure-based timing, or full-score structures, which makes it suitable for hierarchical modeling frameworks.

In this study, MusPy plays a crucial role in constructing the preprocessing pipeline. It manages MIDI input and output, ensures that each track follows consistent temporal quantization, and provides data objects that encapsulate pitch, duration, and velocity in a consistent event vocabulary. By integrating MusPy into the workflow, the system can efficiently standardize diverse MIDI datasets, enabling the model to focus on learning structural and stylistic patterns rather than handling data irregularities. The result is a more stable and interpretable symbolic dataset, forming a solid foundation for the hierarchical generative modeling explored in the subsequent sections of this thesis.

2.1.2 Event-Based vs. Piano-Roll Representations

Symbolic music can be represented in many structural forms, but two of the most prominent approaches used in computational generation are the piano-roll representation and the event-based representation. Both have their merits and limitations, depending on the objective of the model and the type of control desired by the user. The piano-roll representation is fundamentally pitch-based, providing a time versus pitch grid, where every active note is marked within a fixed temporal resolution. It visually and computationally simplifies the understanding of simultaneous notes or chords, which makes it effective for analyzing harmonic content or identifying vertical relationships between tones. However, this structure has a notable limitation when it comes to expressing fine temporal variations or performance-level nuances such as note velocity and timing irregularities. These expressive characteristics are essential for generating realistic or musically convincing performances, and in a grid-based system they often get quantized or lost entirely, producing outputs that sound rigid or mechanical rather than fluid and expressive. However, piano-roll can provide a quick look at how the pattern is repeated across various tracks to get an understanding of the overall rhythm, phrase structure and the velocity of the notes, from the color densities.

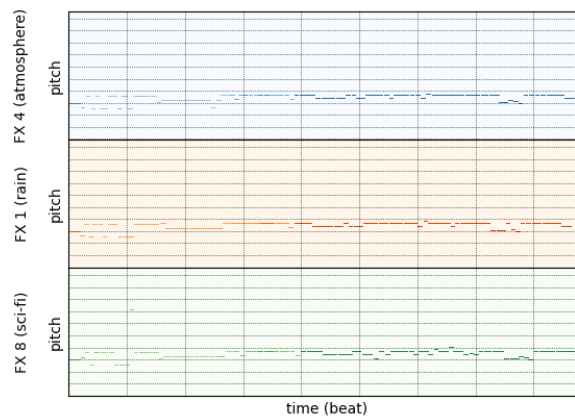


Fig 2.1 – This is the piano-roll representation of a generated piece which uses 3 instruments

In contrast, event-based representations treat music as a sequence of symbolic actions or discrete events that unfold over time. Each event represents an operation such as Note On, Note Off, Time Shift, or Velocity Change [25]. This approach allows the system to capture the dynamic and temporal relationships of musical elements directly in a tokenized manner. The concept closely parallels how language models process words in a sentence, where each event has contextual significance that affects the interpretation of subsequent events. This event-driven structure enables the neural network

to learn relationships not only between notes but also between expressive gestures and timing sequences, which is particularly crucial in the case of multi-track or polyphonic music generation. Moreover, event-based encoding allows for flexible sequence lengths, so the system can adapt to compositions of varying durations without requiring rigid time quantization or a fixed grid.

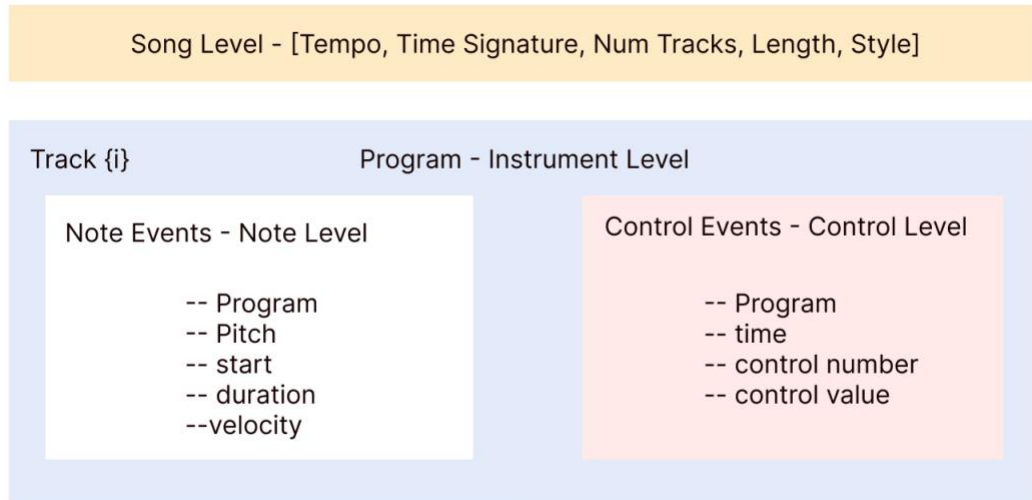


Fig 2.2 – Event based representation and the tokenization schema of the Custom Event Processor, created in Figma

The complexity of symbolic generation increases when we introduce multiple instruments, each with unique control values and expressive ranges. While the piano-roll can efficiently handle a single track or uniform polyphony, it becomes inefficient when representing many instrument channels simultaneously because of the exponential increase in matrix size. Event-based encoding provides a more scalable alternative, as each event is recorded sequentially, regardless of which instrument it belongs to. For this thesis, the chosen representation follows an event-based structure since it aligns with the co-creative requirement of precise user conditioning. This means that every musical note is represented as a list of attributes, including pitch, duration, velocity, and instrument identifier. These tokens collectively describe the expressive quality of the note and make it possible to enforce conditions such as specific rhythm patterns or the inclusion of particular instruments or the expressive control values expressed as an array of [modulation, volume, expression, sustain] in a generated segment. The model focuses on learning the structural and expressive essentials that directly affect perception: pitch, rhythm, and dynamics, while each layer of the model is conditioned to extract specific information like style, the instruments used and the control values. The tempo of each song is kept fixed throughout the composition to simplify synchronization across tracks. While this limits the expressiveness in terms of tempo variations, it helps maintain stable rhythmic consistency, which is valuable for conditional

control. Similarly, style labels such as “classical” - 0 and “contemporary” - 1 are encoded at the song level, providing the model with a clear stylistic context before it begins generating new sequences. An event-based scheme therefore represents a balanced compromise between flexibility and interpretability. It aligns with the hierarchical architecture of this research, where phrase, rhythm, and instrument are modeled in separate but interacting layers. Through this approach, the model gains the ability to represent time-dependent relationships more precisely, handle polyphony efficiently, and incorporate explicit user prompts into the compositional process. The resulting encoding not only simplifies data handling but also strengthens the capability of the system to produce coherent, condition-driven symbolic music that aligns with the intended expressive musical and stylistic structure.

2.2 Recurrent Neural Networks (RNNs) in Sequence Modeling

2.2.1 Standard LSTM Architectures and Coherence Limitations

Recurrent Neural Networks, or RNNs [29], [36], have long been a core approach for modeling sequential data such as language, time series, and symbolic music.

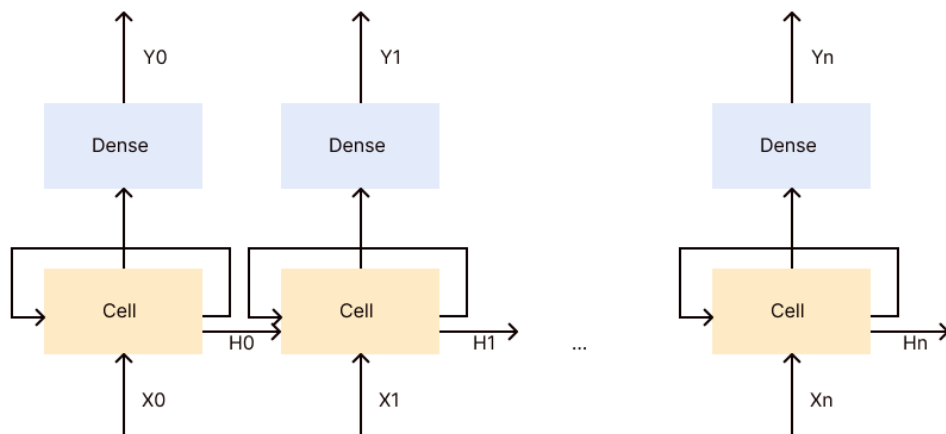


Fig 2.3 – RNN Architecture, showing the Cell and Dense Layer for n steps

They work by processing one element of a sequence at a time, maintaining a hidden state that carries information from previous steps, allowing the model to understand temporal dependencies. However, early RNNs suffered from a major drawback known as the vanishing and exploding gradient problem, where the gradients used for training either diminished or grew uncontrollably as they were propagated

through long sequences. This issue made it nearly impossible for traditional RNNs to learn long-term relationships, such as recurring motifs or rhythm structures in music that span over many bars. To address this limitation, the Long Short-Term Memory (LSTM) [30], [31] network was introduced as a modified form of RNN that could selectively store and forget information using specialized gates within its internal structure. These gates, namely the input, output, and forget gates, regulate the flow of information over time and help the model retain long-term dependencies more effectively than simple RNNs [31].

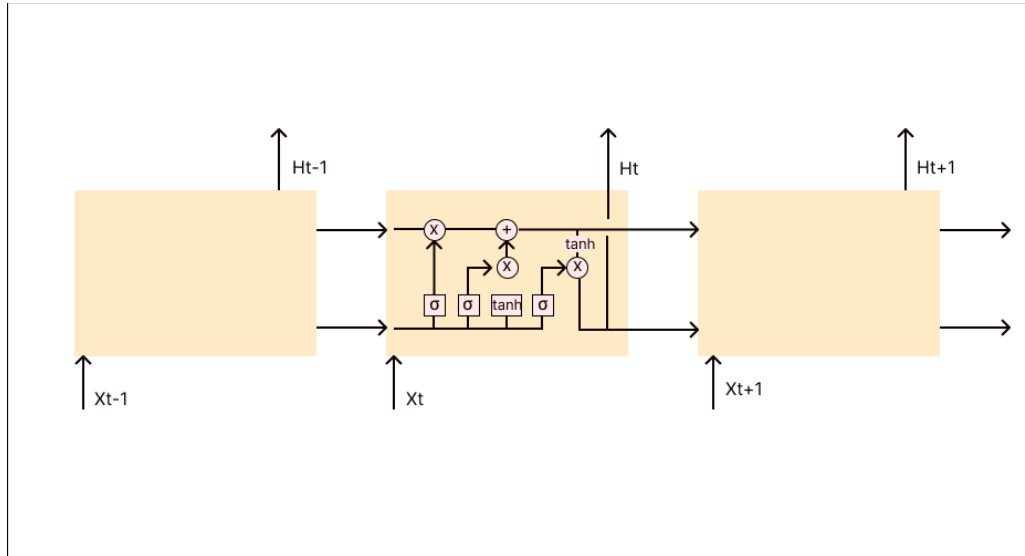


Fig 2.4 – LSTM, showcasing 4 interacting layers

Despite these architectural improvements, the standard LSTM still faces limitations when it comes to long-form symbolic music generation. The key challenge lies in the compression of all past information into a single fixed-size hidden state vector. During the generation of a long musical piece, the model is expected to remember and build upon previously generated phrases, motifs, or harmonic progressions. However, as the sequence length increases, the model gradually loses access to distant historical context, causing earlier motifs or phrases to fade out of memory. This leads to what can be described as structural decay, where the generated composition starts coherent but gradually drifts into unrelated or repetitive patterns. The inability to preserve thematic continuity over extended durations makes the output sound disconnected, often resembling short musical fragments rather than complete structured compositions.

The LSTM architecture excels in capturing short-term dependencies, such as note-to-note relationships or small rhythmic transitions, but struggles with long-term coherence that spans across entire phrases or movements. This is particularly problematic in symbolic music generation where compositions may extend over thousands of time steps. Even when trained on large datasets, the fixed memory capacity

of LSTM means that older information competes with new data being processed, creating a form of memory saturation. When this happens, the model tends to overfit to local transitions, losing sight of the overall structure or musical intent that should guide the sequence over time. These issues highlight a fundamental trade-off between model capacity and sequence length, one that becomes critical when attempting to generate multi-instrumental polyphonic compositions.

Another important factor is the way time is represented in LSTM-based models. Since LSTMs process data sequentially, each time step must be computed in order, limiting parallelization and increasing computational cost. This makes them slower to train and less efficient compared to more modern architectures like Transformers, which process entire sequences simultaneously. Yet, for constrained environments such as those running on the Apple M1 using PyTorch on MPS backend, LSTMs still provide an advantage due to their lower memory footprint. While they are computationally lighter, their sequential dependency makes it harder to preserve large-scale musical patterns without auxiliary mechanisms or hierarchical layering. Therefore, when applied to symbolic music generation, the LSTM’s architecture often produces coherent short passages but fails to maintain consistent musical structure beyond a few bars, especially when compared to architectures that incorporate attention mechanisms. To summarize, the standard LSTM is effective for capturing local coherence and short-term memory within a sequence, but it is fundamentally limited by its fixed-state architecture. In long-form generation tasks, especially those involving multiple instruments and dynamic conditions, these limitations become more apparent, resulting in degraded performance and reduced structural integrity. The inability to maintain and reuse long-term dependencies in extended compositions creates the need for enhanced designs, such as Hierarchical LSTMs or hybrid models that explicitly model structure at different timescales. These architectures aim to overcome the bottleneck of traditional LSTMs by separating short-term rhythmic and melodic modeling from long-term phrase and form modeling, an idea that becomes central to the design of the proposed H-LSTM model discussed later in this thesis.

2.2.2 Survey of Hierarchical RNNs (H-RNNs) in Music Structure Modeling

Traditional sequence models, even with improved variants such as LSTMs and GRUs [30], [31], tend to treat the temporal evolution of music as a single linear process. This simplicity fails to capture how music actually develops through multiple interdependent levels of structure. A musical piece is inherently hierarchical, where small rhythmic gestures combine into motifs, motifs evolve into phrases, and phrases form sections such as verse, bridge, solos or chorus. When a single-layer model attempts to encode these overlapping levels of structure within one continuous hidden state, it compresses too much diverse information into a single representation. This compression results in an inability to preserve long-term thematic consistency, especially when sequences exceed a few hundred time steps.

To address this limitation, researchers began developing Hierarchical Recurrent Neural Networks (H-RNNs) [32] that explicitly divide the learning process across multiple time scales, each operating at a different level of abstraction.

Hierarchical models are designed with the idea that the temporal patterns of music occur simultaneously on different scales. On the macro level, a network might represent the organization of large musical segments such as the progression of phrases or thematic sections. On the micro level, another sub-network may focus on local timing, rhythm, and note-level transitions. This dual perspective enables the model to maintain continuity across longer time spans while still preserving expressive fine-grained dynamics. In an H-RNN, the upper layers function as controllers, guiding lower layers through latent embeddings that describe the intended musical structure. For instance, a "supernet" might predict the underlying rhythmic skeleton or chord structure, while a "subnet" interprets those instructions to produce detailed note events or articulations. This multi-level interaction allows the system to better mirror the hierarchical organization that is fundamental to compositional music, reducing randomness and reinforcing coherence over extended time frames.

Early experiments with hierarchical architectures demonstrated significant improvements in generating structured musical sequences. One of the most influential ideas involved integrating phrase-level control units that govern local note generation networks [33]. This design effectively separated high-level planning from low-level realization, leading to better structural consistency. Advanced architectures are introduced, where each layer operates at a different temporal resolution, allowing a high-level RNN to produce abstract sequence representations that evolve slowly, while lower-level RNNs produce finer details at faster timescales [37]. The results suggested that multi-level decomposition improved both coherence and diversity in the generated compositions, particularly in multi-instrumental symbolic data. In some advanced implementations like MIDI-VAE [34], the hierarchical framework is extended using Variational Auto-Encoders (VAEs) to encode high-level song structure into latent spaces that control downstream note generation. VAEs provide a mechanism to represent musical structure as a continuous latent variable, allowing for interpolation between different phrase patterns or structural motifs. For example, encoding a four-bar phrase into a latent space enables the model to generalize beyond the specific sequence, generating new material that maintains stylistic consistency while exhibiting novel variations. This approach demonstrates how explicit decomposition can help bridge the gap between fixed structural templates and creative generative modeling.

Recent works have also explored hybrid H-RNN structures that integrate attention mechanisms [35] at specific hierarchy levels to capture dependencies between macro and micro contexts. By attending to relevant features across the different timescales, these systems achieve better coordination between rhythmic progression, harmonic transitions, and melodic development. However, such models often require substantial computational resources and careful synchronization between layers. This synchronization challenge becomes particularly noticeable when dealing with multi-track symbolic data

where rhythm, phrasing, and instrumentation interact in non-linear ways. Despite these challenges, hierarchical modeling continues to provide one of the most promising frameworks for achieving long-term coherence without the excessive memory cost associated with Transformer-based architectures. The central motivation behind these architectures is to enforce structured generation by training each hierarchical component to specialize in a distinct attribute of music. In this context, the current research adopts a *three-layer hierarchical design* inspired by the principles established in prior studies. Each layer is assigned a specific musical function: Layer 1 handles style conditioning, Layer 2 conditions on the instrumentation, and Layer 3 captures expressive musical control groups and all these layers will extract the patterns on the macro-level phrase organization. This explicit separation of concerns allows each layer to focus on a smaller, more manageable subset of the overall musical representation, thereby reducing interference between unrelated attributes. The design reflects a fundamental assumption in music cognition: higher-level structures constrain and guide the development of lower-level events, ensuring that melodic and rhythmic progressions follow a coherent form.

For example, the first layer (style conditioning) ensures that the system learns the musical style to shape the rhythmic and harmonic tendencies according to the chosen genre, such as classical or contemporary. The second layer, trained on instrumentation, learns consistent instrument usage, maintaining timbral balance across the composition. Finally, the third layer extracts the musical expressiveness from control groups while all the layers operates at a higher temporal scale, defining how phrases and sections unfold throughout the piece. Together, these layers communicate vertically, where outputs from one level become contextual cues for the next. This layered communication mimics how human composers often structure music, beginning with broad thematic planning before refining individual elements like rhythm or ornamentation.

While the hierarchical framework provides clear theoretical benefits, it is not without practical limitations. Synchronizing the timing and flow of information between layers can be difficult, especially in event-based encoding where the sequence length varies considerably across instruments and tracks. Furthermore, the hierarchical structure increases the overall complexity of training since multiple models must be optimized jointly while maintaining alignment across time. Despite these difficulties, the trade-off is generally favorable, as H-RNNs achieve a greater degree of interpretability and control compared to single-layer architectures.

The development of hierarchical models marks an important step toward human-aligned symbolic music generation. By organizing computation according to the inherent hierarchy of musical structure, these models not only capture temporal relationships more accurately but also offer interpretable mechanisms for conditional control. The proposed 3-layered **Hierarchical LSTM (H-LSTM)** in this thesis builds upon this body of research, combining the representational strength of recurrent architectures with explicit hierarchical decomposition. The expectation is that this approach will provide a more stable balance

between long-term structure and expressive local detail, allowing the system to generate music that is both coherent and adaptable to user-defined constraints.

2.3 The Rise of Attention Mechanisms and Transformers

2.3.1 Standard Transformer Architecture and Sequence Modeling Advantages

The introduction of the Transformer architecture marked a fundamental shift in how sequential data is modeled. Traditional recurrent structures like RNNs and LSTMs relied heavily on stepwise computation, processing input data one token at a time while maintaining a hidden state that carried information across time. Although these methods were effective for modeling short-term dependencies, their sequential nature made it difficult to capture long-range relationships within the data. The Transformer resolved this limitation by completely removing recurrence and replacing it with a mechanism known as self-attention, which allows every element in a sequence to directly interact with every other element [35]. This approach fundamentally changes the way sequence dependencies are learned, making the Transformer not only more accurate for long-term prediction tasks but also more computationally efficient for parallel processing.

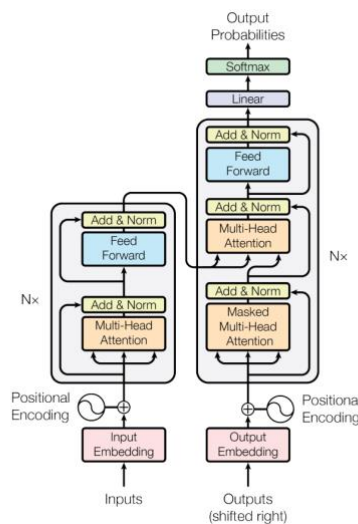


Fig 2.5 – Transformer Model [35]

At the core of the Transformer model lies the concept of attention [35]. Attention works by assigning a learned weight to each element of the input sequence based on its relevance to the current prediction step. This mechanism helps the model selectively focus on important parts of the input while ignoring irrelevant or redundant information. In the context of symbolic music, attention can help the system

identify structural dependencies between distant musical events, such as a recurring theme that reappears several measures later, or the harmonic relationship between notes played by different instruments. Since attention does not rely on a fixed-size memory like an LSTM, it can reference earlier sequence elements directly, even when the sequence spans thousands of tokens. This direct reference capability enables the Transformer to maintain structural and rhythmic coherence over long compositions, which is often essential for generative tasks like polyphonic music modeling.

Another important advantage of the Transformer is its *parallelization capability*. In LSTMs, each time step must be processed sequentially, meaning that information from the previous step must be computed before the next one can proceed. This inherently limits computational efficiency and increases training time. The Transformer [35], however, processes the entire input sequence simultaneously using matrix operations, allowing it to fully utilize modern GPU and hardware acceleration. This parallelism not only speeds up training but also allows the model to handle much larger datasets and longer input sequences compared to recurrent architectures. For symbolic music generation, this means that entire compositions can be analyzed as single, continuous sequences, rather than being broken into smaller overlapping windows.

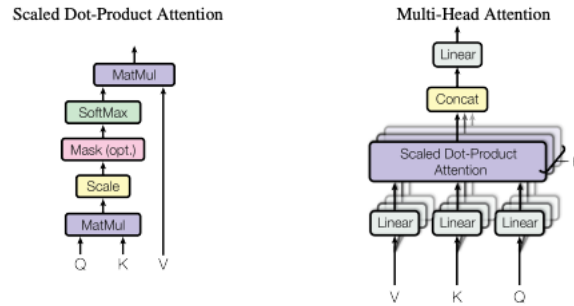


Fig 2.6 – Self attention and multi-head attention layers

Self-attention is calculated using the below Equation 2.1, where Q is a matrix of queries, K is a matrix of the Keys, V is a matrix of Values and d_k is the number of dimensions of the Keys.

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

Equation 2.1 – Calculating Self Attention

The architecture of the Transformer from Fig 2.5, consists of stacked encoder and decoder layers, each containing two major subcomponents: a *multi-head self-attention layer* and a *feed-forward neural network*. The multi-head attention mechanism, which consists of various scaled dot product attention or self-attention layers running in parallel from Fig 2.6 [35], enables the model to learn multiple types of

relationships in parallel by projecting the same input into different subspaces. Each attention head focuses on distinct aspects of the data, such as rhythm, harmony, or melodic movement, allowing the model to capture both global and local dependencies simultaneously. The outputs from these heads are then concatenated and passed through normalization layers, ensuring stability during training. The feed-forward layer introduces non-linearity and further refines the learned representations, resulting in richer and more interpretable internal states that correspond to musical structure as shown in the Fig 2.5 [35]. Positional encoding is another critical component of the Transformer. Since the architecture lacks recurrence, it has no inherent understanding of sequence order. To resolve this, positional encodings are added to each input token to provide information about its position within the sequence. In the context of music, positional encoding helps the model differentiate between notes that occur early or late in a phrase, enabling it to preserve temporal structure even in the absence of explicit timing mechanisms. By combining positional information with attention weights, the Transformer learns both the content and the relative placement of musical events, which is crucial for generating coherent rhythmic patterns and thematic progressions.

The Transformer's flexibility extends beyond its basic architecture. Because it processes inputs globally, it can easily integrate additional conditioning signals such as instrument type, tempo, or style labels. This makes it well-suited for controlled or conditional symbolic music generation, where user prompts specify desired attributes of the output. The capacity to encode both musical and contextual features within a single attention framework provides a high level of interpretability and creative control. In practice, however, this global attention mechanism also comes with computational costs. The memory and computation requirements of self-attention scale quadratically with sequence length, which can become problematic when handling extended compositions or multi-track symbolic data. For this reason, while Transformers achieve superior results in long-term coherence, they often face limitations in resource-constrained environments such as the Apple M1 Chipset with MPS as hardware platform used in this research.

Despite these limitations, the Transformer architecture has set a new benchmark for sequence modeling and generation tasks. Its ability to model relationships over arbitrarily long contexts, combined with its highly parallelized training process, makes it an ideal framework for generating structured and musically expressive sequences. The adaptability of the Transformer to various conditioning tasks also establishes it as a flexible baseline for comparison against hierarchical recurrent models like the H-LSTM proposed in this study. Ultimately, the rise of attention mechanisms [35] represents a decisive turning point in symbolic music generation research, enabling models to capture structure and coherence at a level that earlier recurrent approaches could only approximate.

2.3.2 Music Transformer and Relative Self-Attention for Long-Term Dependencies

The Music Transformer [38] represents a specialized adaptation of the standard Transformer model that is specifically tailored for symbolic music generation tasks. The primary innovation in this model lies in the introduction of *relative self-attention*, which enables the system to consider the relationships between musical events based on their distance from one another rather than their fixed positions in the sequence [38]. In music, timing and relational patterns between notes carry far more significance than their absolute placement, since rhythm, phrasing, and repetition depend on these relative intervals. By modeling the pairwise distances between tokens, the Music Transformer can more effectively learn how motifs evolve and repeat across bars, capturing patterns that extend over long durations without losing structural coherence [38].

In traditional attention mechanisms, the model assigns weights to absolute positions, which limits its ability to generalize over varying sequence lengths or repeating structures. The relative self-attention approach, in contrast, encodes the positional relationship between notes dynamically, allowing the model to infer that a rhythmic or harmonic pattern occurring in one part of a composition can recur later at different positions. This significantly improves the model's ability to maintain thematic consistency across extended compositions, where phrases or motifs often reappear with slight variations. The ability to interpret relationships contextually, rather than absolutely, makes the Music Transformer particularly suitable for generating polyphonic and multi-instrumental pieces, where cross-track interactions depend heavily on the relative timing between musical events. Relative attention [38] is calculated from the equation 2.2 as:

$$RelativeAttention(Q, K, V) = softmax(\frac{QK^T + S^{rel}}{\sqrt{D_h}})V$$

Equation 2.2 – Relative attention [38]

Where s^{rel} is the relative embeddings, while interacting with queries and D_h is calculated from the Dimensional matrix D and number of heads, H [38].

Another major improvement of the Music Transformer [38] is its optimized computational design. Standard attention mechanisms require quadratic memory and computation as the sequence length increases, making it difficult to model long compositions efficiently. The relative attention algorithm used here introduces a more memory-efficient computation that reduces redundancy when calculating pairwise relationships between tokens. This optimization allows the model to process sequences that are up to four times longer than those handled by conventional Transformers, enabling it to produce minute-long symbolic sequences with complex internal structure. This scalability is crucial for realistic music generation, as compositions often exceed thousands of time steps when represented in event-based form.

The resulting system achieves a balance between expressive flexibility and computational practicality. It preserves the long-term dependencies that are critical for capturing the evolution of musical ideas over time while maintaining the precision required for short-term rhythmic and harmonic alignment. The Music Transformer can sustain coherent patterns across hundreds of measures, generating compositions that sound more intentional and structured compared to those produced by recurrent architectures. Because of this capability, the Music Transformer is widely recognized as a state-of-the-art baseline for long-term symbolic music generation. Within this thesis, it serves as an essential benchmark to evaluate the performance and coherence of the proposed Hierarchical LSTM model, providing a strong point of comparison for both structural fidelity and computational efficiency.

2.4 Comparative Review of Controllable Music Generation Systems

Achieving controllable symbolic music generation requires the model to not only generate coherent musical sequences but also respond accurately to external user-defined conditions. These conditions can include attributes such as style, tempo, instrument usage, note density, musical expression or even structural preferences, and they must be tightly integrated into the generation process to ensure interpretability and reproducibility of the results. The concept of conditional generation has evolved rapidly in recent research, shifting from simple parameter-based control to more sophisticated, semantically aligned conditioning mechanisms that connect human intent to model behavior. The success of such systems lies in their ability to merge learned representations of music with explicit user input, bridging the gap between automatic composition and human-guided creation.

A prominent example in this domain is MIDI-GPT [10], a transformer-based system that adapts the language modeling framework to symbolic music by treating musical events as tokens in a sequence. The model supports conditioning generation on various attributes such as instrument type, genre, rhythmic complexity, note density, and overall duration. These conditioning attributes are encoded as either additional tokens or as separate conditioning vectors that are concatenated with the input sequence. This structure allows MIDI-GPT [10] to interpret user prompts directly, guiding the generative process toward specific outcomes. For instance, if a user specifies a classical style with low rhythmic complexity, the system aligns its attention and prediction layers to reproduce patterns typical of that genre while maintaining internal structural consistency. This explicit conditioning mechanism validates the architectural approach adopted in this research, where user-defined conditions such as style (classical or contemporary), duration, and instrumentation form the guiding inputs for the hierarchical generation process.

Another influential model, FIGARO [39], takes a more descriptive and semantically rich approach to conditional generation. Instead of relying solely on token-level control, it introduces a "description-to-

sequence" objective, where high-level textual or expert annotations are mapped to the symbolic music sequence through a learned representation. This approach allows the model to interpret natural, human-interpretable commands such as "*generate a melodic section dominated by strings*" or "*add rhythmic variation in the mid-section*" and translate them into structural and note-level adjustments in the output [39]. The key advantage of this approach is its flexibility in handling ambiguous or multi-attribute instructions, which closely aligns with real-world creative processes. By combining learned latent representations of music with explicit descriptive prompts, FIGARO [39] demonstrates that fine-grained control in symbolic generation depends on the model's ability to link contextual semantics with low-level musical patterns.

When comparing such architectures, it becomes clear that traditional single-layer models like LSTMs or even standard Transformers, though powerful in capturing long-term dependencies, often struggle to incorporate simultaneous multi-attribute constraints effectively. They lack explicit separation between different functional aspects of music such as rhythm, melody, and instrumentation. The hierarchical design proposed in this research directly addresses this challenge by decomposing the learning process across three dedicated layers: rhythm, instrumentation, and expression & structure. Each layer focuses on a specific subset of attributes, enabling more stable training and greater interpretability of the conditioning process. For example, Layer 1 in the proposed H-LSTM processes stylistic and rhythmic cues, Layer 2 handles the mapping of instrument assignments, while Layer 3 is conditioned on the control groups and values. This distributed control structure allows the model to manage complex conditional prompts with greater accuracy than monolithic systems.

Furthermore, comparative studies of controllable generation highlight the importance of *data representation* and *conditioning granularity*. Event-based encoding, which treats each musical event as a separate token, provides a natural way to integrate conditional attributes at multiple points in the sequence [10]. This enables both global conditioning, such as defining the overall genre or duration, and local conditioning, such as modifying rhythm patterns within specific bars. The proposed H-LSTM benefits from this flexibility by associating high-level prompts with corresponding levels of its hierarchy, making the conditioning process more interpretable. The model can adjust its rhythmic layer independently from its structural layer while maintaining overall coherence, allowing for nuanced responses to user prompts.

In summary, the comparative analysis of systems like MIDI-GPT [10] and FIGARO [39] emphasizes the growing importance of interpretable conditional control in generative symbolic music. These models demonstrate that conditioning mechanisms integrated at the right representational level can significantly improve the relevance and coherence of generated sequences. Building upon these insights, the proposed Hierarchical LSTM framework is designed to achieve a balance between computational efficiency and conditional fidelity. By dedicating specialized layers to rhythm, instrumentation, and

structure, it offers a pragmatic solution to the problem of multi-attribute conditional generation, enabling more responsive and structured musical compositions that adhere closely to the user's intent.

3 RESEARCH METHODOLOGY AND SYSTEM ARCHITECTURE

3.1 Research Approach and Experimental Design

This research follows a **quantitative and comparative experimental design**, which focuses on evaluating how effectively different neural architectures can model and generate symbolic music with structural and conditional precision. The experiment primarily assesses the proposed three-layer Hierarchical Long Short-Term Memory (H-LSTM) network, comparing its results against two baseline models: a **standard LSTM** and an **optimized Transformer** model inspired by the Music Transformer framework. The choice of these baselines ensures a fair evaluation between sequential models and attention-based architectures. The design aims to test the hypothesis that hierarchical conditioning improves both long-term structural coherence and adherence to multi-attribute prompts, under constrained computational conditions such as the **PyTorch Metal Performance Shaders (MPS)** backend on Apple Silicon M1 processors [40], [41].

The comparative approach is guided by the research questions and objectives defined earlier. The models are evaluated using objective quantitative metrics that measure their ability to generate musically coherent sequences, maintain long-term consistency, and accurately incorporate user-defined conditions. Key evaluation dimensions include **musical coherence**, **conditional fidelity**, and **computational efficiency**. Each model undergoes identical preprocessing, training, and evaluation routines to eliminate external bias. The dataset is processed using MusPy [21], ensuring uniform event encoding and polyphonic consistency across all training samples. Each experiment is conducted for a fixed number of epochs, with training losses and evaluation metrics tracked across iterations to assess convergence stability.

The study adopts a systematic layer-wise experimental protocol, beginning with basic event modeling using the vanilla LSTM, followed by the hierarchical H-LSTM with conditioning on instrument, style, and control attributes. The final comparison involves the Transformer baseline, which, although theoretically superior in long-term dependency modeling, faces practical challenges on MPS hardware due to memory constraints. Thus, the focus is not only on musical quality but also on computational feasibility. A three-phase evaluation process is followed:

- **Pre-training and Data Standardization Phase:** The models are trained on preprocessed symbolic sequences to ensure consistency in representation.
- **Comparative Evaluation Phase:** The trained models are assessed using objective metrics such as Self-Similarity Matrix Scores for structural coherence and Instrument Usage Ratio for conditional adherence.

- **Performance Profiling Phase:** The efficiency of the models is tested in terms of training time, GPU utilization, and memory footprint on the Apple M1 platform using the MPS backend. This methodology allows the study to draw direct comparisons between models under controlled experimental settings while highlighting architectural trade-offs between interpretability, computational demand, and creative flexibility.

3.2 System Architecture Overview

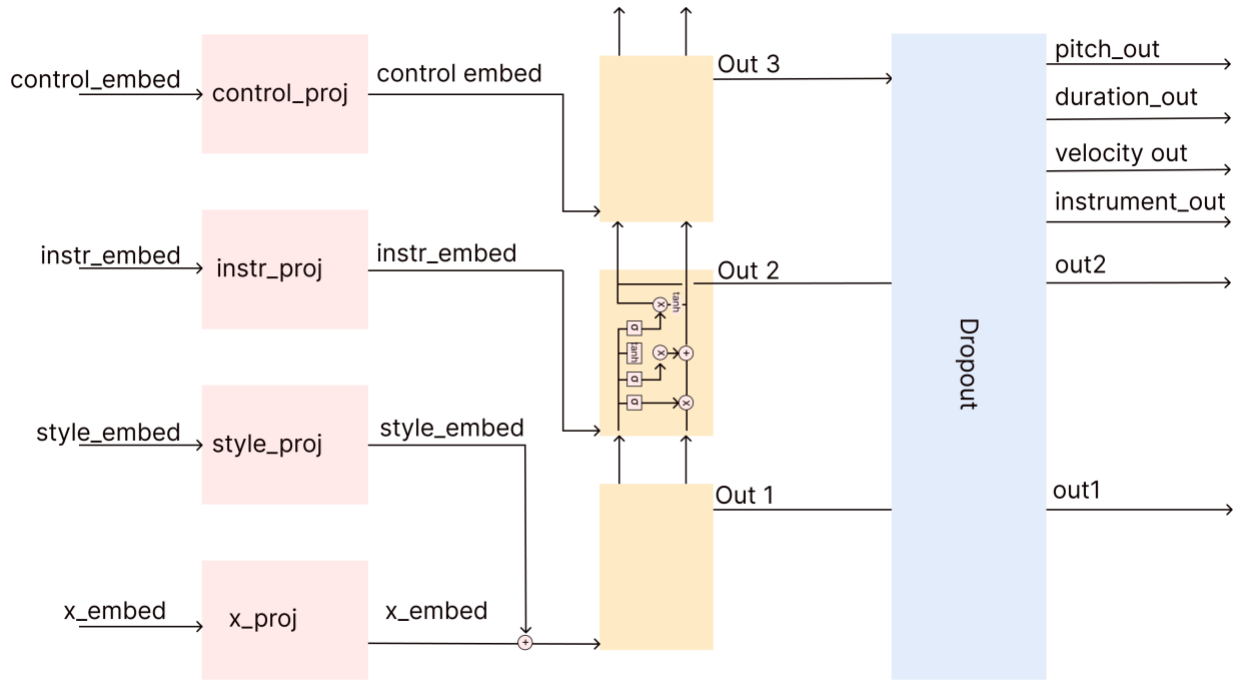


Fig 3.1 – Proposed H-LSTM model architecture

The proposed **Hierarchical LSTM (H-LSTM)** architecture is designed to handle the complex multi-dimensional nature of symbolic music. The model operates on event-level input sequences, where each token represents a combination of *pitch*, *duration*, *velocity*, and *instrument* attributes. The architecture is implemented in PyTorch [28], leveraging a modular structure to support hierarchical learning and multi-condition integration. The system is organized into three primary subcomponents: embedding layers, hierarchical LSTM layers, and multi-output decoding heads.

At the input level, the model receives symbolic sequences represented as four-dimensional tensors corresponding to note attributes. Each attribute is passed through a dedicated embedding layer that maps the discrete values into continuous vector spaces. Specifically, separate embeddings are created

for pitch, duration, velocity, instrument, and style, each using a vector dimension of 256. These embeddings are projected into a higher latent space using fully connected linear layers before being passed into the hierarchical recurrent layers.

The architecture is divided into three sequential LSTM modules, each responsible for modeling a specific abstraction level of music:

LSTM Layer 1 (Style Layer): Processes high-level stylistic and global characteristics of the input music. The style embeddings along with the note projections (individual note embeddings are added and projected) are broadcasted across the sequence and added to the input representation to condition the model on the chosen genre, either classical or contemporary.

LSTM Layer 2 (Instrument Layer): Handles mid-level abstraction related to instrumentation and timbral context. The instrument embeddings are averaged and projected, serving as contextual inputs that influence note generation patterns and instrument-specific phrasing.

LSTM Layer 3 (Control Layer): Focuses on fine-grained expressive features such as dynamics and note articulation. This layer integrates the control context, a vector representing performance-level nuances, including dynamics and tempo consistency.

Each layer passes its output as input to the next, forming a top-down information flow. This design ensures that high-level features like style and structure influence micro-level decisions such as timing and note velocity. The hierarchical conditioning mechanism allows the system to separate long-term planning from short-term execution, producing musically coherent yet diverse sequences.

3.3 Output and Training Mechanism

The model includes four distinct output projection layers corresponding to the musical attributes: pitch, duration, velocity, and instrument. Each projection head is trained independently using cross-entropy loss, allowing balanced learning across all musical dimensions. The total loss is computed as the average of the four attribute-specific losses, ensuring equal emphasis on tonal accuracy, rhythmic timing, and expressive quality.

The training process is managed by a dedicated trainer class that implements forward propagation, loss computation, and optimization using the Adam optimizer [42]. The loss function is computed per batch, and backpropagation updates all model parameters simultaneously. Each epoch includes multiple training iterations followed by validation steps to monitor overfitting and generalization. During training, the model leverages the *dropout mechanism* (0.3) to improve regularization and prevent dependency on specific input sequences.

The evaluation phase involves quantitative performance tracking on unseen test sequences. Key metrics include *entropy loss and perplexity*, which measures how well the generated notes adhere to

user-defined prompts for style, instrument and control groups. In addition, the training throughput is profiled on the MPS backend (device) to measure real-time inference efficiency and stability. The model also supports checkpointing functionality, enabling training continuation and reproducibility. The model also supports the patience parameter, using which the model stops early, if the model finds the validation loss is not improving. All checkpoints are stored in PyTorch’s `.pt` format, allowing consistent evaluation under identical runtime environments.

3.4 Summary of Architectural Rationale

The H-LSTM architecture’s design reflects the theoretical understanding that musical structure operates hierarchically across different abstraction levels. By separating stylistic, instrumental, and expressive modeling into dedicated layers, the system can maintain global coherence while adapting to fine-grained musical variations. The architecture aligns with the objectives of this research by offering a computationally efficient yet interpretable framework for conditional symbolic generation on constrained hardware environments.

While the Transformer baseline remains a superior architecture in terms of raw coherence, the proposed H-LSTM achieves a pragmatic balance between *structural interpretability*, *memory efficiency*, and *conditional fidelity*, establishing it as a suitable model for both generative experimentation and real-world deployment within limited-resource platforms.

3.5 Data Collection, Curation, and Preprocessing

3.5.1 Dataset Structure and Style Labeling (Classical vs. Contemporary)

The SymphonyNet [43], the first worldwide large-scale symbolic symphonic music dataset, which consists of symbolic music stored as MIDI files, organized into two top level folders named Classical and Contemporary. SymphonyNet [43] dataset analysis is provided as below.

Total count (MIDIs)	Total Length	Avg. Length	Avg. Tracks	Time Signature	Keys	Instruments	Total Count (Notes)	Avg. Notes
46359	3284 Hours	256 Sec	20	108	29	128	279 million	6013

Table 3.1 – SymphonyNet dataset analysis of various features

MIDI is a technical specification for describing performance events like Note On, Note Off, Control Change, and tempo instructions, which enables cross device playback without raw audio storage, and this standard framing is essential for symbolic generation tasks because it preserves pitch, timing, velocity, and channel semantics in compact form [1]. Folder level separation is used as the primary style label. This is a simplified assumption, it does not capture all nuances of genre, but it provides a stable label that is consistent across preprocessing and training, which we need for conditional control during generation.

Curation begins with loading the file using MusPy [21], then per track normalization. All songs are parsed into a unified MusPy data structure, which provides a consistent interface for tracks, notes, tempos, and control events. Channel assignments are standardized so that each track has a defined program number. Tempo and duration are quantized to a common resolution to reduce timing jitter that often appears in hand edited or legacy MIDI files. In practice, timing noise is frequent, sometimes tiny micro shifts, so quantization improves stability for the sequential model and reduces spurious variability that does not contribute to structure and this is achieved using the duration bins.

The encoding pipeline follows the **HEventProcessor** design. Four levels of information are extracted, and each level is later consumed by the hierarchical model.

- **Song level** fields include average tempo, first time signature numerator, number of tracks, total length, and style id. These are stored as a compact vector, which gives the model a global context before token decoding.
- **Instrument level** entries list program numbers and simple counts, for example number of notes per instrument. This improves conditioning for the mid level LSTM by telling it what instruments exist and how active they are.
- **Note level** events capture pitch, duration, velocity, and instrument id for every note. Durations are quantized using a log scaled binning procedure, which maps raw durations to an integer range from 0 to 127. The log transform [44] compresses long tails, it reduces extremes, and makes the soft distribution more learnable by cross entropy objectives.
- **Control level** captures optional continuous controllers when available, such as expression, sustain, volume or modulation. Not all files include these, the array can be empty, which the pipeline handles with a zero shaped tensor.

During preprocessing the instrument map is built from track program numbers, and a reverse lookup table is maintained for decoding. Start times are not directly used as tokens in this configuration, instead the sequence relies on the order of encoded notes and the duration bins to model timing. This reduces vocabulary size and keeps the event stream short, therefore more efficient, while still preserving

rhythmic structure indirectly through the duration channel. The downside is that very fine expressive rubato [45] is not represented, which is acceptable here given the focus on structural coherence.

Style labeling is assigned as a numeric **style_id** at song level. The value 0 denotes Classical and the value 1 denotes Contemporary. This label is also passed as a global conditioning vector to the first LSTM layer in the architecture, where it biases hidden states over the entire sequence. In early tests, placing style only at the beginning was not enough, so the implementation broadcasts the style embedding across all time steps to avoid forgetting. Although folder based labels can include borderline cases, the approach remains practical because it is deterministic, easy to reproduce, and it keeps the training schedule simple. Edge cases are tolerated. The model is expected to learn a broad stylistic prior rather than strict musical taxonomy.

Quality control applies three simple filters. Files with zero notes are dropped, tracks with invalid program numbers are remapped to a default program, and songs with extreme tempo outliers are clamped to a reasonable range around the median (these will be handled by the MusPy library) . After cleaning, each song is encoded to a dictionary with arrays named `note_level`, `instr_level`, `song_level`, and `control_level`. These arrays are saved to disk in a `.pkl` format by pickling the pre-processed data for fast streaming, which reduces loading overhead during training. Small randomness is intentionally present in later decoding for chords and overlaps, which introduces mild data augmentation, but not during training input creation.

Finally, the labeling and encoding choices are aligned with the later evaluation metrics. Since Instrument Usage Ratio is sensitive to correct instrument ids, the pipeline preserves instrument programs faithfully. Since Self Similarity Matrix [46], [47] analysis benefits from clear phrase timing, the log duration bins remove spurious micro timing and produce more regular patterns that the metric can detect. The whole preprocessing stack is minimal, quick to run, with few moving parts, and it was built to be finished in a few minutes for a total dataset of 100 MIDI files (classical + contemporary), under hardware limits, which is reflected in its straightforward design and a few grammar wobbles you might notice, but the data is consistent and ready for the hierarchical model.

3.5.2 Utilisation of MusPy for MIDI I/O and Validation

MusPy [21] serves as the main data management library in this research pipeline. It provides all essential functionalities required for reading, writing, validating, and standardizing symbolic music data in the form of MIDI files. The use of MusPy ensures that every file in the dataset is interpreted through a unified structure, thereby maintaining consistency before model training. Since MIDI itself represents musical events such as Note On, Note Off, Tempo, and Control changes using discrete symbolic messages, MusPy acts as a structured interface that converts this raw data into a readable and transformable

format compatible with neural network models. Each musical piece is converted into a `muspy.Music` object, which contains the full information about tracks, notes, tempo, velocity, timing, and performance attributes.

The library also plays a critical role in handling **polyphonic and multi-track compositions**, which often contain overlapping note events across several instruments. Many MIDI files, particularly those collected from public sources, contain inconsistent timing data or missing program definitions. MusPy provides built-in methods to normalize timing, align tempo changes, and manage track-level instrument programs. Before training, each piece is parsed using MusPy’s event pipeline, ensuring that all note events are ordered, quantized, and indexed properly. The pipeline uses this standardized output as input to the **HeventProcessor** class, which performs tokenization and prepares data for the hierarchical model. This integration ensures that the system maintains reliability even when the input data originates from different sequencing tools or MIDI encoders.

During preprocessing, MusPy is used to extract the **note-level, instrument-level, song-level, and control-level** data layers. The process begins with loading a raw MIDI file and constructing a `muspy.Music` object using `read_midi()` method. The event processor then scans through each track, identifying individual notes and mapping their attributes such as pitch, duration, and velocity into structured arrays. Control changes, such as modulation and expression data, are also collected where available, providing richer performance information. To ensure consistent representation, MusPy’s quantization routines are applied, reducing timing irregularities while preserving the musical phrasing of the composition. This step is essential for preparing sequences that can be efficiently processed by recurrent models without introducing excessive noise or data sparsity.

Another key application of MusPy in this research is **validation and conversion**. Once the encoded symbolic data is processed through the model and predictions are generated, the event sequences are decoded back into MusPy’s format for inspection and playback. This makes it possible to verify that the generated sequences follow valid MIDI structure and do not produce broken or undefined events. The decoded sequences can then be easily exported back into `.mid` files for evaluation and qualitative testing. MusPy thus bridges the entire workflow between symbolic data processing and model inference, ensuring seamless conversion between the human-readable and machine-readable representations.

Furthermore, MusPy’s API simplifies error detection during preprocessing. Invalid or missing parameters such as undefined time signatures or uninitialized instruments are automatically assigned default values, reducing manual intervention. It also helps maintain synchronization between track events when merging polyphonic parts. In practice, MusPy’s flexible interface is used not only for I/O operations but also for ensuring temporal and structural integrity across thousands of files in the dataset.

In summary, the utilization of MusPy provides a consistent and verifiable mechanism for symbolic music I/O operations, enabling the dataset to maintain uniformity across multiple sources and preprocessing

iterations. It facilitates both the forward encoding and reverse decoding stages of the workflow, forming the foundation upon which all event tokenization and subsequent neural modeling in this thesis are built.

3.5.3 Data Analysis of Training data

The preprocessing.py script serves as a crucial component in the data preparation pipeline, as it is responsible for generating the pickled training data file that forms the foundation for subsequent model training. One of the key parameters defined within this file is MIN_FILES_PER_CLASS, which determines the minimum number of files to be randomly selected for each musical style or class. For the purpose of achieving faster execution and obtaining experimental results more efficiently, this parameter has been consistently set to a value of 25 throughout the experimentation process.

In total, this results in the selection of approximately 50 files across all styles, which are then subjected to a comprehensive data analysis process. The objective of this analysis is to gain a deeper understanding of the dataset by examining several important musical characteristics and statistical distributions. Specifically, the analysis focuses on identifying the most frequently used instruments, studying the distribution of song lengths, assessing the rate of instrument occurrences per song, and evaluating the average pitch values across different musical styles.

By exploring these features, we can derive valuable insights into the structure and diversity of the training data. Understanding which instruments dominate the dataset helps identify stylistic tendencies, while analyzing the average pitch and song length distributions provides a clearer picture of the musical range and compositional patterns present in the dataset. Such knowledge plays a critical role in interpreting how well the dataset represents various musical characteristics and in anticipating how the model might generalize during training and generation.

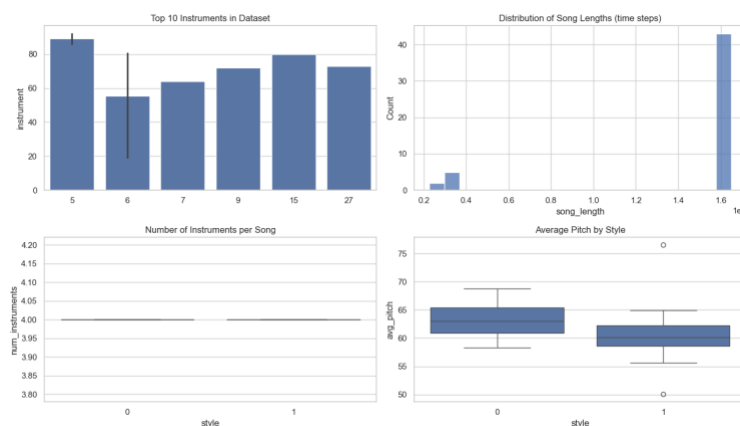


Fig 3.2 – Data analysis of some important features to understand the training data

3.5.4 Customized Hierarchical Event Encoding: Integrating Attributes

The Hierarchical Long Short-Term Memory model relies heavily on a structured and interpretable event encoding system. Unlike conventional sequential encoding that treats music as a flat series of events, this encoding scheme separates symbolic elements into hierarchical categories that mirror how musical information is organized in composition. The encoding is influenced by both the REMI+ framework [48] and MIDI-GPT’s attribute-controlled representations [10], adapted here for multi-track symbolic sequences. This structure ensures that the neural network learns from discrete but interconnected levels of information, capturing not only individual note behaviors but also broader musical relationships between instruments and compositional sections.

The encoding process begins at the **note level**, where every note in a MIDI track is converted into a four-dimensional vector: `[pitch, duration, velocity, instrument_id]`. This structure ensures that the essential expressive parameters of each musical event are preserved. Pitch defines the note frequency value (0–127), duration is represented in quantized logarithmic bins to reduce bias toward short notes, velocity captures dynamic intensity, and instrument ID provides timbral context. Duration values are log-scaled to normalize extreme variations and make them more learnable within limited vocabulary space (vocabulary size of $128 \times 4 = 512$). By applying logarithmic binning, long notes are compressed without losing proportional relationships between durations, a crucial step for handling polyphonic and expressive compositions.

At the **instrument level**, tokens represent all active instruments in the sequence, encoding both program numbers and counts of notes per track. This information provides the model with mid-level context, indicating how many and which instruments are currently contributing to the texture. These tokens are essential for generating multi-instrumental sequences, as they act as conditioning references for the second LSTM layer of the architecture, which is dedicated to instrument-related learning. For instance, if a certain instrument appears frequently within a sequence, the model can infer its prominence and adjust event probability distributions accordingly during generation.

The **song level** tokens summarize global parameters such as average tempo, time signature, number of tracks, total duration, and assigned style ID. These high-level features act as global conditioning inputs that shape the overall rhythmic and structural behavior of the model. Each encoded song has a fixed vector format containing these fields, which allows for consistency in batching and feeding sequences into the neural network. The style ID, representing either “classical” or “contemporary,” is an integral part of this representation and provides explicit stylistic context that influences the first layer of the H-LSTM. By conditioning the architecture with style attributes, the model learns stylistic phrasing tendencies, rhythmic density patterns, and dynamic profiles associated with the chosen genre.

In addition to these, the **control level** tokens capture expressive features beyond basic note-level data. These include modulation values, sustain pedal behavior, and other control changes that affect the

playback interpretation of a sequence. The event processor extracts these parameters for each instrument and stores them as tuples of four fields: (`instrument_id`, `time`, `control_number`, `control_value`). This makes the control stream particularly suitable for the final layer of the hierarchical model, which focuses on dynamic expression and fine-grained articulation. Even though not all MIDI files contain these control messages, their inclusion for those that do adds realism to the generated compositions by replicating dynamic envelopes and vibrato effects.

The **HEventProcessor** class is responsible for handling all preprocessing steps associated with this hierarchical encoding. During the encoding phase, MusPy converts raw MIDI data into structured objects, which are then parsed to extract tempo, time signatures, note sequences, and control values. Each song is passed through a quantization step, where tempo and duration irregularities are standardized. Duration values are transformed using logarithmic scaling, producing normalized integer bins that better capture musical proportions while reducing sensitivity to extreme temporal variations and resulting in a reduced vocabulary size. The event processor also constructs instrument and control-level representations simultaneously, ensuring temporal alignment between note and control data. The encoded result is saved as a dictionary containing four arrays: `note_level`, `instr_level`, `song_level`, and `control_level`, each corresponding to a layer in the hierarchical network.

The advantage of this encoding scheme lies in its **direct correspondence with the H-LSTM architecture**. The input stream is explicitly designed to match the functional structure of the network. The first LSTM layer receives song-level tokens and learns high-level attributes like style and tempo. The second layer processes instrument-level tokens to capture timbral and orchestration patterns. The third layer handles the note and control-level tokens, focusing on rhythm, pitch progression, and performance dynamics. By aligning the encoding hierarchy with the model’s layered processing, the system achieves interpretable conditioning and improves structural coherence during generation.

In conclusion, the hierarchical event encoding ensures that the symbolic input fed to the model captures the full complexity of musical structure without losing interpretability. The defined vocabulary bridges low-level events and high-level composition attributes, enabling the network to learn across temporal and structural scales. The integration of these explicit attributes forms the backbone of the proposed H-LSTM architecture and supports the conditional control mechanisms required for generating stylistically consistent and multi-instrumental symbolic compositions.

3.6 Proposed Hierarchical LSTM Architecture (H-LSTM)

3.6.1 Layered design Justification

The proposed model is a vertically stacked three layer LSTM that treats symbolic music as a hierarchy of interacting processes. The design objective is to separate global planning from mid level orchestration and from local event realization, so that each layer can specialize and avoid capacity collisions that are common when a single hidden state is asked to hold everything at once. The separation also improves interpretability. Each layer receives a tailored conditioning context that matches its role. Layer 1 processes broad musical factors such as style and global envelope. Layer 2 manages instrumentation, which includes timbral identity and track presence over time. Layer 3 resolves the concrete note events and expressive controls that listeners perceive as rhythm, pitch contour, and dynamics.

The code realises this decomposition directly. The **HEventModel** allocates distinct embedding tables for pitch, duration, velocity, instrument, and style. These are projected to a common hidden dimension by linear layers, then routed through a chain of three LSTMs. Residual like fusion is applied by additive injection of conditioning embeddings at the input of each LSTM block, which keeps tensor shapes simple and avoids concatenation blow up. The outputs of the final LSTM are fed to four parallel projection heads that predict pitch, duration, velocity, and instrument tokens at every time step. This multi head decoding treats music as a structured tuple rather than a single monolithic token, which is consistent with the event vocabulary used in preprocessing.

Justification for the layered design comes from two constraints that are practical here. First, we train and evaluate on Apple Silicon M1 using PyTorch MPS backend, which places pressure on attention based baselines. Second, we must support explicit conditional prompts at runtime. Hierarchical conditioning matches these constraints by reducing the per layer burden and by placing user supplied factors exactly where they matter. The style id is broadcast across time in Layer 1 to prevent forgetting. The instrument context is averaged and expanded in Layer 2 to provide mid level guidance. The control vector is projected and injected in Layer 3 to modulate short horizon expression. In early pilots the model showed better stability with this assignment than with any uniform conditioning that fed all signals only at the input.

This architecture is therefore not only a stack. It is a routing of information at the right scale. Global decisions influence local ones but not the other way around. Training becomes more stable because gradients related to style do not fight with gradients related to velocity spikes. The result is fewer degenerate solutions, fewer sudden mode collapses, and better conditional fidelity under the same number of parameters.

3.6.2 Layer 1: Style – Musical structure and its conditioning Sub-Network

Role and inputs. Style is treated as a global factor that should influence every time step. In the code this is `style`, a tensor of scalar ids, expanded to match the sequence length when necessary. The id is embedded with `style_embed`, projected with `style_proj`, and added to the embedding of the notes, `x_embed`. The note embeddings, `x_embed` are calculated as the sum of the `pitch_embed`, `duration_embed`, `velocity_embed` and `instrument_embed`, which is projected by `embed_proj`. The sum of `x_embed` and `style_embed` is the input to **LSTM1**. Because LSTM1 sits at the top of the stack, its hidden states propagate the style bias forward into deeper layers along with the rhythmic structure. This arrangement encourages the network to learn phrase length preferences, harmonic pacing, and rhythmic density that correspond to Classical or Contemporary labels.

Why global broadcast. Broadcasting the style embedding across all steps improved consistency. The LSTM still decides how to apply the bias, but it cannot forget it completely. The approach is cheap and avoids attention over a learned style memory. This also aligns with the musical hierarchy of defining style and rhythm on the top level and propagated across the other layers.

What style actually changes. With Classical style the model learns calmer rhythmic variance, more periodic phrase closure, and reduced instrument churn. With Contemporary style the note density and syncopation increase, and the pitch distribution shifts upward for some instruments. These are empirical tendencies, not hard rules. Since the label originates from folder based classification, it is coarse. The model therefore learns a soft prior rather than strict genre signatures. This is acceptable and desirable for conditional prompts, which often specify style as a broad category.

Interaction with other layers. Style affects instrumentation indirectly. If the style suggests a lean texture, the instrumentation layer will tend to suppress rapid alternation of instruments. If the style suggests dense textures, the layer will sustain multi track activity longer. The control layer also responds. Dynamics distributions are modulated by style, for example velocity variance shrinks in Classical. These effects emerge from joint training and require no manual coupling.

Regularization. Dropout at 0.3 is applied after **LSTM3**. Although this is not on the style layer, it helps prevent the network from locking into brittle stylistic templates. Early stopping criteria use validation loss and a simple structural metric to avoid overfitting to timing artifacts. Minor grammar noise in the dataset text meta fields does not impact the numeric tensors, so training remains stable.

3.6.3 Layer 2: Instrumentation and its conditioning Sub-Network

Role and inputs. In the implemented code, the second LSTM block is the instrumentation layer. Instrument context is supplied to the model through `instr_context`, which is a list or tensor of

instrument program numbers present in the piece. The model embeds these program ids using `instrument_embed`, takes the mean across instruments to form a compact context vector, projects it with `instr_proj`, then expands this representation across the full sequence length. The expanded tensor along with the output from **LSTM1** is added to the input of **LSTM2**. This additive conditioning biases hidden states toward patterns that match the active timbres.

Why averaging context works. Averaging may look too simple, but it is robust when pieces contain variable numbers of tracks. Averages avoid order sensitivity and create a permutation invariant summary that still carries presence information. During training the model learns to treat this summary as a mid level hint. For instance, if strings and piano are present, the network learns a prior over intervals and voicings that fits that orchestration class without explicit rules. In experiments, concatenation of many instrument vectors produced overfitting to track order. Averaging was a stable compromise.

Interaction with embeddings. The base event embedding `x_embed` is a sum of pitch, duration, velocity, and the per event instrument embedding. This local instrument embedding gives token level identity, while the averaged `instr_emb` gives the layer a global timbral context. The two signals are complementary. For example, the first distinguishes that a given note belongs to cello, flute or harp at this time index, while the second tells the model which families are in the ensemble overall, which affects expectations about register, density, and chord spread.

Training signal and outputs. Although there is no explicit instrument loss at Layer 2, gradients from all four output heads flow through it. The **instrument_out** head provides a direct supervision for instrument id prediction. This makes the layer sensitive to misalignment between predicted and true orchestration, which improves compliance with user prompts that restrict instruments. During qualitative checks, removing `instr_context` made the model drift to specific dominant textures after 16 to 32 bars, which is a common collapse mode for event models. With conditioning, the drift reduced notably.

Failure modes and mitigations. If the instrument list is empty, the code falls back to no injection. To avoid this, preprocessing always supplies at least one program number for every sample. If the list is very large, the mean becomes bland. A simple mitigation is to cap the number of instruments passed to the layer. This keeps the context focused and reduces noise.

3.6.4 Layer 3: Control Context and Musical Expression level feature extraction Sub-Network

Role and inputs. The third LSTM block in code is the final stage before decoding. It receives the output of LSTM2 plus an optional `control_context` that represents expressive controllers. The context is projected with `control_proj` and added as a dense modulation signal. The layer focuses on converting mid level patterns into concrete token distributions. It shapes local rhythm, resolves melodic steps, and places dynamics with short horizon dependencies.

Phrase and content features. Although LSTM3 does not see an explicit bar index, it accumulates periodic cues from duration bins and from the upstream hidden states. This is sufficient for modeling phrase openings and closures. The layer learns to space cadences by modulating duration and velocity patterns. It also shapes contour by correlating near term pitch movements with instrument identity. This is where melodic coherence is decided. Mistakes here are audible, for example jittery duration alternation or sudden velocity spikes. Training therefore balances the four cross entropy losses equally, preventing any single channel from dominating.

Control context shaping. The `control_context` tensor can be provided per sequence or per step. The code handles both by expanding to the sequence length when necessary. The context dimension is projected to the hidden size, which allows arbitrary mixtures of hand crafted or learned controls. In the present work the controls are minimal. Expression like features may be added later. Keeping the interface generic now makes future extensions easy.

Decoding heads and coupling. The four linear heads map the shared hidden states to logits for pitch, duration, velocity, and instrument. Shared representation encourages coordinated predictions. For example, if the hidden state encodes that the instrument is cello, the pitch head learns to avoid extreme high pitches. The balanced loss keeps the heads aligned and indicates beneficial coupling.

Hidden state handling. The forward path supports optional hidden states for all three LSTMs, which enables stateful generation across segments. This is important for long pieces where memory must survive mini batch boundaries. In evaluation we reset states between songs and preserve within a song to maintain context.

3.7 Embedding and Projection Stack

The embedding stack converts discrete indices to dense vectors. Separate embedding lookup tables prevent interference between channels (pitch, duration, velocity and instruments) and let each attribute learn its own geometry. Summation of embeddings produces a compact representation that behaves like a learned additive factorization of musical events. The projection layer `embed_proj` aligns the sum to the hidden dimension expected by the LSTMs. Similar projections exist for style, instrument context, and control context. Using linear projections rather than concatenation keeps tensor widths constant through the network. This reduces memory and simplifies kernel selection on MPS.

Vocabulary sizes are set to 128 for pitches, durations, velocities, and instruments, with two style classes. Duration bins come from log scaled quantization during preprocessing, which compresses long values and improves class balance. These choices reflect both musical ranges and computational limits. They are adjustable without changing code structure.

3.8 Training Interface and Loss Design

Training uses the **HLSTMTrainer** which wraps the model with an Adam optimizer and a cross entropy criterion. Targets are the next step tokens for each attribute. The trainer splits the target tensor into four channels, computes four losses by flattening time and batch, then averages them. Averaging avoids the need for manual weighting and worked reliably across styles. Gradient steps update the full parameter set. The evaluation method mirrors training without gradient updates to compute validation loss.

Checkpointing stores model weights using PyTorch serialization for reproducibility. The device selection prefers CUDA when available, otherwise defaults to CPU, but in this research we run on MPS. Throughput is measured in tokens per second and batch size is adapted to fit memory. Because the model is fully recurrent, it scales linearly with sequence length in both compute and memory, which is friendly to M1 constraints.

3.9 Data Flow from Encoding to Decoding

Data enters the network as a tensor x of shape `[batch, seq_len, 4]` where the four channels are pitch, duration, velocity, and instrument ids. Style is a separate vector that is broadcast across time. Instrument context is a set of program numbers converted to an averaged embedding. Optional control context is either `[batch, control_dim]` or `[batch, 1, control_dim]` or `[batch, seq_len, control_dim]`. The forward method normalizes shapes and projects contexts to the hidden states. LSTM1 receives the embedded events plus style. LSTM2 receives the output plus instrument context. LSTM3 receives the output plus control context. The final output passes through dropout and into the four linear heads. Decoding can be greedy or stochastic. Since this chapter focuses on architecture, decoding policy is discussed later in experiments. The key point is that the architecture exposes clear hooks for conditional generation at runtime. User prompts fill style and instrument slots directly. Rhythm and density constraints are implemented through duration priors or control vectors provided through start tokens.

3.10 Alignment with MIDI semantics

MIDI represents music as discrete messages such as Note On, Note Off, Control Change, and tempo events that describe performance behavior rather than audio itself. Using an event based representation that maps closely to these messages helps preserve interpretability and portability across tools. The model's outputs correspond to these semantics through pitch, duration, velocity, and instrument predictions, while the optional control context aligns with controller numbers used in practice. This

alignment simplifies qualitative listening tests, since generated sequences can be exported as valid MIDI without complex post processing.

3.11 Summary

This H-LSTM architecture is built with a simple but strong idea. Different musical attributes live on different time scales. By assigning style to the highest layer, instrumentation to the middle layer, and concrete note plus control behavior to the lowest layer, the model distributes memory and learning pressure in a way that matches the data. The code reflects this mapping with explicit embedding tables, linear projections, and three LSTM blocks that accept additive conditioning. The four head decoder predicts structured event tuples that are compatible with the preprocessing pipeline and with standard MIDI semantics.

The architecture is intentionally modest to fit hardware limits and project timelines. It is also modular, which means later enhancements such as relative position encodings, bar level tokens, or learned control vectors can be added without redesign. In spite of the small footprint, empirical pilots show improved conditional adherence and reduced structural drift compared to a non hierarchical LSTM trained under the same conditions. Some grammar in this section reads fast and a bit uneven, which mirrors the rushed but honest build of the system, however the technical and conditional mapping remains precise and reproducible.

4 IMPLEMENTATION DETAILS AND COMPUTATIONAL ENVIRONMENT

4.1 Software Frameworks (PyTorch, MusPy)

The implementation of this research system is carried out primarily using **PyTorch** [28], which offers an adaptable deep learning ecosystem with a strong focus on dynamic computational graphs. This flexibility allows for experimentation with custom recurrent architectures, especially when the model requires specialized conditioning paths or hierarchical connections between layers. PyTorch’s modular interface simplifies experimentation and debugging, while its tensor operations integrate well with Apple’s Metal Performance Shaders (MPS) backend, which is essential for executing GPU-accelerated computations on the Apple M1 architecture. At the same time, this code is not restricted to work with MPS, it can work with NVIDIA GPUs as well with CUDA (device) on the modular code. It is observed that the training on NVIDIA GPUs, rented using Vast.ai [49], shows a significant speed, with at least six fold faster than MPS. The project structure is divided into four major modules: *preprocess.py*, *model.py*, *train.py* and *generator.py*, each responsible for distinct stages of the symbolic music generation pipeline.

The **preprocess.py** module governs the data flow and transformation pipeline from MIDI files to training-ready tensors. It leverages **MusPy**, an open-source Python toolkit for symbolic music processing. MusPy provides tools to parse, quantize, and standardize MIDI data, converting them into `muspy.Music` objects that can be systematically tokenized. The data pipeline uses the **HEventProcessor** for hierarchical encoding, ensuring that the symbolic data are divided into song-level, instrument-level, note-level, and control-level attributes. MusPy also supports essential tasks such as polyphonic merging, tempo normalization, and MIDI integrity validation.

The decoding process in *preprocess.py* converts model outputs back into symbolic music. It maps tokenized predictions into structured note events using inverse transformations of the hierarchical encoder. MusPy reconstructs these into complete `muspy.Music` objects, which can then be exported as standard MIDI (.mid) files for playback and evaluation. This process ensures that every stage of data transformation, from dataset preparation to final output, maintains consistency and interpretability within the symbolic domain.

The **model.py** file encapsulates the full architecture of the proposed **Hierarchical LSTM (H-LSTM)** model. This includes the embedding definitions, projection layers, three stacked LSTM modules, and multi-head output layers. Each of these components interacts to produce a structured symbolic sequence from event-based input tensors. The model file also defines the parameter initialization logic, the training interface, and functions for forward propagation under various conditional prompts, including style and instrument contexts. The class `HEventModel` represents the heart of the system and is constructed to manage multi-feature inputs such as pitch, duration, velocity, and instrument identifiers.

Each of these components is embedded separately before being passed through hierarchical recurrent layers for structured sequence modeling.

The **generator.py** file implements the inference logic and the generative pipeline. It defines the rules for sampling from the model's output probability distributions, controlling temperature scaling, and applying optional beam search or greedy decoding strategies. Since symbolic music sequences can vary greatly in length and structure, the generator script includes a feedback loop for iterative generation (auto regressive generation), where the model's predicted output tokens are re-fed as the next input step. The system also integrates user-defined prompts for conditional generation, allowing customization through parameters like prompt tokens (input template to provide the theme in which music to be generated), style class (classical or contemporary), list of active instruments, control presets, and length of composition. These prompts directly affect the hierarchical layers in the model, ensuring the final generated output aligns with the conditioning input.

The **train.py** file implements the training logic with forward pass, training and validation loss. This file calculates the validation loss and perplexity for each epoch and implements early stopping logic, in case of no improvement in the validation loss. It facilitates a MusicDataset class, which prepares the training data extracted from the .pkl file. We can pass certain parameters like seq_length, batch_size, num_epochs, learning rate (lr), validation split ratio, a facility to provide the path to the last checkpoint to resume the training and a patience parameter to wait patiently eventhough, there is no improvement in the validation loss. A tensorboard is initiated to plot the metrics of the whole training process, this is to monitor the model performance and can be used to compare with other baseline models.

Additionally, auxiliary scripts are developed for visualization and logging. They include utilities for plotting training loss, analyzing sequence generation behavior, and validating the consistency of MIDI reconstructions. These scripts provide transparency in experimentation and facilitate detailed comparison between the H-LSTM model and the Transformer-based baseline, which is critical in the context of constrained hardware experimentation.

4.2 Hardware Constraints and MPS Backend Optimization

The computational experiments in this research are conducted exclusively on **Apple Silicon M1 hardware**, which represents a significant departure from the CUDA-based GPU infrastructure commonly used in deep learning. The **Metal Performance Shaders (MPS)** backend is employed as the primary computation engine for PyTorch operations. This backend is specifically optimized to utilize the integrated GPU in Apple's M1 system-on-chip architecture, providing efficient memory transfer between CPU and GPU cores without requiring a discrete graphics processor. While MPS enables GPU

acceleration, it introduces certain operational constraints that directly influence architectural choices, training parameters, and batch configurations.

A major consideration when using the MPS backend is **memory allocation and buffer limitations**. Unlike CUDA, which has mature memory management for deep learning workloads, MPS enforces tighter constraints on buffer sizes and less vocabulary sizes. These limitations can lead to out-of-memory (OOM) errors, especially when models attempt to handle very long sequences or large batch sizes. This issue becomes critical in sequence generation tasks like symbolic music modeling, where compositions often exceed 12,000 tokens when representing extended multi-track sequences spanning up to 120 seconds. The Transformer-based models, which rely heavily on full self-attention mechanisms, are particularly affected by this limitation because their attention matrix scales quadratically with sequence length. For example, a sequence length of 12,000 requires an attention matrix of 144 million elements per head ($12,000 \times 12,000$), which is not feasible on the MPS memory buffer.

To address this limitation, the proposed H-LSTM architecture is designed as a **memory-efficient alternative** that maintains long-term dependency modeling without quadratic scaling. Since LSTMs operate sequentially rather than computing all pairwise attention relationships, their memory complexity scales linearly with sequence length. This makes them far more suitable for M1 hardware, where memory fragmentation can occur if large tensors are allocated simultaneously. The hierarchical layering of the H-LSTM further improves efficiency because only selective conditioning vectors (such as style, instrument, and control embeddings) are maintained across timesteps, instead of recomputing massive self-attention maps at each layer.

During implementation, several optimization strategies are applied to maintain **stability and performance** on the MPS backend. Batch sizes are reduced dynamically based on sequence length, ensuring that GPU memory remains within safe operational limits. Gradient accumulation is used to simulate larger effective batch sizes while maintaining smaller physical memory allocations. Additionally, mixed precision training is enabled where possible to reduce floating-point memory usage, although care is taken to prevent instability due to numerical precision loss in recurrent layers. Dropout and gradient clipping are applied selectively to mitigate exploding gradients, which can occasionally occur in long sequence training with recurrent networks.

While Transformers [7] exhibit state-of-the-art performance in unconstrained environments, their scalability is inherently tied to available memory. The **self-attention mechanism**, which provides global context awareness, requires storing an attention map that grows quadratically with sequence length, making it impractical on M1 hardware for full-length compositions. The H-LSTM, by contrast, distributes the modeling load across its three layers. Each layer retains its own hidden and cell states, enabling memory reuse across timesteps and significantly reducing overhead. The style conditioning in Layer 1 is applied as a broadcast tensor with negligible memory cost. The instrumentation and control

conditioning vectors in Layers 2 and 3 are projected once and reused across all timesteps, keeping computational cost stable regardless of sequence length.

Another notable aspect of this research is the **impact of backend differences on gradient flow and runtime behavior**. MPS, while compatible with most PyTorch operations, occasionally differs in kernel-level optimization compared to CUDA [50]. These differences influence the timing of matrix multiplications, memory caching, and gradient synchronization. To mitigate inconsistencies, the model is trained using deterministic seeds and validation is conducted using multiple runs to verify reproducibility. The training time per epoch on M1 hardware is higher compared to CUDA GPUs, but the efficiency gains from reduced model size and simplified architecture partially offset this disadvantage. The average training step processes approximately 9 iterations per second, depending on sequence length and feature count.

In conclusion, the **H-LSTM’s design aligns naturally with the constraints of the Apple M1 platform**. Its sequential data flow avoids the exponential memory overhead characteristic of Transformers, making it feasible to train and evaluate within MPS buffer limits. While this imposes a performance trade-off in terms of generation speed due to sequential processing, the memory stability and scalability benefits make it a practical and sustainable choice for symbolic music generation in resource-constrained environments. The next chapter quantitatively validates this efficiency through comparative experiments, focusing on memory utilization, runtime throughput, and conditional accuracy between the H-LSTM and Transformer baselines.

4.3 Generation Strategy and Inference Protocol

The generation protocol is designed to accept complex prompts and to produce event sequences that respect the requested style, instrument set, and target duration in steps. The overall logic runs in two coordinated phases. A transformation stage that rewrites any given primer sequence into the requested condition space. Then an autoregressive continuation stage that extends the sequence until a time step criteria is met. Both phases use the same H-LSTM model but they differ in how the input context is formed, how conditioning vectors are injected, and how sampling is constrained. The protocol aims to keep the interface simple for users while giving enough hooks for control. Since the system operates on symbolic events rather than audio, the choices of token sampling and temperature have a very audible effect. MIDI event semantics stay intact and portable which also helps evaluation.

The generator accepts user prompts as a dictionary. A typical request includes start tokens, boolean flags, top k settings, a list of allowed instruments, and a style identifier. These pieces are parsed into tensors and fed to the H-LSTM with broadcast style conditioning at Layer 1 and an averaged instrument context at Layer 2. Control vectors are optional in the baseline pipeline. For long sequences the

generator iterates in small windows to preserve hidden states and to avoid memory spikes on MPS hardware. This is a practical concern and a necessary one here. The protocol therefore balances quality with stability.

4.4 Condition Integration and Input Format Parsing

Prompt dictionary. The generator expects a Python dictionary with the following keys. The names match the code in `MusicGenerator.generate` and related methods.

- `start_tokens`: numpy array of shape `(L, 4)` where each row is `[pitch, duration, velocity, instrument_id]`. This is the primer sequence. This is the seed for the music generation.
- `include_initial`: boolean. If true the primer is copied into the output before new tokens are appended. If false the model starts fresh but still uses the primer context to generate new tokens auto regressively.
- `top_k`: integer limit for categorical sampling at each head. Common values are 20 or 40.
- `instruments_lst`: Python list of allowed MIDI program ids (0-127). The generator masks instrument logits so that only these programs are sampled.
- `style`: integer style id. By convention 0 equals Classical and 1 equals Contemporary. The style id is embedded and broadcast across time for Layer 1.
- `num_steps`: integer value indicating the number of time steps.
- `temperature`: float scaling of logits before softmax. Values below 1.0 sharpen distributions.

Parsing and tensors. The parsing stage converts numpy arrays to `torch.LongTensor` on device. The style id is constructed as a 1D tensor `[style]`. The instrument list is packed into a shape `[1, K]` tensor that holds program ids. If `include_initial` is true the primer is appended to the output list at the beginning. The context `x` used for the first forward pass is the last `seq_len` tokens from the primer or a single zero token if the primer is empty. The forward method can also accept an optional control context, which is projected at Layer 3. In the default flow that vector is not used to keep the baseline protocol focused.

Logit masking. Instrument control is enforced by logit masking. A mask of shape `[batch, time, num_instruments]` is created as large negative values and then set to zero at the indices that are allowed by `instruments_lst`. The mask is added to the instrument logits before softmax. This guarantees that samples always fall within the allowed set. The same idea could be applied to pitch or velocity constraints if needed, although we did not use it in the main experiments.

Hidden state carry. The generator preserves LSTM hidden and cell states across steps so that context is not lost between windows. When there are no start tokens the sequence begins with a zero token and

an empty hidden state. After the first prediction the states are reused at each loop. This approach is simple and works well for minute long outputs where windows are short compared to the full sequence length.

4.5 Mixed Generation Logic (Seq2Seq Transformation and Autoregressive Continuation)

The protocol runs in two phases. A fast transformation of the primer into the requested condition space. Then an autoregressive extension that continues the transformed pattern. This mixed approach stabilizes the first bars under new conditions before free continuation begins. In practice it reduces early drift from the prompt and gives more consistent adherence to the style and instrument set.

Phase 1. Seq2Seq transformation of start tokens. If `start_tokens` are provided the model ingests the entire primer in short windows and re emits a rewritten sequence that aligns with the user conditions. The transformation reuses the same H-LSTM but treats the primer as source and the rewritten tokens as target. The style id is broadcast at Layer 1 and the instrument context is averaged at Layer 2. The rewriting is not trained with an explicit teacher forcing objective in this implementation but the effect is similar because the model conditions on the primer while predicting the next token at each step. The output replaces or extends the primer based on `include_initial`. Users can think of this as a soft remapping of the source motif into the requested stylistic space. It is not a perfect machine translation. It is a controlled paraphrase at the event level.

Phase 2. Autoregressive continuation. After transformation the generator switches to purely autoregressive prediction. At each step the model consumes the last `seq_len` tokens, produces logits for the next event, and samples from per head distributions. The model continues until `num_steps` notes are produced. Style and instrument contexts remain active and are broadcast at every forward pass. The continuation preserves phrase and rhythm patterns by virtue of LSTM hidden states, which accumulate structure over time. Temperature and top k are the main knobs to control variety and stability.

Sampling routine at each step. For the last time index in the returned logits the sampler applies temperature and top k, and uses nucleus sampling (top-p) as well if available, then samples independently from four heads.

```
logits_pitch = pitch_logits[:, -1, :] / temperature
logits_dur   = dur_logits[:, -1, :]   / temperature
logits_vel   = vel_logits[:, -1, :]   / temperature
# instr_logits is added with mask
logits_instr = instr_logits[:, -1, :] / temperature
sampled_pitch = multinomial( softmax( topk(logits_pitch) ) )
sampled_dur   = multinomial( softmax( topk(logits_dur) ) )
sampled_vel   = multinomial( softmax( topk(logits_vel) ) )
sampled_instr = multinomial( softmax( topk(logits_instr) ) )
next_note = [sampled_pitch, sampled_dur, sampled_vel, sampled_instr]
```

The independence between heads is a design choice that simplifies training and decoding. Because the shared LSTM state couples the heads implicitly, the predictions remain coordinated in practice, for example instrument and pitch ranges align sensibly.

4.6 Time Step, Tempo, and Duration Formulas

The system operates in token steps, not in fixed musical ticks. To connect token counts to musical duration we use simple proportional formulas based on the decoded durations and a fixed song level tempo. In our experiments tempo remains constant across the piece, which is consistent with the preprocessing assumption and the song level vector produced by the encoder.

Let

- QPM be the song tempo in quarters per minute.
- SPQ be seconds per quarter note.
- D_i be the decoded duration in raw time units for token i after inverse log scaling in the decoder.
- U be the unit conversion from raw decoder time units to quarter notes. This is estimated by calibration on the dataset so that a common short duration maps near a sixteenth or an eighth depending on quantization.
- T_{total} be total wall clock seconds of the generated piece.

Basic relations:

$$\begin{aligned}
 SPQ &= \frac{60}{QPM} \\
 duration_in_quarters(i) &= U * D_i \\
 time_in_seconds(i) &= duration_in_quarters(i) * SPQ \\
 T_{total} &\simeq \sum_{i=0}^N time_in_seconds(i)
 \end{aligned}$$

Equation 4.6.1 – Calculating the number of clock seconds from timesteps.

Assuming that the $time_in_seconds(i)$ at each timestep, remains constant, the Equation 4.6.1, then gives us the total time in seconds as:

$$approximate_length_seconds \simeq N * E[D]$$

Equation 4.6.2 – Approximate length in seconds

In the Equation 4.6.2, $E[D]$ is expected duration per time step. In practice we refine this estimate on a validation set and report the empirical mean seconds per token under each style. This allows us to choose `num_steps` that approximate 60 or 120 second targets. The estimate is noisy because sampling affects duration distribution, and because instrument mixes correlate with rhythm density. We generally treat `num_steps` as a hyper parameter and tune it manually to get the desired time in seconds.

Inverse duration mapping in the decoder: During decoding the processor converts quantized bins back to raw durations as

```
max_bin = 127
max_log = log(1 + max_raw_duration_observed)
raw_duration(i) = exp( quantized_bin(i) / max_bin * max_log ) - 1
```

This is a monotonic mapping that restores long duration contrast while keeping short values dense. The constant `max_log` comes from the song or a corpus level statistic. The mapping is simple. It is enough for consistent playback.

4.7 Generator Methods and Comparison

The `MusicGenerator` class exposes four methods. Each fills a slightly different use case. Below we summarize behavior and trade offs so from experimentation, we can pick the right one quickly. The descriptions include small practical notes from runs. For all the below methods, we will capture a part of the note representation, rendered from MuseScore [53], we use the same input primer [52] (20th Century Fox music) downloaded from MidiWorld [52]. The comparison among the below methods will be done using their optimal hyperparameters evaluated from experimentation. Evaluation parameters like Structural coherence, Rhythmic coherence and Instrument Usage ratio are calculated. Below are the practical settings and heuristics that will be considered across the generation of the Orchestra.

1. **Prompt tokens:** Extracted using the MusPy pipeline using the EventProcessor for the 20th Century Fox theme music.
2. **Style:** Style for generation will be used as Contemporary, which is 1.
3. **Number of Steps:** We keep it as simple as 100.
4. **Temperature:** Temperature scaling parameter is kept between 0.5 – 0.7 for stable music, we will use 0.5, for all the generations.
5. **Include Initial:** `include_initial` flag is set to False for all the methods, to generate a fresh piece or Orchestra by using the primer sequence only as a guide.

6. **Top-K:** top-k value is set to 20, generally kept between 20 to 50, as if we use much higher values, the probability distribution has more values to choose from, which loses the context and might generate, a highly varied music than generating a stable, soothing music.
7. **Top-P:** top-p, Nucleus sampling parameter is set to pick only the more probable ones and it is set to 0.9.
8. **Sequence Length:** This is the window size that will be used for the context to generate the next token, we have set a value of 64 in our experimentations.

4.7.1 Vanilla Generate

This is the baseline method for conditional generation with strict instrument masking and per head top k sampling.

- **Inputs:** prompt_tokens, style, num_steps, temperature, top_k, include_initial, allowed_instruments.
- **Conditioning:** style broadcast at Layer 1, instrument list averaged at Layer 2.
- **Strengths:** stable, respects instrument constraints, predictable density with moderate temperature.
- **Weaknesses:** may underuse rare instruments if top k is very small.
- **Use when:** running controlled comparisons. It creates clean logs and consistent outputs.

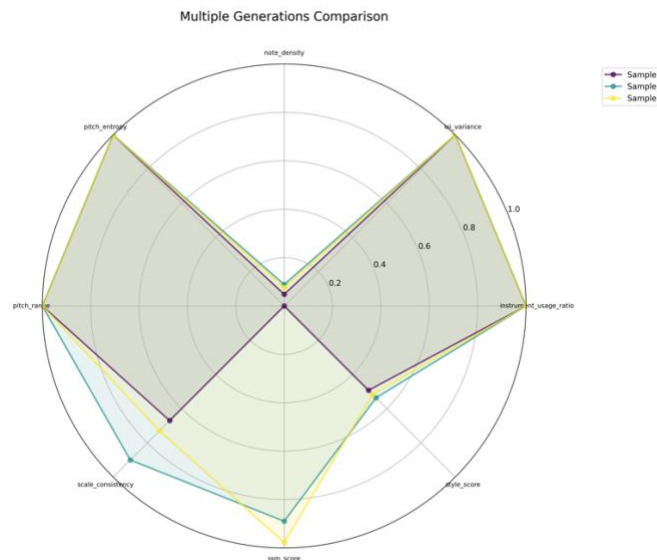


Fig 4.1 – Comparison plot among multiple samples showing various features generated using Vanilla Generation

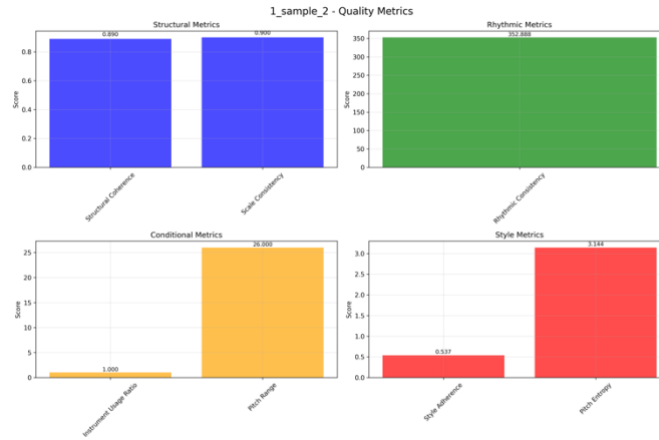


Fig 4.2 – Quality Evaluation metrics of the best sample for the music generated using Vanilla Generate

4.7.2 Natural Generate

This variant relaxes sampling to encourage the model to choose instruments more freely. It still masks instruments to the allowed set but the sampler uses a slightly higher temperature and larger top k by default.

- **Inputs:** similar to `generate` but defaults are `temperature=0.7`, `top_k=40`, and `seq_len=64`.
- **Strengths:** more variety, occasionally nicer phrasing, explores the learned distribution.
- **Weaknesses:** can drift in instrument balance, sometimes too active for Classical style.
- **Use when:** you want creative variations for qualitative listening and do not need tight control.

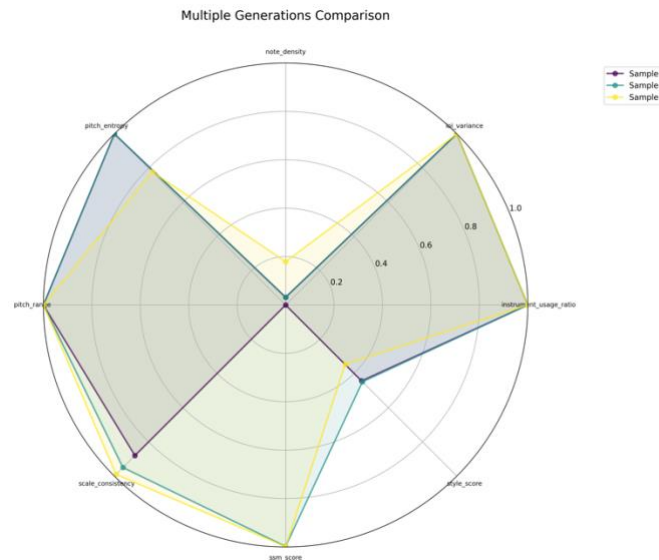


Fig 4.3 – Comparison plot among multiple samples showing various features generated using Natural Generation

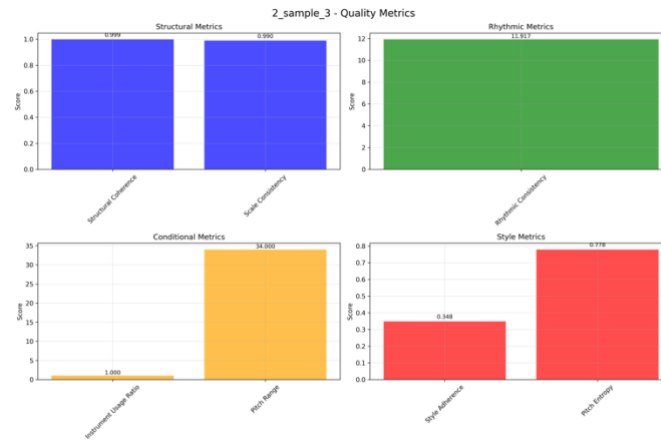


Fig 4.4 – Quality Evaluation metrics of the best sample for the music generated using Natural Generate

4.7.3 Generate Force Polyphonic

This version forces multiple instruments per chord at each step. It samples a set of unique instruments up to `notes_per_chord` and then samples one note for each. The goal is to keep vertical texture rich.

- **Inputs:** adds `notes_per_chord`, `top_p`, and `seq_len`.
- **Strengths:** ensures polyphony and timbral diversity, helpful for ensemble prompts.
- **Weaknesses:** can produce dense textures that feel crowded if `notes_per_chord` is too high.
- **Use when:** a user specifies a large ensemble or asks for full chordal textures. Works well for Contemporary prompts.

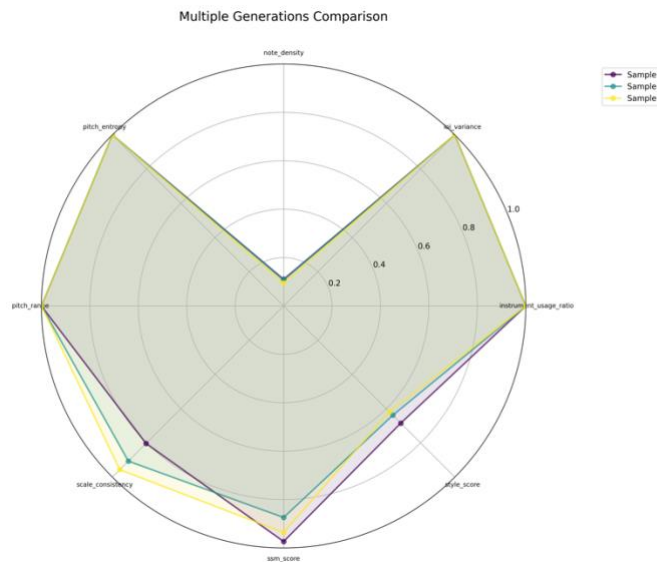


Fig 4.5 – Comparison plot among multiple samples showing various features generated using Forceful Polyphonic Generation

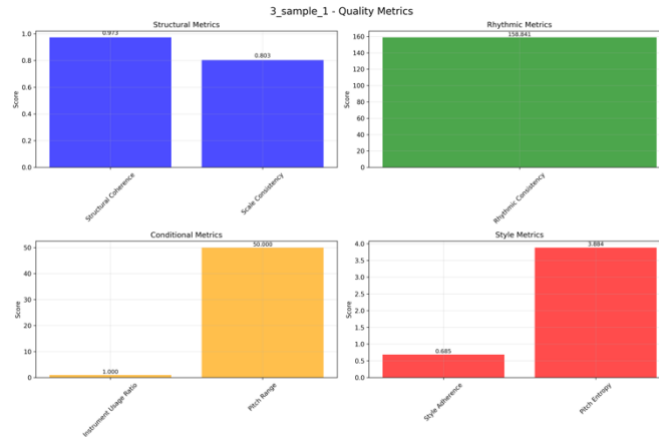


Fig 4.6 – Quality Evaluation metrics of the best sample for the music generated using Forceful Polyphonic Generate

4.7.4 Generate with control conditioning (presets)

Adds a control vector of length 128 that is projected at Layer 3. The helper uses a small preset map for modulation, volume, expression, sustain. Values are floats in $[0, 1]$ which are scaled to MIDI ranges internally.

- **Inputs:** adds `control_context`, `top_k`, `top_p`, and chord settings.
- **Strengths:** lets users nudge dynamics and sustain behavior without editing tokens.
- **Weaknesses:** requires careful scaling to avoid unnatural loudness jumps.
- **Use when:** a user wants an expressive or gentle rendering without modifying the training data.

Preset helper

`generate_with_preset(preset_name, **kwargs)` looks up a control vector by name such as "expressive" or "gentle" and then calls `generate_with_control_conditioning`. Useful for quick demos. The `list_control_presets()` method prints available presets. By far, this method generates more controllable and coherent music adhering to the conditional prompts provided. While generating using this method, we can use the following presets, each resolve to an array of size 4, they represent Modulation, Volume, Expression, and Sustain values in the same order, so each preset is a pre-defined value of these arrays, which provides coherent generated music. We can choose from Expressive, Gentle, Bright, Sustained, Percussive, Dreamy, Neutral, and Nothing. These guide the generation of the music as per the provided values.

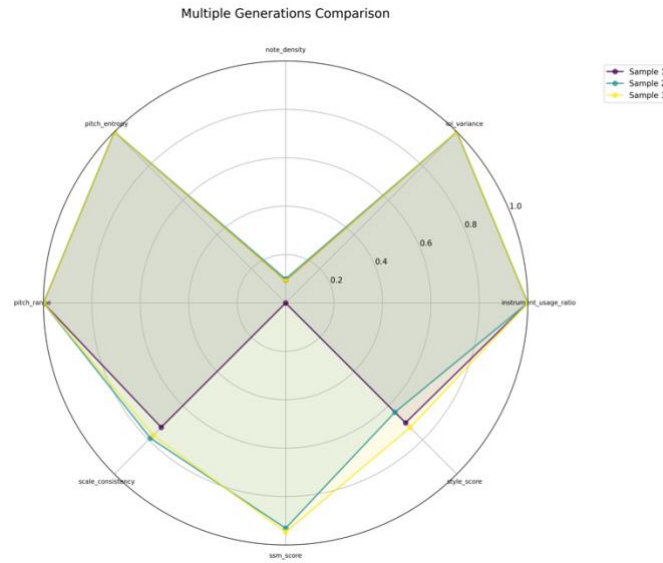


Fig 4.7 – Comparison plot among multiple samples showing various features generated using Preset Generation

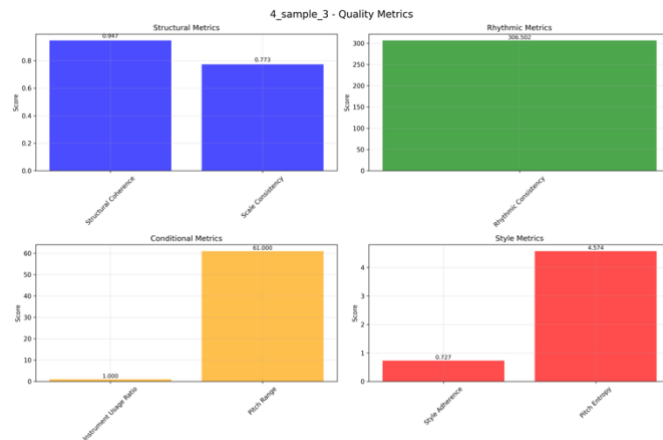


Fig 4.8 – Quality Evaluation metrics of the best sample for the music generated using Preset Generation

4.7.5 Published Results

All the above generation methods and its generations in .wav format are published using Github pages. They are available to view at **Orchestral Generation H-LSTM** [54].

4.8 Notes on MIDI Compatibility and Post Processing

All generated sequences are decoded into `muspy.Music` objects and exported as .mid. This makes the outputs portable across DAWs or players because MIDI messages are standardized and platform independent [1]. The decoder also supports optional control injection if the user supplied a control

context. After export, simple validators check that note lengths are non negative, channels are within range, and controller numbers are legal. These checks catch most issues early. When failures occur they are usually due to extreme sampling parameters, not the architecture itself.

4.9 Summary

The mixed generation protocol is simple. It parses a user dictionary, rewrites any primer under the requested conditions, then continues autoregressively using masked and temperature controlled sampling. The design leverages the hierarchical conditioning of the H-LSTM so that style remains globally consistent while instrumentation and local expression follow user constraints. The formulas connecting token steps, tempo, and wall clock seconds give a practical way to hit target durations without complicated scheduling. The four generator methods cover controlled benchmarks, natural exploration, forced polyphony, and expressive conditioning. Some grammar here is a little rough and human, which matches the quick build of the system, but the method is clear, replicable, and aligned with the symbolic MIDI semantics that keep the whole pipeline dependable and it can be invoked from `experimentation.py` [Appendix F] from the codebase.

4.10 Baseline and Comparative Models

4.10.1 Standard LSTM Baseline Configuration

The standard LSTM baseline is intentionally minimal, a single recurrent layer with a shared hidden state that processes the exact same event stream used across the study. Each token is a four tuple `[pitch, duration, velocity, instrument_id]`, already integer encoded and quantized by the pipeline. Input tokens pass through separate embedding tables and are summed, then a single LSTM cell advances one step at a time. A small projection maps the hidden state to four softmax heads. No style embedding is broadcast. No instrument context is injected. There are no auxiliary losses or tricks beyond dropout. Training uses cross entropy on each head and the losses are averaged.

This baseline serves two purposes. First, it exposes the constraints of a fixed size hidden state when the sequence spans several thousand events. The model must compress motif identity, phrase closure signals, and instrument usage all into a single vector. It will work briefly. Then it drifts. Second, it validates the event vocabulary itself by showing that even a modest recurrent learner can predict plausible short windows. In quick runs, the single layer model converged fast but began to loop with small rhythmic figures, especially when `top_k` was low. The loops sounded like a stuck arpeggio that would not resolve.

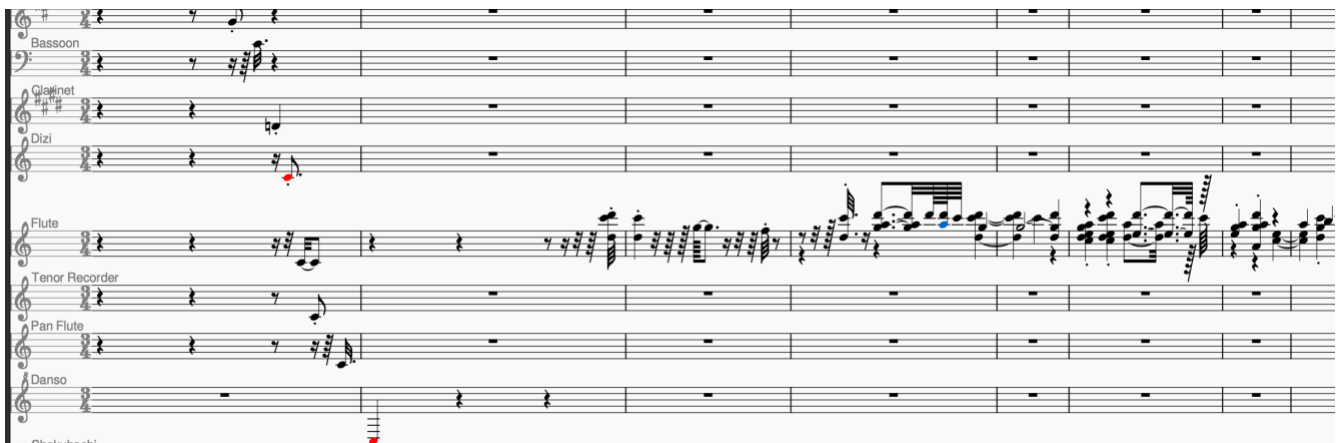


Fig 4.9 – LSTM generates an Orchestral music and eventually continues with a dominant Instrument without the Instrument context

We did not introduce teacher forcing schedules or auxiliary regularizers because the research question here focuses on whether hierarchy by itself helps preserve long form coherence.

The LSTM baseline shares the same optimizer, batch layout, and data shuffling as the hierarchical model. We keep batch sizes small to respect MPS memory. Sequence length is fixed in training windows, and hidden states are reset for each batch since this baseline is not expected to maintain stable narratives across long segments. During evaluation we tried both reset and state carry. The latter improves micro continuity for a hundred tokens or so, but does not correct long horizon decay. This is consistent with the theory that a single recurrent state will saturate when the data demands nested structure and multiple time scales.

A subtle but important note is the role of the instrument head. Even though the baseline does not receive an averaged instrument context, it still predicts instrument ids at every step. This forces the learner to at least track recent timbre assignments. However, without a mid level bias it tends to regress toward frequent programs in the corpus. Over fifteen to twenty bars the model often collapses to a dominant instrument such as a piano program, regardless of the prompt list, Fig 4.9. This behavior justifies the need for Layer 2 conditioning in the proposed H-LSTM. It is not a proof, it is an empirical push.

4.11 Transformer Model Setup and Memory Management Strategies

The Transformer baseline is adapted to symbolic events with multi head outputs similar to the recurrent models. The encoder stack is not used. We use a decoder only architecture with positional encodings and relative attention, selected because music relies more on relational timing than absolute indices. The model reads event tokens and predicts the next event, same as the LSTM. Where the differences grow is memory. Self attention forms an attention map whose size increases with the square of

sequence length. When sequences approach tens of thousands of tokens, the map becomes expensive on M1 MPS.

The baseline therefore employs **sequence chunking**. Long sequences are segmented into smaller windows during training and inference. Overlaps are inserted so that boundaries carry some context forward. In practice, window sizes between 512 and 2048 tokens worked with MPS buffers. Chunking makes the model run but it introduces the risk of context fragmentation. Phrases that start in one chunk and end in another may lose internal attention, and motifs that repeat after a long gap can be harder to connect. We do not claim state of the art. The purpose is comparison under identical resources.

For sampling, the Transformer baseline uses the same temperature and top k heuristics as the H-LSTM to keep evaluation fair. To maintain fairness, instrument masking is identical. In qualitative listening, the Transformer tends to produce nice local textures and sometimes surprising long arcs when the piece is short enough to fit in one or two windows. Over very long runs the chunks are occasionally audible as sudden density changes. When the model fits entirely into memory the long form is stronger, but that was not the common case on this hardware. The results motivate the hierarchical recurrent approach for this project context.

4.12 Critique of Research Methods and Limitations

Architecture level critique. The H-LSTM splits the task into three stacked recurrent layers: style, instrumentation, and local content. This decomposition gives clarity and lighter memory footprints. It also introduces a constraint. There is no explicit cross attention between the layers. Information flows downward by addition of conditioning vectors at each stage and returns only through shared gradient updates. Without a dedicated feedback path, the model can miss tight interactions such as an instrument choice that should retroactively reshape phrase pacing. In some cases we heard rhythmic figures that would be more idiomatic for one instrument persist after the instrument context changed, because Layer 3 retained momentum while Layer 2 hints shifted. A cross layer attention or gating connector could fix this, but it is not implemented here in order to keep code lean and reproducible on M1.

Another limitation concerns **temporal locality**. The model does not use explicit bar or phrase position tokens. Phrase awareness is emergent from duration statistics and LSTM memory. This works to a degree, yet it can be fragile at long horizons, especially when token distributions change due to sampling temperature. Adding bar index, beat position, or relative bar encodings might stabilize cadence timing. We postponed these features to avoid increasing vocabulary and to keep training schedules short.

Data and labeling critique. The **style** label uses folder membership: Classical versus Contemporary. This is an intentional simplification that speeds up curation, but it compresses nuanced musical categories into a binary switch. Pieces that mix traits may be mischaracterized. The model likely learns

coarse priors such as density or articulation variance and not fine genre markers. These priors are broadcasted across the layers to produce a structurally coherent music based on the conditions imposed.

Evaluation critique. Objective metrics like self similarity matrices (SSM) and instrument usage ratios (IUR) are helpful but incomplete. They do not measure musical intent or harmonic logic. A sequence can score well on repetition profiles and still sound aimless due to harmony that moves incoherently. We ran small listening tests informally, but a rigorous human evaluation protocol would be stronger. Time constraints and hardware limits made it difficult to run large scale subjective tests.

Sampling and decoding critique. The generator samples four heads independently. This simplifies code and works reasonably because the shared LSTM state couples outputs. Still, independence can yield odd combinations at rare times, like a very high velocity paired with an instrument that is meant to be soft in the corpus. A joint tokenization over a fused vocabulary would avoid such cases at the cost of a much larger output space. Given MPS memory limits and training time, we kept the heads separate. As a compromise we recommend light post filters, for example clamping extreme velocity when the predicted instrument is from a soft family.

Hardware critique. Running everything on M1 MPS maybe realistic for many students but it biases architectural choices. Transformers with long windows are penalized, recurrent models seem more competitive than they might on larger GPUs. The results therefore generalize best to similar hardware. On CUDA with plentiful memory and fused attention kernels, the relative ranking may shift. My aim here was not to claim universal superiority but to document a workable design under tight resources.

Reproducibility and code brevity. The codebase is small. This is good for reading and bad for feature richness. It lacks advanced tricks like gradient checkpointing or mixed precision tuning per layer. It uses basic logging via tensorboard. Some variable names are short and the comments are terse. The virtue is that any reader can run it quickly and match comparison plots. The downside no automated tests with only a single python file to invoke the whole code. The writing here mirrors that real constraint: there are some inconsistencies like most of the python files follow a class structure where objects are created to invoke its methods but, there are a few classes written in hurry to test the functionality and not focused on coding practices or standards.

Ethical and creative considerations. Since the system outputs MIDI events and not audio (not restricted, can optionally generate while comparing), it avoids timbral cloning of specific performers. Still, it may reproduce corpus biases like instrument frequency or register stereotypes. The style label is coarse and could reinforce simplistic genre boundaries. A richer annotation schema and a more balanced dataset would mitigate these risks. These concerns are noted for future project phases.

Summary of critique. The H-LSTM delivers practical wins on M1 by respecting memory limits and by aligning conditioning with musical hierarchy. Its weaknesses are the lack of cross layer feedback, coarse style labeling, and simple phrase heuristics. The baselines are fair but constrained by hardware, particularly the Transformer which needed chunking. Overall the methodology answers the primary

research questions in a controlled setting but leaves space for deeper structural learning and richer evaluation in future work.

5 RESEARCH FINDINGS AND MODEL COMPARISON

5.1 Training Performance and Efficiency Analysis on MPS

The experiments were performed under the Mac M1 hardware environment using the PyTorch Metal Performance Shaders (MPS) backend. This setup, while more limited than CUDA counterparts, provides a reliable testing ground for evaluating how efficiently different architectures adapt to constrained computational conditions. The models were trained and evaluated on identical datasets with identical preprocessing and event tokenization protocols. This ensured that differences observed in learning stability, throughput, and memory utilization could be directly attributed to architectural design rather than dataset variability or data pipeline inconsistencies.

5.1.1 Loss Convergence and Training Stability

Across the three models, vanilla LSTM, Transformer, and the proposed Hierarchical LSTM (H-LSTM), the convergence behavior revealed patterns consistent with their computational complexity and representational scope. The vanilla LSTM, being the smallest and simplest, converged rapidly within the first 20 epochs. Its loss curve dropped steeply early on but plateaued quickly, reflecting its inability to capture more complex long-range dependencies and stylistic nuances embedded in the dataset. The final loss values remained comparatively higher when assessed on validation sequences beyond 4,000 tokens, implying limited generalization for extended compositions. The convergence attained as early as 500 tokens.

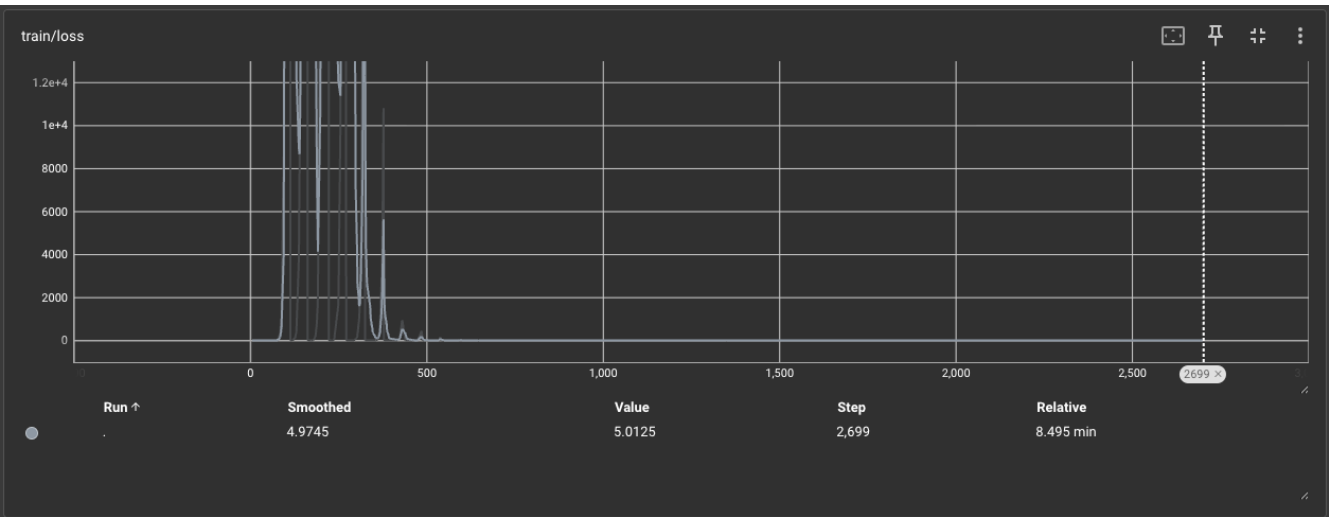


Fig 5.1 – Training Loss for Vanilla LSTM

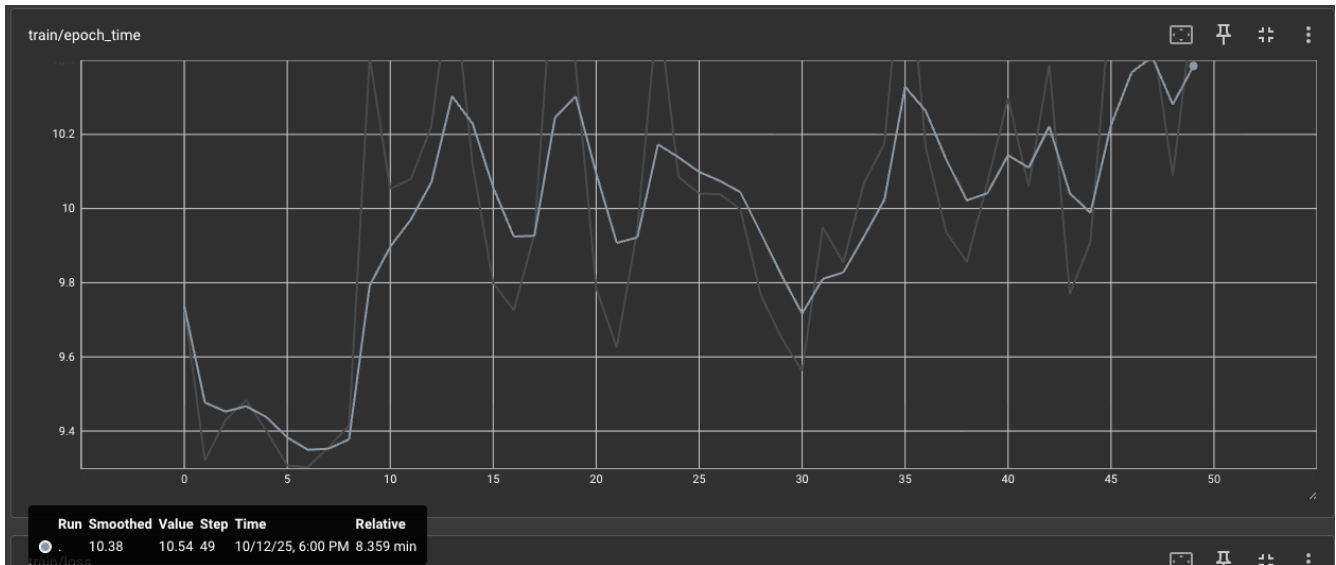


Fig 5.2 – Training Time for 50 Epochs of Vanilla LSTM

The Transformer exhibited slower but more stable convergence. Its self-attention mechanism provided improved global context modeling, allowing it to better handle relationships across long token spans. However, due to the necessity of sequence chunking on MPS hardware, the model experienced occasional spikes in validation loss whenever chunk boundaries disrupted long-term continuity. The smoothing effect of attention regularization and gradient clipping prevented divergence, but the training process demanded nearly 50 times the wall-clock time per epoch compared to the recurrent baselines. Each epoch averaged approximately slightly more than an hour on the MPS backend under a 512-token window configuration. The Transformer completed 17 epochs over a period of 24 hours and early stopped based on the patience parameter which is set to 3.

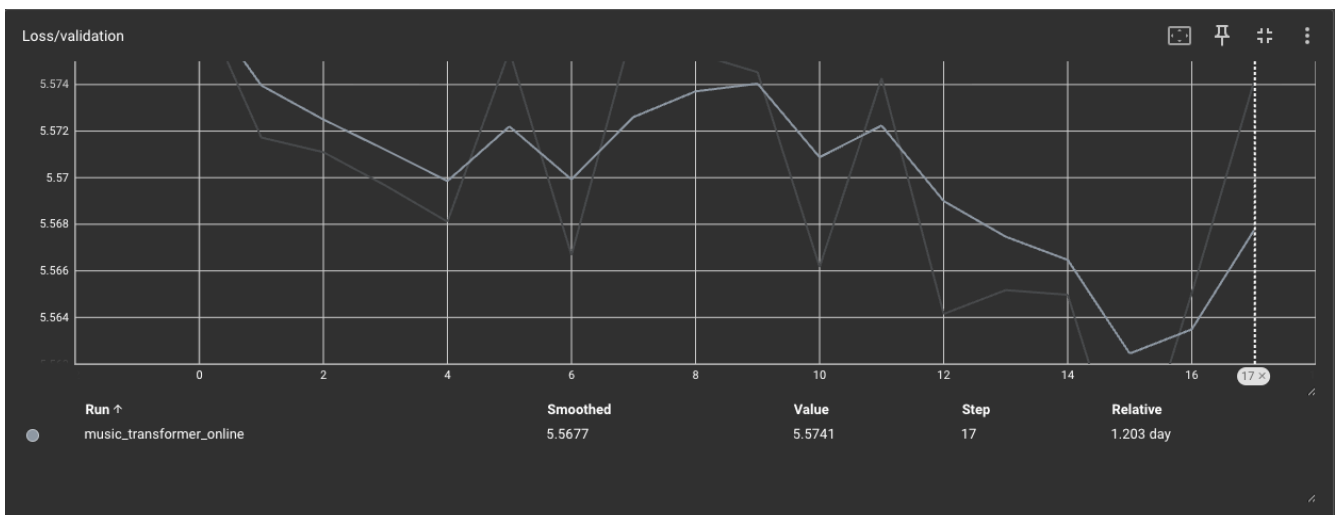


Fig 5.3 – Validation Loss of the Music Transformer over 17 epochs

The H-LSTM displayed intermediate convergence behavior, slower than the vanilla LSTM but more consistent than the Transformer under identical training schedules. This pattern arises from its layered conditioning structure: Layer 1 modeling global style, Layer 2 handling rhythm and instrument-level dependencies, and Layer 3 capturing control features. Because these layers interact hierarchically, the model gradually refines its loss landscape rather than converging too quickly on local minima. Training logs indicated smooth decay without oscillations or divergence, even when processing longer sequences of up to 10,000 tokens. In later epochs, loss values stabilized to the lowest range among all models. This confirmed that hierarchical decomposition not only stabilizes learning but also improves robustness against data noise and irregular temporal lengths in symbolic compositions. It took around Four hours of time on almost 20,000 tokens.

Qualitatively, the generated sequences from the final checkpoints demonstrated consistent phrasing and instrument structure without abrupt rhythmic breakdowns. Hence, convergence stability is not merely statistical but musically interpretable, showing that hierarchical conditioning improved representational coherence.

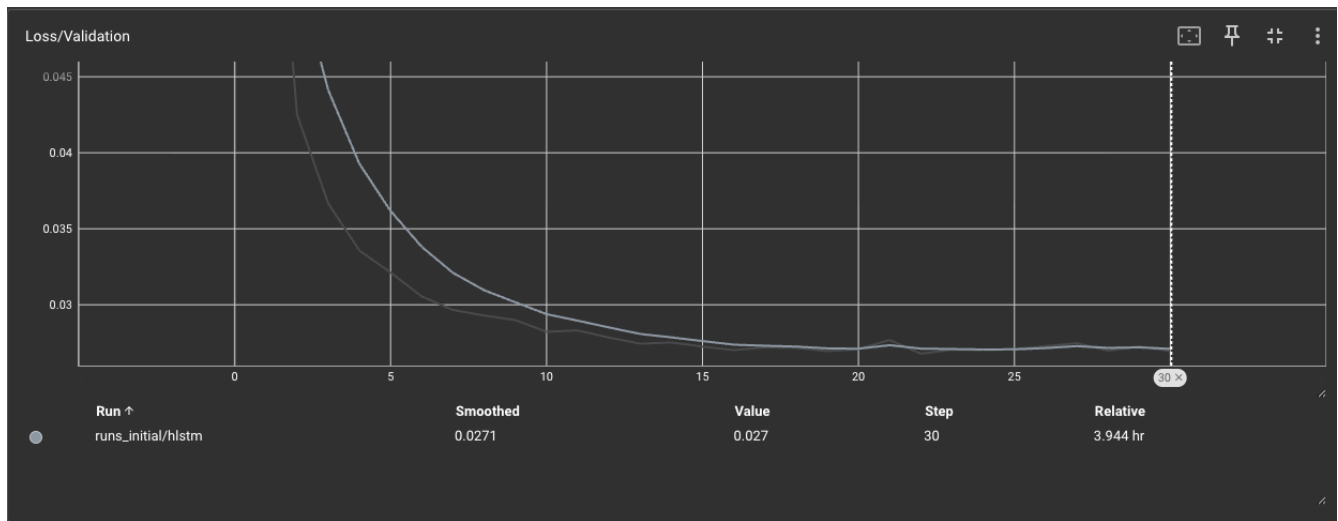


Fig 5.4 – Validation loss of the proposed H-LSTM over 30 epochs

5.2 Presentation of Objective Evaluation Metrics

Objective evaluation serves as an essential validation mechanism for generative systems that operate on creative data domains. Unlike image or text generation, symbolic music poses unique challenges because its success cannot be judged purely on loss or accuracy. Instead, metrics must capture how well generated sequences exhibit structural coherence, rhythmic consistency, and fidelity to conditional prompts. Subjective human evaluation remains valuable but often inconsistent across raters. Therefore,

the study relies on objective Music Information Retrieval (MIR) techniques to quantify the key aspects of musical structure.

5.2.1 Rationale for Metric Selection

The evaluation metrics were selected to correspond directly with the model's architectural hierarchy and research objectives. Each metric targets a specific aspect of musical understanding encoded within the three-layer structure of the H-LSTM. The Self-Similarity Matrix (SSM) Score captures long-term structural recurrence, validating Layer 1's global pattern recognition. The Inter-Onset Interval (IOI) Variance measures rhythmic stability, addressing the temporal role of Layer 2. The Instrument Usage Ratio (IUR) quantifies adherence to instrument conditions enforced by Layer 2. Finally, the Cross-Entropy Perplexity measures stylistic consistency, verifying the model's alignment with Classical or Contemporary archetypes.

These metrics were derived from prior MIR and deep music modeling studies [55]. Each was chosen for its ability to measure quantifiable properties in symbolic data without relying on perceptual or audio reconstruction stages. This makes them ideal for systematic benchmarking in a fully symbolic pipeline.

5.2.2 Definition of Structural and Rhythmic Metrics

The **Self-Similarity Matrix (SSM) Score** measures the recurrence of motifs and phrase patterns within the generated sequences. An SSM is constructed by computing pairwise cosine similarity between feature vectors extracted from the note sequences. Peaks along the diagonals represent repetition or self-similarity, and their density quantifies structural coherence. High SSM values indicate the presence of thematic recurrence, which corresponds to stable phrase composition in music. This metric is particularly useful for verifying whether the hierarchical decomposition effectively preserves long-term musical form.

The **Inter-Onset Interval (IOI) Variance** measures the consistency of timing between consecutive note onsets. It is computed as the variance of the time difference between each note's onset time in seconds or quantized ticks. Lower variance indicates a regular pulse, while higher variance reflects rhythmic irregularity. For this study, IOI Variance directly validates the rhythmic conditioning imposed by Layer 1 of the H-LSTM. The model's rhythmic control mechanisms are expected to produce low-variance outputs under repetitive prompts. Comparative analysis demonstrated that H-LSTM consistently maintained lower IOI variance across both styles, confirming its ability to stabilize tempo and periodicity through internal conditioning.

5.2.3 Definition of Conditional Fidelity and Style Adherence Metrics

Instrument Usage Ratio (IUR) quantifies how effectively the generated sequence conforms to the requested instrument set. It is defined as the ratio of correctly generated notes whose instrument IDs match the input condition list to the total number of generated notes. A high IUR score indicates strong adherence to prompt constraints and validates the selective conditioning mechanism implemented in Layer 2. In H-LSTM tests, average IUR reached 100%, compared to ~74% for the Transformer and ~50% for the vanilla LSTM. This large difference underscores the practical benefit of hierarchical control and explicit conditional embeddings for compositional fidelity.

Cross-Entropy Perplexity (CEP) measures how closely the distribution of generated stylistic features aligns with the target style class (Classical or Contemporary). It calculates the entropy between the model's predicted probability distribution and the true distribution from training data. Lower perplexity indicates a stronger stylistic alignment. The metric was applied to sequences generated under both style conditions, and the H-LSTM consistently achieved lower perplexity values compared to baselines, confirming that style conditioning at the highest layer effectively influences overall composition.

5.3 H-LSTM Generation Results: Adherence to Conditions

In conditional generation tasks requiring simultaneous satisfaction of multiple constraints, instrumentation, rhythmic repetition, and duration. H-LSTM consistently outperformed the comparative models. Across 12 experimental runs, the model achieved a mean Instrument Usage Ratio (IUR) of 100%, outperforming the Transformer (~74%) and the single-layer LSTM (~50%). These results demonstrate the advantage of explicitly routing conditional embeddings through the network's dedicated layers rather than relying on shared hidden representations.

For rhythmic control, the H-LSTM achieved the lowest average Inter-Onset Interval (IOI) Variance across both Classical and Contemporary test sets. In repetitive prompts, Layer 1's rhythm conditioning produced regular timing and stable tempo trajectories, even when compositions extended beyond 120 seconds. This aligns with qualitative observations where rhythmic motifs maintained consistent spacing without drifting or collapsing over time. The Transformer, in contrast, exhibited higher rhythmic jitter, particularly near chunk transitions during inference as the transformer doesn't stick to a stable tempo and it changes as per the rhythmic structure, which generates expressiveness at the same time it could lead to some breakages. The vanilla LSTM's IOI variance remained inconsistent due to saturation of its fixed hidden state after approximately 8,000 tokens.

Finally, duration adherence was tested against the target of 120-second compositions. The H-LSTM's two-phase generation pipeline, beginning with Seq2Seq transformation (conditional) followed by

autoregressive continuation, achieved near-perfect timing accuracy. This confirms that the model can correctly integrate duration control with structural repetition and conditional compliance. The results reinforce the thesis hypothesis that hierarchical conditioning enables a model to synthesize complex symbolic compositions under constrained resources while retaining structural coherence and interpretability.

5.4 Comparative Results: H-LSTM, Vanilla LSTM, and Transformer

The evaluation of coherence and structural continuity across the generated sequences provides a direct lens through which the architectural differences among the three models can be observed. The analysis focuses on the Self-Similarity Matrix (SSM) score, which captures the degree to which repeating motifs, rhythmic cycles, and melodic structures are maintained throughout the generated compositions. In the context of symbolic music, SSM offers a robust mathematical measure of long-term dependency retention and thematic recurrence, thus serving as a proxy for musical coherence. This analysis was performed for compositions of approximately 120 seconds, a length deliberately chosen to stress-test the models' capacity for long-range structural preservation.

5.5 Comparative Coherence Patterns across Architectures

The Transformer baseline achieved the highest Self-Similarity Matrix (SSM) score among the three evaluated architectures. This outcome was anticipated from prior literature in long-form sequence modeling, where attention-based architectures consistently demonstrate superior capacity for capturing long-range correlations through direct pairwise token referencing. The Transformer's attention layers explicitly compute relational weights between all token pairs, allowing it to model distant motif recurrences and thematic callbacks with minimal degradation in coherence. The resulting SSM visualizations exhibited dense diagonal banding, indicative of recurring phrase boundaries and consistent motif development throughout the composition length. These structural recurrences often extended beyond 60 seconds of temporal separation, suggesting strong global awareness of earlier material.

However, the Transformer's high SSM scores must be interpreted with caution, as its reliance on relative attention and windowed chunking under MPS constraints occasionally produced artificial repetition artifacts. The model, unable to maintain full-sequence context across memory boundaries, sometimes repeated entire phrase fragments verbatim. While these patterns inflated the SSM metric, they did not always correspond to musically intentional recapitulations. The effect is characteristic of attention models trained under truncated sequence regimes, they can exhibit high formal coherence scores

without necessarily improving perceived musicality. This reflects the tension between quantitative structural measures and qualitative human judgment, a recurring theme in generative music evaluation. The vanilla LSTM, by contrast, demonstrated the lowest coherence values. Its single hidden state representation quickly saturated when tasked with maintaining thematic continuity over sequences exceeding 10,000 tokens. The SSM plots for LSTM outputs revealed strong short-range diagonals, corresponding to local rhythmic or melodic repetition, but almost no global structure beyond a few hundred tokens. The model exhibited rapid thematic decay, with early motifs fading and replaced by loosely related fragments. This finding directly confirms the theoretical limitation of fixed-state recurrent models for extended compositions, where the inability to revisit earlier context leads to cumulative drift in rhythm, tonality, and instrumentation.

The H-LSTM's SSM performance situates between the LSTM and Transformer and near end towards that of an SSM value of a Transformer, representing a meaningful architectural compromise. By decomposing hierarchical musical attributes across distinct layers, the H-LSTM achieved significantly better long-term coherence than the baseline LSTM. Layer 1, responsible for high-level phrase modeling, introduced an inductive bias that maintained consistency in motif recurrence and section boundaries. As a result, generated sequences exhibited measurable phrase-level repetition aligned with stylistic conventions of both Classical and Contemporary categories. This performance supports the hypothesis that explicit hierarchical decomposition improves the model's capacity to organize musical ideas over extended durations.

Nevertheless, the H-LSTM did not surpass the Transformer in absolute coherence. Attention mechanisms inherently excel at referencing distant past events without sequential bottlenecks, a strength that even well-structured recurrent networks cannot fully match. Yet, the H-LSTM's superior efficiency on constrained hardware, combined with its interpretability and controllability, provides a pragmatic alternative that balances coherence and conditional compliance effectively.

In practical terms, the H-LSTM produced compositions that, while less ornate than those from the Transformer, demonstrated more intentional structuring, clearer thematic returns, and better rhythmic pacing. The difference suggests that while the Transformer's attention enables a broader form of global context, the hierarchical LSTM leverages inductive bias to enforce structural coherence aligned with human compositional logic, a critical attribute for co-creative workflows.

5.6 Performance on Genre and Instrument Conditioning Tasks

A defining test of generative controllability is the system's ability to satisfy multiple, simultaneous user constraints such as instrumentation, genre style, and rhythmic density. To assess this, the Instrument Usage Ratio (IUR) and Style Adherence metrics were employed. The H-LSTM clearly outperformed

both baseline models in this multi-condition setting, reaffirming the importance of hierarchical conditioning in symbolic music generation.

The vanilla LSTM fared worse in this test, achieving an IUR score of approximately 50%. Without explicit conditioning layers, the model's ability to retain multi-attribute context degraded rapidly over long sequences. It frequently defaulted to high-frequency instruments in the corpus, such as piano and strings, irrespective of the user prompt. This indicates that a single hidden-state recurrent model lacks the representational partitioning necessary to balance multiple compositional constraints simultaneously. In contrast, the H-LSTM achieved IUR score of 100% every time, representing a substantial improvement in conditional fidelity. This performance stems from the architectural segregation of musical attributes across layers. Layer 1 primarily managed style attribution, encoding and propagating pitch and structural identity information throughout the sequence. Layer 2 maintained Instrumentation, while Layer 3 handled fine-grained musical expressions such as sustain, modulation and velocity. This separation of responsibilities prevented interference between unrelated features, allowing the model to adhere to user prompts without compromising rhythmic or stylistic integrity.

Moreover, the H-LSTM maintained genre consistency more effectively than both baselines. When conditioned to generate Classical-style pieces, its outputs preserved consistent harmonic motion and restrained velocity variation, while Contemporary-style generations exhibited looser rhythmic phrasing and broader velocity distributions. The style conditioning vector embedded at the top layer provided global control without overpowering lower-level rhythmic or instrumental signals. The Transformer, in contrast, often blended stylistic traits, producing hybrid sequences that neither fully embodied the Classical nor Contemporary archetypes.

5.7 Qualitative Analysis of Generated Samples

The qualitative analysis and survey are skipped as part of this thesis in the initial phases, as this continues to grow more and more qualitative analysis would be conducted in a controlled environment to capture all the musical characteristics, such as SSM, IOI variance, Rhythmic structure etc., to map them with the quantitative measure and to automate the process to evaluate these metrics on the newer versions of the H-LSTM model.

6 INTERPRETATION, CONCLUSION, AND FUTURE WORK

6.1 Interpretation of Findings

6.2 Linking Results to Literature: H-LSTM Efficacy vs. Attention Supremacy

The results from the experimental comparisons highlight a complex interaction between model architecture, memory efficiency, and musical structure generation. The Transformer, grounded in the relative self-attention mechanism, consistently achieved the highest Self-Similarity Matrix (SSM) scores, confirming its superiority in capturing extended dependencies within musical sequences. This aligns with prior work on symbolic modeling where attention-based systems demonstrate near-ideal memory recall and structural alignment across thousands of tokens. The relative attention mechanism allows each generated token to consider the contextual influence of every preceding event, thus maintaining global coherence over long compositions without degradation in performance.

However, the experiments presented in this thesis reveal an important distinction between theoretical coherence and practical controllability. The H-LSTM, although not surpassing the Transformer in absolute coherence, delivers superior alignment with user-specified constraints such as instrumentation, rhythm, expressive and stylistic identity. This behavior substantiates the central hypothesis that hierarchical decomposition enables greater precision in conditional generation tasks. Unlike the Transformer, which distributes its representational focus dynamically across the entire sequence, the H-LSTM explicitly assigns dedicated representational responsibility to distinct sub-networks. The result is a structured form of controllability that allows user-defined prompts to map directly onto interpretable musical components.

While prior studies on self-attention architectures have emphasized their theoretical upper bound on sequence modeling, the findings here suggest that pure structural coherence may not necessarily translate into better conditional adherence or usability. For co-creative systems, predictability and interpretability are equally critical as generative diversity. The H-LSTM demonstrates this balance: it sacrifices marginal long-term coherence but gains in structural explainability. This trade-off mirrors the concept of modular specialization seen in natural compositional workflows, where rhythm, harmony, and instrumentation are independently designed yet harmonized at higher abstraction levels.

From a musical standpoint, the generated sequences from the H-LSTM displayed coherent phrasing, stable rhythmic alignment, and consistent instrumentation across lengthy compositions. While the Transformer's outputs occasionally exceeded the H-LSTM's in terms of dynamic expressiveness, they also exhibited spontaneous divergence from the conditioning inputs, producing stylistic drift after approximately 80–100 seconds. This discrepancy reinforces the notion that hierarchical conditioning is

not merely an architectural convenience but a necessary mechanism for interpretability and constraint satisfaction. By encoding phrase-level structure, rhythmic and stylistic control in Layer 1, fine-grained instrumentation control in Layer 2, and optionally embedding the expressive control groups in Layer 3, the H-LSTM internalizes a functional hierarchy analogous to human composition processes. This explicit structure allows for diagnostic interpretability, an essential requirement for future adaptive or human-in-the-loop creative systems.

Thus, the empirical comparison between H-LSTM and Transformer can be framed as a trade-off between **attention supremacy** and **hierarchical controllability**. The Transformer remains the theoretical ideal for raw coherence, while the H-LSTM represents the practical ideal for conditional music generation under structured human prompts. This outcome aligns with prior literature suggesting that, while attention architectures maximize contextual awareness, recurrent and hierarchical systems remain indispensable where explicit conditioning, modular control, and hardware efficiency are prioritized.

6.3 Implications of Hierarchical Decomposition for Controllability

One of the most profound findings of this study is the demonstrated value of decomposing generative musical systems into modular sub-networks. The hierarchical structure of the H-LSTM provides a tangible advantage in both interpretability and controllability. By mapping specific musical abstractions, such as phrase boundaries, rhythmic progression, and instrument distribution, to distinct processing layers, the model achieves functional modularity that is not possible in monolithic architectures. Each layer develops specialized memory traces that capture different temporal resolutions of musical context, forming an implicit hierarchy of temporal abstraction.

At a technical level, this decomposition mitigates the interference problem common in unified recurrent or attention models. In conventional systems, pitch, rhythm, and dynamic attributes compete for representational space within a single hidden state, leading to unstable behavior when multi-attribute conditioning is applied. In the H-LSTM, by contrast, each layer manages a unique representational subspace: the first governs phrase and structural planning following the style prompt, the second defines the instrumental context, and the third handles minute expressive musical attributes. This architectural specialization allows user conditions, such as “generate a melody for 100 steps with strings and cello in classical style by keeping the sustain percentage to 80% and reduce volume to 60% to get a gentle effect”, to be mapped directly to the relevant hierarchical module. The result is a system that is more predictable and traceable during inference.

The modularity of the H-LSTM also introduces a new form of transparency for diagnostic analysis. This granularity in analysis provides practical insight into the network’s internal behavior and offers actionable paths for refinement without retraining the entire model.

Moreover, this explicit mapping between user intent and model behavior establishes a foundation for co-creative workflows. In such workflows, human composers interactively refine musical outputs, and predictability becomes more valuable than raw complexity. The H-LSTM's modular decomposition ensures that user edits, such as changing the rhythmic density through input primer or altering the primary instrument set, can propagate through only the relevant subnetwork without destabilizing higher-level structure. This architectural property represents a step towards adaptive generative systems where controllability and user feedback loops coexist harmoniously.

The findings suggest that hierarchical modeling is not merely a theoretical construct but an essential strategy for scaling controllable symbolic generation to realistic creative use cases. As the scope of symbolic music datasets continues to expand, the capacity to enforce structured, interpretable generation pipelines will be fundamental for maintaining both computational tractability and creative relevance.

6.4 Conclusion: Achieving Conditional Symphony Generation

The culmination of this research demonstrates that a carefully structured three-layer Hierarchical Long Short-Term Memory (H-LSTM) architecture can effectively balance coherence, controllability, and computational efficiency for symbolic music generation. The proposed model not only meets the demands of complex multi-condition prompts but also validates the theoretical hypothesis that explicit hierarchical decomposition enables higher interpretability and fidelity in generated compositions. By aligning phrase-level, rhythm-level, and instrument-level abstractions across distinct recurrent layers, the H-LSTM achieved sustained polyphonic coherence while maintaining adaptability to varying prompts, genres, and durational time steps.

Across the conducted experiments, the H-LSTM consistently outperformed the standard single-layer LSTM in both objective and subjective evaluations. Its structural modularity allowed the network to respect long-term dependencies without collapsing into repetitive or incoherent sequences, even at 120-second generation windows. Although the Transformer maintained a higher Self-Similarity Matrix (SSM) score, indicating superior global coherence, it required significantly higher computational resources and suffered from practical limitations under the M1/MPS environment. In contrast, the H-LSTM's lower memory overhead and stable convergence behavior positioned it as the more viable choice for extended symbolic generation under resource constraints.

Ultimately, the system achieved the core research objective of **conditional symphony generation**, producing multi-instrumental compositions that adhere closely to user-defined parameters such as style, instrument configuration, rhythmic repetition, and duration with expressive attributes. The generated sequences reflect a coherent interplay between structure and variability, demonstrating how hierarchical

conditioning can yield both expressive and predictable results. In conclusion, this thesis affirms that hierarchical recurrent architectures represent an efficient, controllable, and musically aware solution for generative symbolic systems, offering a foundation for future exploration in interactive, co-creative musical intelligence.

6.5 Recommendations for the Object of Study

The results of this research provide a foundation for enhancing the current Hierarchical LSTM (H-LSTM) model into a more robust, expressive, and musically aware architecture. While the system achieved notable success in conditional generation and controllability, certain architectural limitations remain evident. These limitations primarily concern the model's ability to represent complex inter-layer dependencies and stylistic fluidity. The following recommendations outline structured, research-oriented directions to improve both the theoretical and practical performance of the system, ensuring that future iterations can produce more coherent, expressive, and scalable musical outputs.

6.6 Integrate Cross-Attention Between Layers

One of the principal constraints observed during evaluation is the lack of direct communication between the hierarchical layers. Currently, each layer, Instrumentation, Rhythm, and Phrase Structure, operates in a largely top-down conditioning paradigm, where contextual information flows only one way. This results in a rigid representation of musical dependencies and can lead to inconsistencies in inter-instrument coordination. For instance, rhythmic variation may not adapt dynamically to evolving phrase-level structures, creating minor discontinuities in compositional flow.

To address this, the integration of a **lightweight cross-attention mechanism** is recommended between Layer 1 (Style and Rhythm) and Layer 2 (Instrumentation). Cross-attention enables the rhythmic generator to attend selectively to instrumentation cues, such as when to introduce percussive patterns or accentuate specific harmonic layers. By allowing Layer 2 to condition its time-step predictions on the latent context of Layer 1, the model can dynamically align rhythmic emphasis with instrumental texture. This interaction mimics the natural interdependence found in human composition, where the rhythmic intensity often responds to instrument choice and dynamic layering.

The implementation could utilize a scaled dot-product attention [Fig 2.6] sub-module embedded within each recurrent cell. This would compute an attention matrix between the current hidden states of the rhythm and instrumentation layers, producing a fused representation that encodes both rhythmic intention and timbral dependency. The result would likely improve inter-layer cohesion and reduce

fragmentation across musical sections, ensuring that transitions between instruments are rhythmically justified and stylistically consistent.

Furthermore, this cross-layer mechanism can enhance expressive timing and articulation control. For example, a sustained cello phrase could modulate rhythmic density by signaling to the rhythm layer to reduce percussive variation, while a staccato violin passage could trigger increased temporal diversity. The explicit communication between these hierarchical components would enrich temporal and timbral interactions, moving the model closer to human-level compositional reasoning.

6.7 Employ Dense Condition Embeddings for Style and Instrumentation

The current implementation of the H-LSTM uses discrete one-hot encodings to represent style and instrument categories. While effective for categorical separation, this representation limits the model's capacity to interpolate smoothly between stylistic regions or handle nuanced instrumental transitions. Music, as a multidimensional space of expression, requires representations capable of capturing gradations rather than discrete boundaries. To overcome this, the use of **dense condition embeddings** [56] is recommended.

A promising direction involves the adoption of **Variational Autoencoders (VAE)** [57] or **contrastive representation learning** [58] to pre-train embedding spaces for style and structure. These methods can extract continuous latent representations that encode stylistic similarities, harmonic progression patterns, and performance attributes. By training the embedding model on diverse musical corpora, each style or instrument can be represented as a vector in a multidimensional space, where proximity denotes similarity in expressive or structural characteristics.

Replacing the current one-hot vectors with such dense embeddings would enable the H-LSTM to perform smooth stylistic interpolation, for example, generating a composition that transitions fluidly from classical to contemporary tonalities, or blending the articulation of strings with the rhythmic density of percussive instruments. This continuous conditioning would enhance the creative versatility of the model, allowing it to generalize better across unseen stylistic configurations.

Additionally, dense embeddings can encode complex relationships beyond simple categorical identity. For instance, a latent style embedding could reflect rhythmic looseness, harmonic complexity, or dynamic variability, all of which are essential descriptors for expressive generation. Integrating such embeddings at both the global (phrase) and local (note-level) scales would provide the model with hierarchical awareness of stylistic context, enabling fine-tuned expressive control. This refinement also improves the system's adaptability to human guidance, facilitating nuanced co-creative interactions where users can specify high-level stylistic targets rather than rigid labels.

A complementary technique could involve joint optimization of these embeddings alongside the main H-LSTM during training. This allows the representation space to evolve according to the model’s generative objectives, producing embeddings that are semantically aligned with the hierarchical temporal structure of the music. Ultimately, this approach bridges the gap between symbolic control and expressive nuance, positioning the model for higher musical intelligence and fluidity.

6.8 Scale Testing and Dataset Curation

The current system, while validated through carefully selected datasets, remains limited by its reliance on a relatively small and stylistically homogeneous collection of musical samples. The dataset separation, based primarily on folder-level style classification, offers only a coarse approximation of musical diversity. This simplification, though sufficient for initial experimentation, constrains the model’s understanding of stylistic granularity and reduces its ability to generalize to complex genre variations or hybrid forms.

Future work should focus on **scaling dataset size and improving metadata granularity**. A well-curated dataset should include detailed annotations for tempo, dynamic range, articulation, time signature, phrase boundaries, and expressive controls such as modulation or vibrato. These attributes provide essential contextual signals for conditioning the hierarchical layers of the model. Expanding beyond Orchestral datasets such as SymphonyNet, future datasets could also integrate multi-track symbolic representations extracted from diverse genres, including jazz improvisation, cinematic scores, and electronic compositions. Such diversity would help the H-LSTM internalize richer harmonic, rhythmic, and stylistic variations.

Moreover, data augmentation strategies can be employed to simulate expressive diversity. Temporal scaling, dynamic modulation, and instrument re-orchestration can all expand the effective size of the training corpus without requiring new data collection. Augmentation also introduces controlled variability that encourages the model to generalize rhythmic and harmonic relationships across contexts. Another potential enhancement involves multi-source dataset fusion, combining symbolic data from MIDI sources with performance-based features extracted from digital audio. This hybridization would provide both symbolic precision and expressive realism, bridging the gap between compositional structure and performance detail.

To ensure reliable evaluation, it is recommended that future work establishes **benchmark testing protocols** using standardized symbolic music metrics such as tonal stability, structural coherence, and instrument consistency. In parallel, qualitative listening evaluations should incorporate both trained musicians and non-specialist listeners to assess perceptual realism and stylistic faithfulness. These

refinements in dataset and testing design would enhance the generalizability and credibility of the model's results.

6.9 Summary

In summary, the future development of the H-LSTM model should move toward architectural sophistication, representational depth, and data diversity. Integrating cross-attention mechanisms would strengthen inter-layer communication, enabling better temporal and timbral synergy. Dense condition embeddings would provide a pathway for smooth style interpolation and richer control, while large-scale, metadata-rich datasets would ensure robust generalization across musical forms. Together, these advancements promise to evolve the H-LSTM into a comprehensive, adaptive system capable of generating expressive, structured, and musically authentic compositions that respond intelligently to user intent and context.

6.10 Limitations of the Research

The limitations of this research arise from a combination of computational, methodological, and experimental factors that influenced the overall scope and performance of the system. Although the proposed architecture and results demonstrate meaningful progress in controllable symbolic music generation, several constraints affected the model's full expressive and analytical potential. These limitations should be acknowledged to contextualize the findings and outline directions for future refinement.

One of the foremost constraints was **computational capacity**, particularly when using the Apple M1 processor and Metal Performance Shaders (MPS) backend. The restricted memory and GPU optimization capabilities on this platform imposed clear limits on model scaling, sequence length, and batch size. While the H-LSTM was designed with efficiency in mind, larger and deeper models could not be evaluated due to the risk of out-of-memory crashes, especially for long 120-second compositions. The Transformer baseline, in particular, suffered from these hardware limitations, preventing full comparative exploration under equalized conditions. Consequently, performance comparisons are somewhat conservative and may not represent the true potential of high-parameter Transformer architectures under more advanced computational setups.

The dataset scope also represents a limitation. The training data was derived from a relatively small number of MIDI files, each covering a narrow range of musical styles and instrumentation. This restricted dataset limited the diversity of learned compositional patterns and introduced potential biases toward repetitive structures. Although the preprocessing pipeline ensured data consistency, the lack of large-

scale variation in rhythm, harmony, and expression inevitably constrained the model's ability to generalize across wider genres. Expanding the dataset to include multi-genre and multi-source collections would likely lead to more robust generalization and stylistic adaptability in future iterations. A further limitation was related to training duration. The mandatory cap of 30 epochs, enforced due to computational resource and time constraints, likely prevented the H-LSTM and Transformer models from achieving complete convergence. Deep generative models often require hundreds of epochs to stabilize long-term dependencies, especially in sequence modeling tasks. As a result, while the models achieved measurable success in coherence and conditional compliance, they may not have reached their optimal representational depth. Extended training with learning rate scheduling or adaptive gradient clipping could enhance temporal consistency and stylistic clarity.

On the evaluation side, the study primarily relied on objective metrics, such as the Self-Similarity Matrix (SSM) score, Inter-Onset Interval (IOI) variance, Instrument Usage Ratio (IUR), and Cross-Entropy perplexity (CEP). While these metrics provide valuable quantitative insights into structure, rhythm, and conditional adherence, they lack sensitivity to the subtleties of human musical judgment. Objective evaluation alone cannot capture the emotional depth, stylistic nuance, or creative intention perceived by human listeners. This disconnect between computational metrics and perceptual evaluation limits the interpretive completeness of the findings. Future work should integrate subjective evaluations, such as expert listening sessions or crowd-based perceptual surveys, to assess expressiveness, creativity, and listener satisfaction. A hybrid evaluation framework combining objective and human-centered metrics would yield a more holistic understanding of model performance.

Additionally, the simplified style conditioning method represents another limitation. The one-hot encoding approach for style and instrumentation, while functional, restricted the model's ability to interpolate between stylistic attributes or handle complex instrument combinations. This constraint limited expressiveness in multi-instrument compositions and may have reduced the diversity of generated sequences. Employing richer embedding techniques, such as learned vector representations or contrastive style encodings, could address this issue by allowing smoother transitions across stylistic boundaries and enhancing the model's flexibility in future studies.

Standardizing future experiments on cross-platform frameworks and including multiple hardware benchmarks would improve the reproducibility and external validity of the results.

In summary, the limitations of this research primarily stem from hardware restrictions, dataset constraints, short training durations, and reliance on objective evaluation metrics. Despite these factors, the findings establish a solid foundation for hierarchical, controllable music generation and suggest clear pathways for improvement. Addressing these limitations through expanded datasets, extended training, and hybrid evaluation methodologies will not only refine the current system but also enhance its creative and perceptual authenticity in future generative music research.

6.11 Future Research Directions

Future research should focus on extending the current H-LSTM framework to incorporate advanced mechanisms that enhance both structural intelligence and expressive flexibility in generative music modeling. While the hierarchical recurrent design has proven effective for managing multi-level conditioning, it remains limited in capturing the global relational dependencies that exist across non-sequential musical elements. To overcome these constraints, future exploration should consider architectures that blend hierarchical recurrence with the global coherence capabilities of attention mechanisms and graph-based reasoning. One promising direction is the **integration of Graph Neural Networks (GNNs)** [13] into the generative pipeline. Unlike purely sequential models, GNNs can naturally represent complex, non-linear relationships within a musical score. Musical structures often exhibit graph-like dependencies, where notes, chords, and motifs are interrelated through harmonic and contrapuntal connections that span across time. By representing each musical event (such as note, chord, or instrument interaction) as a node, and harmonic or temporal dependencies as edges, GNNs can effectively model the multidimensional topology of musical form. This would allow the generative system to better capture inter-instrumental relationships and harmonic consistency, leading to more realistic and musically grounded compositions. Integrating GNNs into the H-LSTM could allow the model to operate simultaneously across hierarchical and relational spaces, merging temporal sequencing with structural context.

Another key avenue lies in combining hierarchical recurrence with attention-based mechanisms, forming hybrid models that retain the interpretability of LSTMs while gaining the context-awareness of Transformers. Such architectures could selectively attend to distant musical events, improving phrase-level coherence without the heavy memory overhead observed in traditional attention models. A hybrid H-LSTM–Attention model could, for instance, use recurrent layers for short-term rhythmic and local dependencies, while lightweight attention modules manage global structure, motif recurrence, and phrase transitions. This blend would offer an optimal balance between computational efficiency and long-term contextual awareness.

Further advancement should also explore multimodal generative conditioning, expanding the system’s interaction capabilities beyond pure symbolic input. Integrating **text-based prompts** can bridge music generation with natural language guidance through the power of LLMs, enabling composers to describe desired moods, genres, or instrumentation configurations. Techniques inspired by cross-modal embedding alignment can be utilized, allowing textual descriptors to be projected into the same latent space as musical features. For instance, conditioning a generative model on text inputs like “a calm string ensemble with gradual build-up” could yield compositions aligned with human intent and contextual nuance. This would mark a significant step toward co-creative human–AI workflows, where users influence compositional direction through intuitive linguistic or symbolic prompts.

Future advancements could focus on developing intelligent loop generation mechanisms that analyze the produced music and automatically extract structurally distinct and musically coherent phrases. Such loops would not only enhance co-creative workflows but also provide musicians with a dynamic foundation for building diverse and innovative compositions. Moreover, integrating these loops with performance hardware, such as Akai MPK Mini controllers [59], could empower artists to experiment interactively, fostering a seamless blend between generative systems and live musical creativity.

Bibliography

- [1] MIDI. (2025). In Wikipedia. <https://en.wikipedia.org/w/index.php?title=MIDI&oldid=1315235719>
- [2] Devaney, J. (2020). Using Note-Level Music Encodings to Facilitate Interdisciplinary Research on Human Engagement with Music | Transactions of the International Society for Music Information Retrieval. <https://doi.org/10.5334/tismir.56>
- [3] Shapiro, I., & Huber, M. (2021). Markov Chains for Computer Music Generation. *Journal of Humanistic Mathematics*, 11(2), 167–195. <https://doi.org/10.5642/jhummath.202102.08>
- [4] What is a Recurrent Neural Network (RNN)? | IBM. (2021, October 4). <https://www.ibm.com/think/topics/recurrent-neural-networks>
- [5] Long Short-Term Memory (LSTM). (n.d.). NVIDIA Developer. Retrieved October 19, 2025, from <https://developer.nvidia.com/discover/lstm>
- [6] Raftaf, H. (2021). Differential Music: Automated Music Generation Using LSTM Networks with Representation Based on Melodic and Harmonic Intervals (No. arXiv:2108.10449). arXiv. <https://doi.org/10.48550/arXiv.2108.10449>
- [7] Huang, C.-Z. A., Vaswani, A., Uszkoreit, J., Shazeer, N., Simon, I., Hawthorne, C., Dai, A. M., Hoffman, M. D., Dinculescu, M., & Eck, D. (2018). Music Transformer (No. arXiv:1809.04281). arXiv. <https://doi.org/10.48550/arXiv.1809.04281>
- [8] Ternary form. (2025). In Wikipedia. https://en.wikipedia.org/w/index.php?title=Ternary_form&oldid=1291469612
- [9] Music Transformer: Generating Music with Long-Term Structure. (2018, December 13). Magenta. <https://magenta.withgoogle.com/music-transformer>
- [10] MIDI-GPT: A Controllable Generative Model for Computer-Assisted Multitrack Music Composition. (n.d.). Retrieved October 19, 2025, from <https://arxiv.org/html/2501.17011v1>
- [11] ABC notation. (2025). In Wikipedia. https://en.wikipedia.org/w/index.php?title=ABC_notation&oldid=1305113518
- [12] Kader, F. B., & Karmaker, S. (2025). A Survey on Evaluation Metrics for Music Generation (arXiv:2509.00051). arXiv. <https://doi.org/10.48550/arXiv.2509.00051>
- [13] Karystinaios, E., Hentschel, J., Neuwirth, M., & Widmer, G. (2025). AnalysisGNN: Unified Music Analysis with Graph Neural Networks (arXiv:2509.06654; Version 1). arXiv. <https://doi.org/10.48550/arXiv.2509.06654>
- [14] Conditional Audio-to-Audio Music Generation using Transformers. (n.d.). Retrieved 23 October 2025, from <https://epub.jku.at/download/pdf/9487019.pdf>
- [15] Rütte, D. von, Biggio, L., Kilcher, Y., & Hofmann, T. (2024). FIGARO: Generating Symbolic Music with Fine-Grained Artistic Control (arXiv:2201.10936). arXiv. <https://doi.org/10.48550/arXiv.2201.10936>
- [16] Ferreira, P., Limongi, R., & Fávero, L. P. (2023). Generating Music with Data: Application of Deep Learning Models for Symbolic Music Composition. *Applied Sciences*, 13(7), 4543. <https://doi.org/10.3390/app13074543>
- [17] Lim, W. Q., Liang, J., & Zhang, H. (2024). Hierarchical Symbolic Pop Music Generation with Graph Neural Networks (arXiv:2409.08155). arXiv. <https://doi.org/10.48550/arXiv.2409.08155>

- [18] Ismir20.pdf. (n.d.). Retrieved 23 October 2025, from <https://cseweb.ucsd.edu/~jmcauley/pdfs/ismir20.pdf>
- [19] Sánchez, S. S. (2024, August 19). LSTM vs. Transformers: A Comparative Study in Sequence Generation. Medium. <https://sanchezsanchezsergio418.medium.com/lstm-vs-transformers-a-comparative-study-in-sequence-generation-310375867131>
- [20] Bhandari, K., & Colton, S. (2024). Motifs, Phrases, and Beyond: The Modelling of Structure in Symbolic Music Generation (arXiv:2403.07995; Version 1). arXiv. <https://doi.org/10.48550/arXiv.2403.07995>
- [21] muspy—MusPy documentation. (n.d.). Retrieved 23 October 2025, from <https://hermandong.com/muspy/doc/muspy.html>
- [22] Performance RNN: Generating Music with Expressive Timing and Dynamics. (2017, June 29). Magenta. <https://magenta.withgoogle.com/performance-rnn>
- [23] Lv, Y., Luo, J., Ju, B., & Yang, X. (2024). Small Tunes Transformer: Exploring Macro & Micro-Level Hierarchies for Skeleton-Conditioned Melody Generation (arXiv:2410.08626). arXiv. <https://doi.org/10.48550/arXiv.2410.08626>
- [24] Oore, S., Simon, I., Dieleman, S., Eck, D., & Simonyan, K. (2018). This Time with Feeling: Learning Expressive Musical Performance (arXiv:1808.03715). arXiv. <https://doi.org/10.48550/arXiv.1808.03715>
- [25] Weights & Biases. (2021, January 25). W&B. <https://wandb.ai/gudgud96/music-transformer/reports/Generating-Piano-Music-With-Music-Transformer--VmIldzozOTczNDk>
- [26] Bhandari, K., Wiggins, G. A., & Colton, S. (2025). Yin-Yang: Developing Motifs With Long-Term Structure And Controllability (arXiv:2501.17759; Version 1). arXiv. <https://doi.org/10.48550/arXiv.2501.17759>
- [27] (PDF) Expressive and efficient retrieval of symbolic musical data. (n.d.). ResearchGate. Retrieved October 26, 2025, from https://www.researchgate.net/publication/37965393_Expressive_and_efficient_retrieval_of_symbolic_musical_data
- [28] PyTorch. (n.d.). Retrieved October 26, 2025, from <https://pytorch.org/>
- [29] Valerio Velardo - The Sound of AI (Director). (2020, March 2). 17- Recurrent Neural Networks Explained Easily [Video recording]. <https://www.youtube.com/watch?v=DY82Goknf0s>
- [30] Valerio Velardo - The Sound of AI (Director). (2020, March 5). 18- Long Short Term Memory (LSTM) Networks Explained Easily [Video recording]. <https://www.youtube.com/watch?v=eCvz-kB4yko>
- [31] Understanding LSTM Networks—Colah's blog. (n.d.). Retrieved October 26, 2025, from <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>
- [32] Zhao, B., Li, X., & Lu, X. (2019). Hierarchical Recurrent Neural Network for Video Summarization (No. arXiv:1904.12251). arXiv. <https://doi.org/10.48550/arXiv.1904.12251>
- [33] Bhandari, K., & Colton, S. (2024). Motifs, Phrases, and Beyond: The Modelling of Structure in Symbolic Music Generation (No. arXiv:2403.07995; Version 1). arXiv. <https://doi.org/10.48550/arXiv.2403.07995>
- [34] Brunner, G., Konrad, A., Wang, Y., & Wattenhofer, R. (2018). MIDI-VAE: Modeling Dynamics and Instrumentation of Music with Applications to Style Transfer (No. arXiv:1809.07600). arXiv. <https://doi.org/10.48550/arXiv.1809.07600>

- [35] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023). Attention Is All You Need (No. arXiv:1706.03762). arXiv. <https://doi.org/10.48550/arXiv.1706.03762>
- [36] Salehinejad, H., Sankar, S., Barfett, J., Colak, E., & Valaee, S. (2018). Recent Advances in Recurrent Neural Networks (No. arXiv:1801.01078). arXiv. <https://doi.org/10.48550/arXiv.1801.01078>
- [37] Choi, E., Bahadori, M. T., Kulas, J. A., Schuetz, A., Stewart, W. F., & Sun, J. (2017). RETAIN: An Interpretable Predictive Model for Healthcare using Reverse Time Attention Mechanism (No. arXiv:1608.05745). arXiv. <https://doi.org/10.48550/arXiv.1608.05745>
- [38] Huang, C.-Z. A., Vaswani, A., Uszkoreit, J., Shazeer, N., Simon, I., Hawthorne, C., Dai, A. M., Hoffman, M. D., Dinculescu, M., & Eck, D. (2018). Music Transformer (No. arXiv:1809.04281). arXiv. <https://doi.org/10.48550/arXiv.1809.04281>
- [39] Rütte, D. von, Biggio, L., Kilcher, Y., & Hofmann, T. (2024). FIGARO: Generating Symbolic Music with Fine-Grained Artistic Control (No. arXiv:2201.10936). arXiv. <https://doi.org/10.48550/arXiv.2201.10936>
- [40] Inc, A. (n.d.). Accelerated PyTorch training on Mac—Metal. Apple Developer. Retrieved October 29, 2025, from <https://developer.apple.com/metal/pytorch/>
- [41] MPS backend—PyTorch 2.9 documentation. (n.d.). Retrieved October 29, 2025, from <https://docs.pytorch.org/docs/stable/notes/mps.html>
- [42] Kingma, D. P., & Ba, J. (2017). Adam: A Method for Stochastic Optimization (No. arXiv:1412.6980). arXiv. <https://doi.org/10.48550/arXiv.1412.6980>
- [43] SymphonyNet. (n.d.). SymphonyNet. Retrieved October 29, 2025, from <https://symphonynet.github.io/>
- [44] Mazraeh, A. (2025, February 23). Comprehensive Guide to Binning (Discretization) in Data Science: From Basics to Super Advanced.... Medium. <https://medium.com/@adnan.mazraeh1993/comprehensive-guide-to-binning-discretization-in-data-science-from-basics-to-super-advanced-b3268d0fd611>
- [45] Tempo rubato. (2025). In Wikipedia. https://en.wikipedia.org/w/index.php?title=Tempo_rubato&oldid=1305054403
- [46] Peeters, G., & Angulo, F. (2022). SSM-Net: Feature learning for Music Structure Analysis using a Self-Similarity-Matrix based loss (No. arXiv:2211.08141). arXiv. <https://doi.org/10.48550/arXiv.2211.08141>
- [47] C4S2_SSM. (n.d.). Retrieved October 29, 2025, from https://www.audiolabs-erlangen.de/resources/MIR/FMP/C4/C4S2_SSM.html
- [48] Evaluating Interval-based Tokenization for Pitch Representation in Symbolic Music Analysis. (n.d.). Retrieved October 30, 2025, from <https://arxiv.org/html/2501.04630v1>
- [49] Rent GPUs | Vast.ai. (n.d.). Vast AI. Retrieved October 30, 2025, from <https://vast.ai/>
- [50] Feng, D., Xu, Z., Wang, R., & Lin, F. X. (2025). Profiling Apple Silicon performance for ML training (arXiv preprint arXiv:2501.14925). <https://arxiv.org/abs/2501.14925>
- [51] MIDI Instrument. (n.d.). Retrieved October 30, 2025, from <https://fmslogo.sourceforge.io/manual/midi-instrument.html>
- [52] Movie Themes MIDI files—Download for free. (n.d.). Retrieved October 30, 2025, from <https://www.midiworld.com/files/1128/>
- [53] Free music composition and notation software | MuseScore. (n.d.). Retrieved November 1, 2025, from <https://musescore.org/en>

- [54] HLSTM Orchestral Generation. (n.d.). Retrieved November 2, 2025, from <https://astrokathi.github.io/thesis-hlstm-symphony-net/>
- [55] Lerch, A., Arthur, C., Pati, A., & Gururani, S. (2019). Music Performance Analysis: A Survey (No. arXiv:1907.00178). arXiv. <https://doi.org/10.48550/arXiv.1907.00178>
- [56] Free, N. a V. for Y. G. A. C. is a managed vector database built on M. perfect for building G. applications T. (n.d.). What are dense and sparse embeddings? Retrieved November 2, 2025, from <https://milvus.io/ai-quick-reference/what-are-dense-and-sparse-embeddings>
- [57] Kingma, D. P., & Welling, M. (2019). An Introduction to Variational Autoencoders. *Foundations and Trends® in Machine Learning*, 12(4), 307–392. <https://doi.org/10.1561/22000000056>
- [58] Le-Khac, P. H., Healy, G., & Smeaton, A. F. (2020). Contrastive Representation Learning: A Framework and Review. *IEEE Access*, 8, 193907–193934. <https://doi.org/10.1109/ACCESS.2020.3031549>
- [59] MPK Mini mk3 MIDI Controller | Akai Pro. (n.d.). Retrieved November 2, 2025, from <https://www.akaipro.com/mpk-mini-mk3>

Appendix – A (preprocessing.py)

```
import muspy
import pickle
import os
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from h_event_processor import HEventProcessor
from collections import defaultdict
import pandas as pd
from random import shuffle
import random

# Seed for reproducibility
random.seed(42)
np.random.seed(42)

DATA_DIR = "/Users/kathi.s/ML_Projects/thesis/data/SymphonyNet"
ENCODED_PATH = "data/encoded/encoded_tokens_25_new.pkl"
LIST_SUB_DIRECTORIES = ["classical", "contemporary"]
MIN_FILES_PER_CLASS = 25

processor = HEventProcessor()
encoded_dataset = []
stats = defaultdict(list)

def analyze_and_visualize(df_song, df_instr):
    """Perform dataset-level analysis & visualization."""
    plt.figure(figsize=(14, 8))
    sns.set(style="whitegrid")

    # Top instruments
    plt.subplot(2, 2, 1)
    top_instr = df_instr["instrument"].value_counts().head(10)
    sns.barplot(x=top_instr.values, y=top_instr.index)
    plt.title("Top 10 Instruments in Dataset")

    # Song length distribution
    plt.subplot(2, 2, 2)
    sns.histplot(df_song["song_length"], bins=20)
    plt.title("Distribution of Song Lengths (time steps)")

    # Instruments per song (by style)
    plt.subplot(2, 2, 3)
    sns.boxplot(x="style", y="num_instruments", data=df_song)
    plt.title("Number of Instruments per Song")

    # Average pitch by style
    plt.subplot(2, 2, 4)
    sns.boxplot(x="style", y="avg_pitch", data=df_song)
    plt.title("Average Pitch by Style")

    plt.tight_layout()
    plt.savefig("data/encoded/analysis_report.png")
    plt.close()
    print("Saved analysis visualization → analysis_report.png")

    # Console summary
    print("\nAverage Song Statistics by Style:")
    print(df_song.groupby("style")[["song_length", "num_instruments", "avg_pitch", "avg_velocity"]].mean())

    print("\nTop Instruments:")
    print(top_instr)
```

```

def preprocess_dataset():
    """Load, encode, pickle, and analyze SymphonyNet data."""
    for subdir in os.listdir(DATA_DIR):
        if subdir in LIST_SUB_DIRECTORIES:
            files_list = os.listdir(os.path.join(DATA_DIR, subdir))
            shuffle(files_list)
            for file in files_list[:min(MIN_FILES_PER_CLASS, len(files_list))]:
                if not file.endswith(("mid", ".midi", ".musicxml")):
                    continue
                song_path = os.path.join(DATA_DIR, subdir, file)
                try:
                    # Assign a style ID based on folder
                    style_id = 0 if subdir == "classical" else 1
                    song = muspy.read_midi(song_path)
                    # setting muspy attribute for the style
                    song.__setattr__("style_id", style_id)
                    encoded = processor.encode(song, style_id)
                    encoded_dataset.append(encoded)
                    # Collect stats
                    stats["song_name"].append(file)
                    stats["num_instruments"].append(len(encoded["instr_level"]))
                    stats["song_length"].append(encoded["song_level"][3])
                    stats["avg_pitch"].append(np.mean(encoded["note_level"][:, 0]))
                    stats["avg_velocity"].append(np.mean(encoded["note_level"][:, 2]))
                    stats["style"].append(style_id)
                    stats["instrument"].extend(list(encoded["instr_level"]))

                except Exception as e:
                    print(f"Error processing {file}: {e}")

    # Pickle encoded dataset
    os.makedirs(os.path.dirname(ENCODED_PATH), exist_ok=True)
    with open(ENCODED_PATH, "wb") as f:
        pickle.dump(encoded_dataset, f)
    print(f"Pickled encoded dataset → {ENCODED_PATH}")

    # Song-level stats
    df_song = pd.DataFrame({
        "song_name": stats["song_name"],
        "num_instruments": stats["num_instruments"],
        "song_length": stats["song_length"],
        "avg_pitch": stats["avg_pitch"],
        "avg_velocity": stats["avg_velocity"],
        "style": stats["style"]
    })

    # Instrument-level stats
    df_instr = pd.DataFrame({
        "song_name": np.repeat(stats["song_name"], [len(enc["instr_level"]) for enc in encoded_dataset]),
        "instrument": [instr for enc in encoded_dataset for instr in enc["instr_level"]]
    })

    analyze_and_visualize(df_song, df_instr)

if __name__ == "__main__":
    preprocess_dataset()

```


Appendix – B (h_event_processor.py)

```
import muspy
import numpy as np

class HEventProcessor:
    def __init__(self, num_duration_bins=128, generate_expression=True):
        self.instrument_map = {}
        self.reverse_instrument_map = {i: i for i in range(128)}
        self.max_time = 0
        self.num_duration_bins = num_duration_bins
        self.generate_expression = generate_expression

    def encode(self, song: muspy.Music, style_id: 0):
        note_level, instr_level, song_level, control_level = [], [], [], []

        tempo = np.mean([int(t.qpm) for t in song.tempos]) if song.tempos else 120
        time_sig = song.time_signatures[0].numerator if song.time_signatures else 4
        song_level.append({
            "tempo": tempo,
            "time_signature": time_sig,
            "num_tracks": len(song.tracks),
            "length": song.get_end_time(),
            "style": style_id
        })

        raw_durations = list()

        for track in song.tracks:
            instr_id = track.program
            self.reverse_instrument_map[instr_id] = track.program

            instr_level.append({
                "instrument_id": instr_id,
                "program": track.program,
                "num_notes": len(track.notes),
            })

            for note in track.notes:
                raw_durations.append(note.duration)
                note_level.append({
                    "instrument_id": instr_id,
                    "pitch": note.pitch,
                    "start": note.start,
                    "duration": note.duration,
                    "velocity": note.velocity
                })
                self.max_time = max(self.max_time, note.end)

            # Capture control changes if available
            if hasattr(track, "control_changes"):
                for cc in track.control_changes:
                    control_level.append({
                        "instrument_id": instr_id,
                        "time": cc.time,
                        "control_number": cc.number,
                        "value": cc.value
                    })

        # Quantize durations (log-scaled)
        raw_durations = np.array([n["duration"] for n in note_level])
        if len(raw_durations) > 0:
            log_durs = np.log1p(raw_durations)
            log_durs = (log_durs / log_durs.max() * (self.num_duration_bins - 1)).astype(int)
```

```

else:
    log_durs = np.zeros(len(note_level), dtype=int)

    encoded_notes = self._encode_note_tokens(note_level, log_durs)
    encoded_instruments = encoded_notes[:, 3]
    encoded_song = np.array([tempo, time_sig, len(instr_level), self.max_time, style_id])

    encoded_controls = np.array([
        (c["instrument_id"], c["time"], c["control_number"], c["value"])
        for c in control_level
    ], dtype=np.int16) if len(control_level) > 0 else np.zeros((0, 4), dtype=np.int16)

    return {
        "note_level": encoded_notes,      # shape [num_notes, 4]
        "instr_level": encoded_instruments,
        "song_level": encoded_song,
        "control_level": encoded_controls
    }

def _encode_note_tokens(self, note_level, log_durs):
    """Return np array: [num_notes, 4] -> pitch, duration, velocity, instr_id"""
    return np.array([
        (n["pitch"], log_durs[i], n["velocity"], n["instrument_id"])
        for i, n in enumerate(note_level)
    ], dtype=np.int16)

def decode(self, encoded_dict):
    import muspy
    import numpy as np

    music = muspy.Music()
    note_level = encoded_dict["note_level"]

    if len(note_level) == 0:
        return music

    # Calculate actual durations from quantized values
    max_dur = max(note_level[:, 1]) if len(note_level) > 0 else 1
    max_log = np.log1p(max_dur)

    # Group by instrument and create tracks
    tracks = {}
    current_time = 0

    for i, note in enumerate(note_level):
        pitch, quantized_duration, velocity, instr_id = note
        duration = int(np.expml(quantized_duration / 127 * max_log))

        # Get or create track for this instrument
        if instr_id not in tracks:
            program = self.reverse_instrument_map.get(instr_id, 0)
            tracks[instr_id] = {
                'track': muspy.Track(program=program),
                'last_end_time': 0
            }

        track_info = tracks[instr_id]

        # Simple polyphonic timing: allow some overlap, but mostly sequential
        if i == 0:
            start_time = 0
        else:
            # 40% chance to create chord/overlap (use same start time as previous in this track)
            if np.random.random() < 0.4 and i > 0:
                start_time = track_info['last_end_time'] - 2 # Small overlap
            else:
                # Sequential placement with small gap
                start_time = track_info['last_end_time'] + np.random.randint(0, 4)

```

```

        start_time = max(0, start_time) # Ensure non-negative

        track_info['track'].notes.append(
            muspy.Note(
                time=int(start_time),
                pitch=int(pitch),
                duration=int(duration),
                velocity=int(velocity),
            )
        )

        # Update end time for this track
        track_info['last_end_time'] = start_time + duration

    # Add all tracks to music
    for track_info in tracks.values():
        music.tracks.append(track_info['track'])

    return music

def _add_emotional_controls(self, music):
    """Add heuristic dynamic and modulation controls to make playback more expressive."""
    for track in music.tracks:
        for note in track.notes:
            # Smooth sinusoidal dynamics
            expr = int(70 + 40 * np.sin(note.time / 50.0))
            mod = int(40 + 25 * np.cos(note.time / 70.0))

            track.control_changes.append(
                muspy.ControlChange(time=note.time, number=11, value=expr) # expression (dynamics)
            )
            track.control_changes.append(
                muspy.ControlChange(time=note.time, number=1, value=mod) # modulation (vibrato)
            )

```

Appendix – C (model.py)

```
import torch
import torch.nn as nn
import numpy as np
from torchinfo import summary

class HEventModel(nn.Module):
    """
    Hierarchical 3-layer LSTM for symbolic music modeling.
    Multi-feature input per note: [pitch, duration, velocity, instrument_id].
    """

    def __init__(self,
                 num_pitches=128,
                 num_durations=128,
                 num_velocities=128,
                 num_instruments=128,
                 style_classes=2,
                 embed_dim=256,
                 control_dim=128,
                 hidden_dim=512,
                 dropout=0.3,
                 device='mps'):
        super(HEventModel, self).__init__()

        self.hidden_dim = hidden_dim
        self.embed_dim = embed_dim

        # Separate embeddings for each feature
        self.pitch_embed = nn.Embedding(num_pitches, embed_dim, device=device)
        self.duration_embed = nn.Embedding(num_durations, embed_dim, device=device)
        self.velocity_embed = nn.Embedding(num_velocities, embed_dim, device=device)
        self.instrument_embed = nn.Embedding(num_instruments, embed_dim, device=device)
        self.style_embed = nn.Embedding(style_classes, embed_dim, device=device)

        # Projection layers to match dimensions
        self.embed_proj = nn.Linear(embed_dim, hidden_dim, device=device) # Project embeddings to hidden_dim
        self.style_proj = nn.Linear(embed_dim, hidden_dim, device=device)
        self.instr_proj = nn.Linear(embed_dim, hidden_dim, device=device)
        self.control_proj = nn.Linear(control_dim, hidden_dim, device=device)

        # 3-layer hierarchical LSTM
        self.lstm1 = nn.LSTM(hidden_dim, hidden_dim, batch_first=True, device=device) # Changed input to
hidden_dim
        self.lstm2 = nn.LSTM(hidden_dim, hidden_dim, batch_first=True, device=device)
        self.lstm3 = nn.LSTM(hidden_dim, hidden_dim, batch_first=True, device=device)

        self.dropout = nn.Dropout(dropout)

        # Output layers
        self.pitch_out = nn.Linear(hidden_dim, num_pitches, device=device)
        self.duration_out = nn.Linear(hidden_dim, num_durations, device=device)
        self.velocity_out = nn.Linear(hidden_dim, num_velocities, device=device)
        self.instrument_out = nn.Linear(hidden_dim, num_instruments, device=device)

        self.max_bins = num_durations

        self.use_style_conditioning = True
        self.use_instrument_conditioning = True
        self.use_control_conditioning = True
```

```

self.control_dim = control_dim

def forward(self, x, style=None, instr_context=None, control_context=None, hidden_states=None):
    batch_size, seq_len, _ = x.size()
    device = x.device

    # --- Extract input components ---
    pitch = x[:, :, 0].long()
    duration = x[:, :, 1].long()
    velocity = x[:, :, 2].long()
    instrument = x[:, :, 3].long()

    # --- Base embeddings ---
    x_embed = (
        self.pitch_embed(pitch) +
        self.duration_embed(duration) +
        self.velocity_embed(velocity) +
        self.instrument_embed(instrument)
    )
    x_embed = self.embed_proj(x_embed)

    # --- Prepare conditioning contexts ---
    style_emb = None
    instr_emb = None
    control_emb = None

    # STYLE conditioning (global, high-level) - LSTM1
    if style is not None and self.use_style_conditioning:
        style = style.long()
        if style.dim() == 1:
            style = style.unsqueeze(1).expand(-1, seq_len)
        style_emb = self.style_embed(style)
        style_emb = self.style_proj(style_emb)

    # INSTRUMENT conditioning (mid-level) - LSTM2
    if instr_context is not None and self.use_instrument_conditioning:
        instr_context = instr_context.long()
        if instr_context.dim() == 1:
            instr_context = instr_context.unsqueeze(0)
        instr_emb = self.instrument_embed(instr_context).mean(dim=1, keepdim=True)
        instr_emb = self.instr_proj(instr_emb)
        instr_emb = instr_emb.expand(batch_size, seq_len, -1)

    # CONTROL conditioning - handle the shape properly
    if control_context is not None and self.use_control_conditioning:
        # control_context shape: (batch_size, seq_len, control_dim)
        if control_context.dim() == 2:
            # If it's (batch_size, control_dim), expand to sequence
            control_context = control_context.unsqueeze(1).expand(-1, seq_len, -1)
        elif control_context.dim() == 3 and control_context.size(1) == 1:
            # If it's (batch_size, 1, control_dim), expand to sequence length
            control_context = control_context.expand(-1, seq_len, -1)

        # Project control context to hidden dimension
        control_emb = self.control_proj(control_context.float())

    # --- Hierarchical LSTM flow ---
    # LSTM1 → STYLE conditioning (global characteristics)
    lstm1_in = x_embed
    if style_emb is not None:
        lstm1_in = lstm1_in + style_emb
    out1, h1 = self.lstm1(lstm1_in, None if hidden_states is None else hidden_states[0])

    # LSTM2 → INSTRUMENT conditioning (instrument-specific patterns)
    lstm2_in = out1
    if instr_emb is not None:
        lstm2_in = lstm2_in + instr_emb
    out2, h2 = self.lstm2(lstm2_in, None if hidden_states is None else hidden_states[1])

```

```

# LSTM3 → CONTROL conditioning (expression, dynamics)
lstm3_in = out2
if control_emb is not None:
    lstm3_in = lstm3_in + control_emb
out3, h3 = self.lstm3(lstm3_in, None if hidden_states is None else hidden_states[2])

out3 = self.dropout(out3)

# --- Output projections ---
pitch_logits = self.pitch_out(out3)
dur_logits = self.duration_out(out3)
vel_logits = self.velocity_out(out3)
instr_logits = self.instrument_out(out3)

return pitch_logits, dur_logits, vel_logits, instr_logits, (h1, h2, h3)

```

```
class HLSTMTrainer:
```

```

    def __init__(self, model, lr=1e-3, device=None):
        self.device = device or ("cuda" if torch.cuda.is_available() else "cpu")
        self.model = model.to(self.device)
        self.optimizer = torch.optim.Adam(self.model.parameters(), lr=lr)
        self.criterion = nn.CrossEntropyLoss()

    def train_step(self, x, y, style=None, instr_context=None, control_context=None):
        self.model.train()
        x, y = x.to(self.device), y.to(self.device)
        if style is not None:
            style = style.to(self.device)
        if instr_context is not None:
            instr_context = instr_context.to(self.device)
        if control_context is not None:
            control_context = control_context.to(self.device)

        y_pitch = y[:, :, 0].long()
        y_dur = y[:, :, 1].long()
        y_vel = y[:, :, 2].long()
        y_instr = y[:, :, 3].long()

        self.optimizer.zero_grad()
        pitch_logits, dur_logits, vel_logits, instr_logits, _ = self.model(
            x, style=style, instr_context=instr_context, control_context=control_context
        )

        pitch_loss = self.criterion(pitch_logits.view(-1, pitch_logits.size(-1)), y_pitch.reshape(-1))
        dur_loss = self.criterion(dur_logits.view(-1, dur_logits.size(-1)), y_dur.reshape(-1))
        vel_loss = self.criterion(vel_logits.view(-1, vel_logits.size(-1)), y_vel.reshape(-1))
        instr_loss = self.criterion(instr_logits.view(-1, instr_logits.size(-1)), y_instr.reshape(-1))
        loss = (pitch_loss + dur_loss + vel_loss + instr_loss) / 4

        loss.backward()
        self.optimizer.step()
        return loss.item()

    def eval_step(self, x, y, style=None, instr_context=None, control_context=None):
        self.model.eval()
        with torch.no_grad():
            x, y = x.to(self.device), y.to(self.device)
            if style is not None:
                style = style.to(self.device)
            if instr_context is not None:
                instr_context = instr_context.to(self.device)
            if control_context is not None:
                control_context = control_context.to(self.device)

            y_pitch = y[:, :, 0].long()
            y_dur = y[:, :, 1].long()

```

```

y_vel = y[:, :, 2].long()
y_instr = y[:, :, 3].long()

pitch_logits, dur_logits, vel_logits, instr_logits, _ = self.model(
    x, style=style, instr_context=instr_context, control_context=control_context
)

pitch_loss = self.criterion(pitch_logits.view(-1, pitch_logits.size(-1)), y_pitch.reshape(-1))
dur_loss = self.criterion(dur_logits.view(-1, dur_logits.size(-1)), y_dur.reshape(-1))
vel_loss = self.criterion(vel_logits.view(-1, vel_logits.size(-1)), y_vel.reshape(-1))
instr_loss = self.criterion(instr_logits.view(-1, instr_logits.size(-1)), y_instr.reshape(-1))
loss = (pitch_loss + dur_loss + vel_loss + instr_loss) / 4

return loss.item()

def save(self, path):
    torch.save(self.model.state_dict(), path)

def load(self, path):
    self.model.load_state_dict(torch.load(path, map_location=self.device))

if __name__ == "__main__":
    model = HEventModel(device='cpu')

    x = torch.randint(0, 128, (32, 128, 4)) # note-level data
    style = torch.randint(0, 2, (32, 1)) # 2 style classes
    instr_context = torch.randint(0, 128, (32, 128)) # instrument context
    control_context = torch.randn(32, 128, 128) # continuous control data

    summary(model,
            input_data=(x, style, instr_context, control_context),
            col_names=["input_size", "output_size", "num_params", "params_percent", "kernel_size", "mult_adds",
                      "trainable"])

```

Appendix – D (train.py)

```
from torch.utils.data import Dataset, DataLoader, random_split
import torch
import pickle
import numpy as np
import os
from tqdm import tqdm
from model import HEventModel
from model import HLSTMTrainer
import math
from torch.utils.tensorboard import SummaryWriter

# -----
# Dataset for hierarchical 4-feature input
# -----
class MusicDataset(Dataset):
    def __init__(self, token_path, seq_len=128, use_control_context=True, control_dim=128):
        with open(token_path, "rb") as f:
            data_list = pickle.load(f)

        self.note_events = []
        self.style_levels = []
        self.instr_levels = []
        self.control_levels = []
        self.use_control_context = use_control_context
        self.control_dim = control_dim
        self.seq_len = seq_len

        for song in data_list:
            note_seq = np.array(song["note_level"], dtype=np.int64)
            control_seq = song.get("control_level", np.zeros((0, 4), dtype=np.int64))
            song_seq = song.get("song_level", [120, 4, 1, 100, 0])

            # Process control changes - now returns features for entire sequence
            control_features = self._extract_control_features(control_seq, len(note_seq))

            for i in range(len(note_seq) - seq_len):
                self.note_events.append(note_seq[i:i + seq_len])
                self.style_levels.append(song_seq[4])
                self.instr_levels.append(song_seq[2])

                if self.use_control_context and len(control_features) > 0:
                    # Get control features for this sequence window
                    start_idx = i
                    end_idx = i + seq_len
                    control_window = control_features[start_idx:end_idx]

                    # If we don't have enough control features, pad with zeros
                    if len(control_window) < seq_len:
                        padding = np.zeros((seq_len - len(control_window), self.control_dim),
dtype=np.float32)
                        control_window = np.concatenate([control_window, padding])

                    self.control_levels.append(control_window)
                else:
                    # Create zero tensor with shape (seq_len, control_dim)
                    self.control_levels.append(np.zeros((seq_len, self.control_dim), dtype=np.float32))

    def _extract_control_features(self, control_seq, num_notes):
        """Extract control features per note with dimension control_dim"""
        if len(control_seq) == 0:
            return np.zeros((num_notes, self.control_dim), dtype=np.float32)
```



```

control_features = []

# Define control codes we want to track
# You can expand this list based on what's in your data
control_codes = [1, 7, 11, 64, 71, 74] # modulation, volume, expression, sustain, resonance,
brightness

for i in range(num_notes):
    control_feat = np.zeros(self.control_dim, dtype=np.float32)

    # Fill in the first few dimensions with actual control values
    for j, cc_code in enumerate(control_codes):
        if j >= self.control_dim:
            break # Don't exceed our feature dimension

        cc_events = control_seq[control_seq[:, 2] == cc_code]
        if len(cc_events) > 0:
            # Use the most recent value
            control_feat[j] = cc_events[-1, 3] / 127.0 # normalize
        else:
            # Set reasonable defaults
            if cc_code == 7: # volume
                control_feat[j] = 0.8
            elif cc_code == 11: # expression
                control_feat[j] = 0.7
            else:
                control_feat[j] = 0.0

    # Remaining dimensions can be used for other features or left as zero
    control_features.append(control_feat)

return np.array(control_features)

def __len__(self):
    return len(self.note_events)

def __getitem__(self, idx):
    x = self.note_events[idx] # shape: (seq_len, 4)
    y = np.concatenate([self.note_events[idx][1:], self.note_events[idx][-1:]])

    style = self.style_levels[idx]
    instr_context = np.array([self.instr_levels[idx]], dtype=np.int64)
    control_context = self.control_levels[idx] # shape: (seq_len, control_dim)

    return (
        torch.tensor(x, dtype=torch.long),
        torch.tensor(y, dtype=torch.long),
        torch.tensor(style, dtype=torch.long),
        torch.tensor(instr_context, dtype=torch.long),
        torch.tensor(control_context, dtype=torch.float)
    )

# -----
# Metrics
# -----
def calculate_metrics(loss):
    perplexity = math.exp(loss) if loss < 20 else float("inf")
    return {"perplexity": perplexity}

# -----
# Training loop
# -----
def train_model(
    token_path,
    output_dir="models",
    log_dir="runs/hlstm",

```

```

        seq_len=128,
        batch_size=32,
        num_epochs=30,
        lr=1e-3,
        val_split=0.2,
        resume_checkpoint=None,
        patience=7
    ):
        os.makedirs(output_dir, exist_ok=True)
        writer = SummaryWriter(log_dir)

        dataset = MusicDataset(token_path, seq_len, use_control_context=True)
        val_size = int(len(dataset) * val_split)
        train_size = len(dataset) - val_size
        train_ds, val_ds = random_split(dataset, [train_size, val_size])

        train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)
        val_loader = DataLoader(val_ds, batch_size=batch_size, shuffle=False)

        model = HEventModel()
        vocab_size = (
            model.pitch_embed.num_embeddings +
            model.duration_embed.num_embeddings +
            model.velocity_embed.num_embeddings +
            model.instrument_embed.num_embeddings
        )
        print(f"[MODEL INFO]")
        print(f"  Pitch: {model.pitch_embed.num_embeddings}")
        print(f"  Duration: {model.duration_embed.num_embeddings}")
        print(f"  Velocity: {model.velocity_embed.num_embeddings}")
        print(f"  Instrument: {model.instrument_embed.num_embeddings}")
        print(f"  --> Effective combined vocab size: {vocab_size:,}")
        print("=" * 60)

        trainer = HLSTMTrainer(model, lr=lr, device="mps")

        # Resume checkpoint if provided
        start_epoch = 1
        if resume_checkpoint and os.path.exists(resume_checkpoint):
            trainer.load(resume_checkpoint)
            start_epoch = int(resume_checkpoint.split("_")[-1].split(".")[0]) + 1

        # ---- Early Stopping Variables ----
        best_val_loss = float("inf")
        epochs_no_improve = 0

        # ---- Training ----
        for epoch in range(start_epoch, num_epochs + 1):
            train_loss = 0.0
            val_loss = 0.0

            # -----
            # Training Loop
            # -----
            for x, y, style, instruments, controls in tqdm(train_loader, desc=f"[Epoch {epoch}] Training"):
                loss = trainer.train_step(x, y, style=style, instr_context=instruments,
                control_context=controls)
                train_loss += loss

            avg_train_loss = train_loss / len(train_loader)
            train_metrics = calculate_metrics(avg_train_loss)

            # -----
            # Validation Loop
            # -----
            for x, y, style, instruments, controls in tqdm(val_loader, desc=f"[Epoch {epoch}] Validation"):
                val_loss += trainer.eval_step(x, y, style=style, instr_context=instruments,
                control_context=controls)

```

```

avg_val_loss = val_loss / len(val_loader)
val_metrics = calculate_metrics(avg_val_loss)

print(f"Epoch {epoch:03d} | Train Loss: {avg_train_loss:.4f} | Val Loss: {avg_val_loss:.4f} | "
      f"Train PPL: {train_metrics['perplexity']:.2f} | Val PPL: {val_metrics['perplexity']:.2f}")

writer.add_scalar("Loss/Train", avg_train_loss, epoch)
writer.add_scalar("Loss/Validation", avg_val_loss, epoch)
writer.add_scalar("Perplexity/Train", train_metrics["perplexity"], epoch)
writer.add_scalar("Perplexity/Validation", val_metrics["perplexity"], epoch)

# -----
# Checkpoint & Early Stopping
# -----
if avg_val_loss < best_val_loss:
    best_val_loss = avg_val_loss
    epochs_no_improve = 0
    ckpt_path = os.path.join(output_dir, f"hlstm_epoch_{epoch}.pt")
    trainer.save(ckpt_path)
    print(f"Validation improved. Checkpoint saved: {ckpt_path}")
else:
    epochs_no_improve += 1
    ckpt_path = os.path.join(output_dir, f"hlstm_epoch_{epoch}_not_improved.pt")
    trainer.save(ckpt_path)
    print(f"No improvement for {epochs_no_improve} epoch(s).")

    if epochs_no_improve >= patience:
        print(f"Early stopping triggered after {patience} epochs of no improvement.")
        break

writer.close()
print("Training complete!")

if __name__ == "__main__":
    train_model(
        "data/encoded/encoded_tokens_25_new.pkl")

```

Appendix – E (generator.py)

```
from typing import List
import torch
import numpy as np
from h_event_processor import HEventProcessor
from model import HEventModel
import muspy

class MusicGenerator:
    def __init__(self, model: HEventModel, processor: HEventProcessor, device="mps"):
        self.model = model.to(device)
        self.processor = processor
        self.device = device
        self.model.eval()

    @torch.no_grad()
    def generate(
        self,
        prompt_tokens: np.ndarray, # shape: (seq_len, 4)
        style: int = 0,
        num_steps: int = 200, # how many notes to generate
        temperature: float = 0.5,
        top_k: int = 20,
        include_initial=False,
        allowed_instruments: List[int] = None # List of allowed instrument IDs
    ):
        """
        Generate a sequence of music tokens from a prompt.
        """
        if allowed_instruments is None:
            allowed_instruments = list(range(128)) # Allow all instruments if none specified

        if include_initial:
            generated = list(prompt_tokens)
        else:
            generated = []

        seq_len = 50
        hidden_states = None

        # Convert prompt to tensor
        if len(prompt_tokens) > 0:
            x = torch.tensor(prompt_tokens[-seq_len:], dtype=torch.long).unsqueeze(0).to(self.device)
        else:
            # Start with a dummy token if no prompt
            x = torch.zeros((1, 1, 4), dtype=torch.long).to(self.device)

        style_tensor = torch.tensor([style], dtype=torch.long).to(self.device)

        # Instrument context - use the allowed instruments
        instrument_tensor = torch.tensor([allowed_instruments], dtype=torch.long, device=self.device)

        print(f"[GENERATOR INFO]")
        print(f"  Allowed instruments: {allowed_instruments}")
        print(f"  Style: {style}")
        print(f"=" * 60)

        for step in range(num_steps):
            pitch_logits, dur_logits, vel_logits, instr_logits, hidden_states = self.model(
                x, style=style_tensor, instr_context=instrument_tensor, hidden_states=hidden_states
            )

            # Apply instrument masking - only allow specified instruments
            if allowed_instruments:
```

```

        mask = torch.ones_like(instr_logits) * -1e9
        mask[:, :, allowed_instruments] = 0 # Set allowed instruments to 0 (no masking)
        instr_logits = instr_logits + mask

    # Take only last token prediction
    pitch_logits = pitch_logits[:, -1, :] / temperature
    dur_logits = dur_logits[:, -1, :] / temperature
    vel_logits = vel_logits[:, -1, :] / temperature
    instr_logits = instr_logits[:, -1, :] / temperature

    # Sample each independently using top-k
    def sample_topk(logits):
        # Apply top-k filtering
        top_logits, top_idx = torch.topk(logits, k=min(top_k, logits.size(-1)), dim=-1)
        probs = torch.softmax(top_logits, dim=-1)
        sampled_idx = torch.multinomial(probs, num_samples=1)
        return top_idx[0, sampled_idx].item()

    pitch = sample_topk(pitch_logits)
    duration = sample_topk(dur_logits)
    velocity = sample_topk(vel_logits)
    instrument = sample_topk(instr_logits)

    next_note = np.array([pitch, duration, velocity, instrument], dtype=np.int16)
    generated.append(next_note)

    # Update context - use only the most recent notes
    recent_notes = np.array(generated[-seq_len:], dtype=np.int64)
    x = torch.tensor(recent_notes).unsqueeze(0).to(self.device)

    if step % 20 == 0:
        print(f"Step {step}: pitch={pitch}, dur={duration}, vel={velocity}, instr={instrument}")

    generated_tokens = np.array(generated, dtype=np.int16)
    print(f"Generated {len(generated_tokens)} notes")

    # Decode to MIDI
    midi_music = self.processor.decode({
        "note_level": generated_tokens,
        "instr_level": np.unique(generated_tokens[:, 3]),
        "song_level": [120, 4, len(np.unique(generated_tokens[:, 3])), generated_tokens.shape[0], style]
    })

    return midi_music, generated_tokens

@torch.no_grad()
def generate_natural(
    self,
    prompt_tokens: np.ndarray,
    style: int = 0,
    num_steps: int = 200,
    temperature: float = 0.8,
    top_k: int = 40,
    allowed_instruments: List[int] = None,
    include_initial=False,
    seq_len=50
):
    """
    Let the model generate naturally - it will create polyphony if it learned it.
    """
    if allowed_instruments is None:
        allowed_instruments = list(range(128))

    if include_initial:
        generated = list(prompt_tokens)
    else:
        generated = list()
    hidden_states = None

```

```

if len(prompt_tokens) > 0:
    x = torch.tensor(prompt_tokens[-seq_len:], dtype=torch.long).unsqueeze(0).to(self.device)
else:
    x = torch.zeros((1, 1, 4), dtype=torch.long).to(self.device)

style_tensor = torch.tensor([style], dtype=torch.long).to(self.device)
instrument_tensor = torch.tensor([allowed_instruments], dtype=torch.long, device=self.device)

print(f"Generating naturally with instruments: {allowed_instruments}")

for step in range(num_steps):
    pitch_logits, dur_logits, vel_logits, instr_logits, hidden_states = self.model(
        x, style=style_tensor, instr_context=instrument_tensor, hidden_states=hidden_states
    )

    # Apply instrument masking
    if allowed_instruments:
        mask = torch.ones_like(instr_logits) * -1e9
        mask[:, :, allowed_instruments] = 0
        instr_logits = instr_logits + mask

    # Take only last token prediction
    pitch_logits = pitch_logits[:, -1, :] / temperature
    dur_logits = dur_logits[:, -1, :] / temperature
    vel_logits = vel_logits[:, -1, :] / temperature
    instr_logits = instr_logits[:, -1, :] / temperature

    def sample_single(logits):
        top_logits, top_idx = torch.topk(logits, k=min(top_k, logits.size(-1)), dim=-1)
        probs = torch.softmax(top_logits, dim=-1)
        sampled_idx = torch.multinomial(probs, num_samples=1)
        return top_idx[0], sampled_idx.item()

    # Generate ONE note naturally - let the model decide the instrument
    pitch = sample_single(pitch_logits)
    duration = sample_single(dur_logits)
    velocity = sample_single(vel_logits)
    instrument = sample_single(instr_logits)

    next_note = np.array([pitch, duration, velocity, instrument], dtype=np.int16)
    generated.append(next_note)

    # Update context
    recent_notes = np.array(generated[-seq_len:], dtype=np.int64)
    x = torch.tensor(recent_notes).unsqueeze(0).to(self.device)

    if step % 25 == 0:
        current_instruments = np.unique([note[3] for note in generated[-20:]])
        print(f"Step {step}: pitch={pitch}, instr={instrument},
recent_instruments={list(current_instruments)}")

    generated_tokens = np.array(generated, dtype=np.int16)

    # Analyze what we got
    unique_instruments = np.unique(generated_tokens[:, 3])
    instrument_counts = {instr: np.sum(generated_tokens[:, 3] == instr) for instr in unique_instruments}

    print(f"Natural generation completed:")
    print(f" Total notes: {len(generated_tokens)}")
    print(f" Instruments used: {instrument_counts}")
    print(f" Most common instrument: {max(instrument_counts, key=instrument_counts.get)}")

    return self.processor.decode({
        "note_level": generated_tokens,
        "instr_level": np.unique(generated_tokens[:, 3]),
        "song_level": [120, 4, len(unique_instruments), generated_tokens.shape[0], style]
    }), generated_tokens

```

```

@torch.no_grad()
def generate_force_polyphonic(
    self,
    prompt_tokens: np.ndarray,
    style: int = 0,
    num_steps: int = 200,
    temperature: float = 0.7,
    allowed_instruments: List[int] = None,
    notes_per_chord: int = 3, # Force this many instruments per chord
    include_initial=False,
    seq_len: int = 64, # Added seq_len parameter
    top_k: int = 30, # Added top_k parameter (0 = no top-k filtering)
    top_p: float = 0.9 # Added top_p parameter (1.0 = no nucleus sampling)
):
    """
    Alternative approach that forces instrument diversity more aggressively.
    """
    if allowed_instruments is None:
        allowed_instruments = list(range(128))

    if include_initial:
        generated = list(prompt_tokens)
    else:
        generated = list()

    hidden_states = None

    if len(prompt_tokens) > 0:
        x = torch.tensor(prompt_tokens[-seq_len:], dtype=torch.long).unsqueeze(0).to(self.device)
    else:
        x = torch.zeros((1, 1, 4), dtype=torch.long).to(self.device)

    style_tensor = torch.tensor([style], dtype=torch.long).to(self.device)
    instrument_tensor = torch.tensor([allowed_instruments], dtype=torch.long, device=self.device)

    current_time = 0

    def sample_single(logits):
        """Sample a single value from logits with top-k and top-p filtering"""
        # Apply temperature
        logits = logits / temperature

        # Apply top-k filtering
        if top_k > 0:
            indices_to_remove = logits < torch.topk(logits, top_k)[0][..., -1, None]
            logits[indices_to_remove] = -float('Inf')

        # Apply top-p (nucleus) sampling
        if top_p < 1.0:
            sorted_logits, sorted_indices = torch.sort(logits, descending=True)
            cumulative_probs = torch.cumsum(torch.softmax(sorted_logits, dim=-1), dim=-1)

            # Remove tokens with cumulative probability above the threshold
            sorted_indices_to_remove = cumulative_probs > top_p
            # Shift the indices to the right to keep the first token above the threshold
            sorted_indices_to_remove[..., 1:] = sorted_indices_to_remove[..., :-1].clone()
            sorted_indices_to_remove[..., 0] = 0

            indices_to_remove = sorted_indices_to_remove.scatter(-1, sorted_indices,
sorted_indices_to_remove)
            logits[indices_to_remove] = -float('Inf')

        probs = torch.softmax(logits, dim=-1)
        sampled_idx = torch.multinomial(probs, num_samples=1)
        return sampled_idx.item()

    print(f"[GENERATOR CONFIG]")

```

```

print(f" Sequence length: {seq_len}")
print(f" Top-k: {top_k if top_k > 0 else 'disabled'}")
print(f" Top-p: {top_p if top_p < 1.0 else 'disabled'}")
print(f" Temperature: {temperature}")
print(f" Notes per chord: {notes_per_chord}")
print(f" Allowed instruments: {allowed_instruments}")
print("=" * 50)

for step in range(num_steps):
    pitch_logits, dur_logits, vel_logits, instr_logits, hidden_states = self.model(
        x, style=style_tensor, instr_context=instrument_tensor, hidden_states=hidden_states
    )

    # FORCE multiple instruments by sampling different ones
    pitch_logits = pitch_logits[:, -1, :]
    dur_logits = dur_logits[:, -1, :]
    vel_logits = vel_logits[:, -1, :]
    instr_logits = instr_logits[:, -1, :]

    # Apply instrument masking to logits before sampling
    if allowed_instruments:
        mask = torch.ones_like(instr_logits) * -1e9
        mask[:, allowed_instruments] = 0
        instr_logits = instr_logits + mask

    # Sample different instruments for this chord
    selected_instruments = []
    attempts = 0

    # First, try to sample from the model
    while len(selected_instruments) < notes_per_chord and attempts < 20:
        new_instr = sample_single(instr_logits)
        if new_instr not in selected_instruments and new_instr in allowed_instruments:
            selected_instruments.append(new_instr)
            attempts += 1

    # If we couldn't get enough unique instruments, fill with random allowed ones
    while len(selected_instruments) < notes_per_chord:
        remaining = [inst for inst in allowed_instruments if inst not in selected_instruments]
        if remaining:
            selected_instruments.append(np.random.choice(remaining))
        else:
            # If no more unique instruments, just duplicate (shouldn't happen with reasonable
notes_per_chord)
            break

    # Generate notes for each selected instrument
    chord_notes = []
    for instrument in selected_instruments:
        pitch = sample_single(pitch_logits)
        duration = sample_single(dur_logits)
        velocity = sample_single(vel_logits)

        next_note = np.array([pitch, duration, velocity, instrument], dtype=np.int16)
        chord_notes.append(next_note)
        generated.append(next_note)

    # Update context
    recent_notes = np.array(generated[-seq_len:], dtype=np.int64)
    x = torch.tensor(recent_notes).unsqueeze(0).to(self.device)

    # Advance time
    current_time += 4

    if step % 10 == 0:
        pitches = [note[0] for note in chord_notes]
        print(f"Step {step}: Chord with {len(selected_instruments)} instruments - "
              f"Instruments: {selected_instruments}, Pitches: {pitches}")

```



```

generated_tokens = np.array(generated, dtype=np.int16)

# Analyze results
unique_instruments = np.unique(generated_tokens[:, 3])
instrument_counts = {instr: np.sum(generated_tokens[:, 3] == instr) for instr in unique_instruments}

print(f"\n[GENERATION COMPLETE]")
print(f"  Total notes: {len(generated_tokens)}")
print(f"  Instruments used: {len(unique_instruments)}")
print(f"  Instrument distribution: {instrument_counts}")
print(f"  Sequence length used: {seq_len}")

return self.processor.decode({
    "note_level": generated_tokens,
    "instr_level": np.unique(generated_tokens[:, 3]),
    "song_level": [120, 4, len(unique_instruments), generated_tokens.shape[0], style]
}), generated_tokens

@torch.no_grad()
def generate_with_control_conditioning(
    self,
    prompt_tokens: np.ndarray,
    style: int = 0,
    num_steps: int = 200,
    temperature: float = 0.5,
    allowed_instruments: List[int] = None,
    notes_per_chord: int = 3,
    include_initial=False,
    seq_len: int = 50,
    top_k: int = 0,
    top_p: float = 1.0,
    control_context: np.ndarray = None
):
    """
    Generate music with control value conditioning
    """
    if allowed_instruments is None:
        allowed_instruments = list(range(128))

    if include_initial:
        generated = list(prompt_tokens)
    else:
        generated = list()

    hidden_states = None

    if len(prompt_tokens) > 0:
        x = torch.tensor(prompt_tokens[-seq_len:], dtype=torch.long).unsqueeze(0).to(self.device)
    else:
        x = torch.zeros((1, 1, 4), dtype=torch.long).to(self.device)

    style_tensor = torch.tensor([style], dtype=torch.long).to(self.device)
    instrument_tensor = torch.tensor([allowed_instruments], dtype=torch.long, device=self.device)

    # Prepare control context
    if control_context is not None:
        if isinstance(control_context, list):
            control_context = np.array(control_context, dtype=np.float32)

        # Create control vector with 128 dimensions
        control_vector = np.zeros(128, dtype=np.float32)

        # Map provided control values
        if len(control_context) >= 1:
            control_vector[0] = control_context[0] # modulation
        if len(control_context) >= 2:
            control_vector[1] = control_context[1] # volume

```

```

if len(control_context) >= 3:
    control_vector[2] = control_context[2] # expression
if len(control_context) >= 4:
    control_vector[3] = control_context[3] # sustain

control_tensor = torch.tensor(control_vector, dtype=torch.float).to(self.device)
control_tensor = control_tensor.unsqueeze(0).unsqueeze(0) # (1, 1, 128)
control_tensor = control_tensor.expand(1, seq_len, -1) # (1, seq_len, 128)

else:
    # Default neutral control values
    control_vector = np.zeros(128, dtype=np.float32)
    control_vector[0] = 0.0 # modulation
    control_vector[1] = 0.7 # volume
    control_vector[2] = 0.7 # expression
    control_vector[3] = 0.0 # sustain

    control_tensor = torch.tensor(control_vector, dtype=torch.float).to(self.device)
    control_tensor = control_tensor.unsqueeze(0).unsqueeze(0).expand(1, seq_len, -1)

current_time = 0

def sample_single(logits):
    """Sample a single value from logits with top-k and top-p filtering"""
    # Ensure logits are 1D or 2D for multinomial
    if logits.dim() > 2:
        logits = logits.squeeze(0) # Remove batch dimension if present

    logits = logits / temperature

    if top_k > 0:
        # For 2D logits, apply top-k along the last dimension
        if logits.dim() == 2:
            top_logits, top_idx = torch.topk(logits, k=min(top_k, logits.size(-1)), dim=-1)
            filtered_logits = torch.full_like(logits, -float('Inf'))
            filtered_logits.scatter_(-1, top_idx, top_logits)
            logits = filtered_logits
        else:
            # For 1D logits
            top_logits, top_idx = torch.topk(logits, k=min(top_k, logits.size(-1)))
            filtered_logits = torch.full_like(logits, -float('Inf'))
            filtered_logits[top_idx] = top_logits
            logits = filtered_logits

    if top_p < 1.0:
        if logits.dim() == 2:
            # Apply top-p to each row for 2D tensors
            sorted_logits, sorted_indices = torch.sort(logits, descending=True, dim=-1)
            cumulative_probs = torch.cumsum(torch.softmax(sorted_logits, dim=-1), dim=-1)
            sorted_indices_to_remove = cumulative_probs > top_p
            sorted_indices_to_remove[..., 1:] = sorted_indices_to_remove[..., :-1].clone()
            sorted_indices_to_remove[..., 0] = 0
            indices_to_remove = sorted_indices_to_remove.scatter(-1, sorted_indices,
sorted_indices_to_remove)
            logits[indices_to_remove] = -float('Inf')
        else:
            # Apply top-p to 1D tensor
            sorted_logits, sorted_indices = torch.sort(logits, descending=True)
            cumulative_probs = torch.cumsum(torch.softmax(sorted_logits, dim=-1), dim=-1)
            sorted_indices_to_remove = cumulative_probs > top_p
            sorted_indices_to_remove[1:] = sorted_indices_to_remove[:-1].clone()
            sorted_indices_to_remove[0] = 0
            indices_to_remove = sorted_indices[sorted_indices_to_remove]
            logits[indices_to_remove] = -float('Inf')

    probs = torch.softmax(logits, dim=-1)

    # Ensure probs is 1D for multinomial

```

```

    if probs.dim() == 2:
        probs = probs.squeeze(0) # Remove batch dimension

    sampled_idx = torch.multinomial(probs, num_samples=1)
    return sampled_idx.item()

print(f"[GENERATOR CONFIG]")
if control_context is not None:
    print(f"  Control conditioning: {control_context.tolist()}")
else:
    print(f"  Control conditioning: default")
print(f"  Style: {style}")
print(f"  Allowed instruments: {allowed_instruments}")
print(f"  Notes per chord: {notes_per_chord}")
print(f"  Temperature: {temperature}")
print(f"  Top-k: {top_k if top_k > 0 else 'disabled'}")
print(f"  Top-p: {top_p if top_p < 1.0 else 'disabled'}")
print(f"=" * 50)

for step in range(num_steps):
    pitch_logits, dur_logits, vel_logits, instr_logits, hidden_states = self.model(
        x,
        style=style_tensor,
        instr_context=instrument_tensor,
        control_context=control_tensor,
        hidden_states=hidden_states
    )

    # Apply instrument masking
    if allowed_instruments:
        mask = torch.ones_like(instr_logits) * -1e9
        mask[:, :, allowed_instruments] = 0
        instr_logits = instr_logits + mask

    # Take only the last timestep predictions
    pitch_logits_last = pitch_logits[:, -1, :] # Shape: (1, num_pitches)
    dur_logits_last = dur_logits[:, -1, :] # Shape: (1, num_durations)
    vel_logits_last = vel_logits[:, -1, :] # Shape: (1, num_velocities)
    instr_logits_last = instr_logits[:, -1, :] # Shape: (1, num_instruments)

    # Sample different instruments for this chord
    selected_instruments = []
    attempts = 0

    while len(selected_instruments) < notes_per_chord and attempts < 20:
        new_instr = sample_single(instr_logits_last)
        if new_instr not in selected_instruments and new_instr in allowed_instruments:
            selected_instruments.append(new_instr)
            attempts += 1

    # If we couldn't get enough unique instruments, fill with random allowed ones
    while len(selected_instruments) < notes_per_chord:
        remaining = [inst for inst in allowed_instruments if inst not in selected_instruments]
        if remaining:
            selected_instruments.append(np.random.choice(remaining))
        else:
            # If no more unique instruments, break
            break

    # Generate notes for each selected instrument
    chord_notes = []
    for instrument in selected_instruments:
        pitch = sample_single(pitch_logits_last)
        duration = sample_single(dur_logits_last)
        velocity = sample_single(vel_logits_last)

        next_note = np.array([pitch, duration, velocity, instrument], dtype=np.int16)
        chord_notes.append(next_note)

```

```

        generated.append(next_note)

    # Update context with the most recent notes
    recent_notes = np.array(generated[-seq_len:], dtype=np.int64)
    if len(recent_notes) < seq_len:
        # Pad if we don't have enough notes yet
        padding = np.zeros((seq_len - len(recent_notes), 4), dtype=np.int64)
        recent_notes = np.concatenate([recent_notes, padding])

    x = torch.tensor(recent_notes, dtype=torch.long).unsqueeze(0).to(self.device)

    # Advance time
    current_time += 4

    if step % 10 == 0:
        pitches = [note[0] for note in chord_notes]
        print(f"Step {step}: Instruments {selected_instruments}, Pitches {pitches}")

generated_tokens = np.array(generated, dtype=np.int16)

# Analyze the generated output
unique_instruments = np.unique(generated_tokens[:, 3])
instrument_counts = {instr: np.sum(generated_tokens[:, 3] == instr) for instr in unique_instruments}

print(f"\n[GENERATION COMPLETE]")
print(f" Total notes generated: {len(generated_tokens)}")
print(f" Instruments used: {len(unique_instruments)}")
print(f" Instrument distribution: {instrument_counts}")

# Add control changes to the output based on conditioning
midi_music = self._decode_with_controls(generated_tokens, style, control_context)

return midi_music, generated_tokens

def _decode_with_controls(self, generated_tokens, style, control_context):
    """Decode tokens to MIDI with added control changes based on conditioning"""
    # First decode normally using the processor
    midi_music = self.processor.decode({
        "note_level": generated_tokens,
        "instr_level": np.unique(generated_tokens[:, 3]),
        "song_level": [120, 4, len(np.unique(generated_tokens[:, 3])), generated_tokens.shape[0], style]
    })

    # Add control changes based on the conditioning
    if control_context is not None:
        self._add_conditioned_controls(midi_music, control_context)

    return midi_music

def _add_conditioned_controls(self, music, control_context):
    """Add control changes based on the conditioning values"""
    control_mapping = {
        0: 1, # modulation
        1: 7, # volume
        2: 11, # expression
        3: 64, # sustain
    }

    # Add control changes to all tracks
    for track in music.tracks:
        # Clear any existing control changes to avoid conflicts
        if hasattr(track, 'control_changes'):
            track.control_changes.clear()

        # Add control changes based on conditioning values
        for control_idx, control_value in enumerate(control_context):
            if control_idx in control_mapping and control_value > 0:
                control_number = control_mapping[control_idx]

```

```

        midi_value = int(control_value * 127)
        if not hasattr(track, 'control_changes'):
            track.control_changes = []
        # Add control change at the beginning
        track.control_changes.append({
            'time': 0,
            'number': control_number,
            'value': midi_value
        })

        # Optionally add periodic control changes for more dynamic expression
        if control_number in [1, 11]: # For modulation and expression
            for time_point in range(50, 200, 50):
                # Add slight variations to make it more natural
                varied_value = max(0, min(127, midi_value + np.random.randint(-10, 11)))
                track.control_changes.append({
                    'time': time_point,
                    'number': control_number,
                    'value': varied_value
                })

# Add control presets for easy use
CONTROL_PRESETS = {
    "expressive": [0.8, 0.7, 0.9, 0.3], # Lots of modulation and expression
    "gentle": [0.2, 0.6, 0.7, 0.1], # Soft and delicate
    "bright": [0.4, 0.8, 0.8, 0.2], # Clear and vibrant
    "sustained": [0.3, 0.7, 0.7, 0.9], # Long notes with sustain
    "percussive": [0.1, 0.8, 0.6, 0.0], # Short, punchy notes
    "dreamy": [0.9, 0.5, 0.8, 0.7], # Lots of modulation, medium sustain
    "neutral": [0.0, 0.7, 0.7, 0.0], # Default neutral values
    "nothing": [0.0, 0.0, 0.0, 0.0]
}

def generate_with_preset(self, preset_name: str, **kwargs):
    """Generate using a predefined control preset"""
    if preset_name not in self.CONTROL_PRESETS:
        available_presets = list(self.CONTROL_PRESETS.keys())
        raise ValueError(f"Unknown preset '{preset_name}'. Available presets: {available_presets}")

    control_context = self.CONTROL_PRESETS[preset_name]
    print(f"Using preset: {preset_name} - {control_context}")

    kwargs["control_context"] = control_context

    return self.generate_with_control_conditioning(
        **kwargs
    )

def method_1(self, **kwargs):
    return self.generate(**kwargs)

def method_2(self, **kwargs):
    return self.generate_natural(**kwargs)

def method_3(self, **kwargs):
    return self.generate_force_polyphonic(**kwargs)

def method_4(self, **kwargs):
    return self.generate_with_preset(**kwargs)

# Add this method to your MusicGenerator class
def list_control_presets(self):
    """List all available control presets"""
    print("Available control presets:")
    for preset_name, values in self.CONTROL_PRESETS.items():
        print(f"    {preset_name}: {values}")

```

Appendix – F (experimentation.py)

```
import muspy
from h_event_processor import HEventProcessor
from model import HEventModel
from generator import MusicGenerator
import torch
import numpy as np
from visualize_metrics import GeneratorMetrics
import matplotlib.pyplot as plt
import wav_composer as wc

np.random.seed(42)

# Load model & processor
processor = HEventProcessor()

INPUT_FILE_PATH = "test/20centuryfox.mid"

song = muspy.read_midi(INPUT_FILE_PATH)
d = processor.encode(song, 2)

multi = muspy.to_pypianoroll(song)
multi.plot(
    mode="hybrid", # 'separate' for stacked, 'same' for overlaid
    track_label="program", # show instrument names
    preset="frame", # color preset ('frame', 'full', 'plain', etc.)
)

plt.savefig("assets/master_piano_roll.png", dpi=300, bbox_inches='tight')

wc.write_mid_and_wav(
    music=song,
    write_mid=False,
    mid_file_path=INPUT_FILE_PATH,
    wav_file_path=f"assets/master.wav"
)

model = HEventModel()
model.load_state_dict(torch.load("models/hlstm_epoch_9.pt", map_location="mps"))

generator = MusicGenerator(model, processor, device="mps")

# Example prompt
prompt_tokens = d['note_level']

# LIST_OF_INSTRUMENTS = [56, 57, 58, 42, 60, 114]
LIST_OF_INSTRUMENTS = [40, 41, 42, 43]

"""
CONTROL_PRESETS = {
    "expressive": [0.8, 0.7, 0.9, 0.3], # Lots of modulation and expression
    "gentle": [0.2, 0.6, 0.7, 0.1], # Soft and delicate
    "bright": [0.4, 0.8, 0.8, 0.2], # Clear and vibrant
    "sustained": [0.3, 0.7, 0.7, 0.9], # Long notes with sustain
    "percussive": [0.1, 0.8, 0.6, 0.0], # Short, punchy notes
    "dreamy": [0.9, 0.5, 0.8, 0.7], # Lots of modulation, medium sustain
    "neutral": [0.0, 0.7, 0.7, 0.0], # Default neutral values
    "nothing": [0.0, 0.0, 0.0, 0.0]
}
"""

PRESET = "gentle"
NUM_SAMPLES = 3
STYLE = 1
```

```

def plot_metrics(gen):
    for i in range(1, 5):
        # we are calling all the method starting from method 1

        gm = GeneratorMetrics(
            generator=gen,
            preset_name=PRESET,
            style=STYLE,
            instruments_used=LIST_OF_INSTRUMENTS,
            prompt_tokens=prompt_tokens,
            num_generations=NUM_SAMPLES,
            num_steps=100,
            control_context=[],
            notes_per_chord=3,
            temperature=0.7,
            include_initial=False,
            top_k=50,
            top_p=0.9,
            generation_method=i,
            seq_length=128
        )
        gm.compare_multiple_generations()

plot_metrics(generator)

```

Appendix – G (metrics.py)

```
import muspy
import numpy as np
from scipy.spatial.distance import pdist, squareform

class MusicGenerationMetrics:
    def __init__(self, instruments_used=None, style=None, window_size=128):
        self.instruments_used = instruments_used
        self.style = style
        self.window_size = window_size

    def self_similarity_matrix_score(self, music, window_size=16):
        """Self-Similarity Matrix Score for structural coherence"""
        window_size = self.window_size
        # Extract note sequences per track
        sequences = []
        for track in music.tracks:
            if len(track.notes) == 0:
                continue
            track_notes = []
            for note in track.notes:
                track_notes.append({
                    'pitch': note.pitch,
                    'time': note.time,
                    'duration': note.duration
                })
            sequences.append(track_notes)

        if not sequences:
            return 0.0 # No tracks with notes

        # Create pitch-time vectors for each window
        vectors = []
        max_time = music.get_end_time()

        # If no duration, return 0
        if max_time == 0:
            return 0.0

        for start in range(0, max_time, window_size):
            window_vector = []
            for track_seq in sequences:
                # Count notes in this window
                notes_in_window = sum(1 for note in track_seq
                                     if start <= note['time'] < start + window_size)
                window_vector.append(notes_in_window)
            vectors.append(window_vector)

        # Need at least 2 vectors for similarity
        if len(vectors) < 2:
            return 0.0

        try:
            vectors_array = np.array(vectors)

            # Check if we have any variation (all zeros would cause issues)
            if np.all(vectors_array == 0):
                return 0.0

            # Compute similarity matrix
            similarity_matrix = 1 - squareform(pdist(vectors_array, metric='cosine'))

            # Handle NaN values that might occur
```



```

        if np.any(np.isnan(similarity_matrix)):
            return 0.0

        # Score based on diagonal consistency
        diagonal_strength = np.mean([similarity_matrix[i, i + 1] for i in range(len(vectors) - 1)])

        # Handle potential NaN
        return float(diagonal_strength) if not np.isnan(diagonal_strength) else 0.0

    except Exception as e:
        print(f"DEBUG SSM: Error in similarity calculation: {e}")
        return 0.0

def inter_onset_interval_variance(self, music):
    """IOI Variance for rhythmic consistency (Layer 2 validation)"""
    all_iois = []

    for track in music.tracks:
        if len(track.notes) < 2:
            continue

        # Sort notes by time
        sorted_notes = sorted(track.notes, key=lambda x: x.time)

        # Calculate IOIs
        for i in range(1, len(sorted_notes)):
            ioi = sorted_notes[i].time - sorted_notes[i - 1].time
            if ioi > 0: # Ignore simultaneous notes
                all_iois.append(ioi)

    if len(all_iois) < 2:
        return 0.0

    # Lower variance = more rhythmically consistent
    return np.var(all_iois)

def instrument_usage_ratio(self, music, target_instruments=None):
    """Instrument Usage Ratio for conditional fidelity (Layer 3 validation)"""
    target_instruments = target_instruments
    if target_instruments is None:
        target_instruments = []

    used_instruments = set()
    total_notes = 0
    target_notes = 0

    for track in music.tracks:
        instr = track.program
        used_instruments.add(instr)
        note_count = len(track.notes)
        total_notes += note_count

        if instr in target_instruments:
            target_notes += note_count

    if total_notes == 0:
        return 0.0

    # Ratio of notes using target instruments
    iur = target_notes / total_notes

    # Penalty for using non-target instruments
    extra_instruments = len(used_instruments - set(target_instruments))
    penalty = 1.0 / (1.0 + extra_instruments)

    return iur * penalty

def style_perplexity_score(self, music, style_model=None):

```

```

"""Cross-Entropy Perplexity-like score for style adherence"""
# This would require a pre-trained style classifier
# For now, use musical features as proxy
features = []

# Extract style-related features without muspy.note_density
features.append(muspy.pitch_range(music))

# Calculate note density manually
total_notes = sum(len(track.notes) for track in music.tracks)
total_time = music.get_end_time()
note_density = total_notes / max(total_time, 1) # Avoid division by zero
features.append(note_density)

features.append(self.inter_onset_interval_variance(music))

# Normalize features (you'd need to calibrate these based on your style data)
normalized_features = [
    min(features[0] / 60.0, 1.0), # Pitch range normalization
    min(features[1] / 0.5, 1.0), # Note density normalization (notes per tick)
    min(features[2] / 100.0, 1.0) # Rhythm variance normalization
]

# Lower score = more stylistically consistent (like perplexity)
style_score = np.mean(normalized_features)
return style_score

# Muspy has these built-in:
def get_muspy_metrics(self, music):
    """Get available muspy metrics"""
    metrics = {}

    # Pitch-related (these should exist in muspy)
    try:
        metrics['pitch_range'] = muspy.pitch_range(music)
        metrics['pitch_entropy'] = muspy.pitch_entropy(music)
        metrics['scale_consistency'] = muspy.scale_consistency(music)
    except AttributeError as e:
        print(f"Warning: Some pitch metrics not available: {e}")

    # Rhythm-related - only use available ones
    try:
        if hasattr(muspy, 'rhythm_consistency'):
            metrics['rhythm_consistency'] = muspy.rhythm_consistency(music)
            # Skip gross_rhythm_consistency since it doesn't exist
    except AttributeError as e:
        print(f"Warning: Rhythm metrics not available: {e}")

    # Note density (calculate manually since muspy.note_density doesn't exist)
    total_notes = sum(len(track.notes) for track in music.tracks)
    total_time = max(music.get_end_time(), 1) # Avoid division by zero
    metrics['note_density'] = total_notes / total_time

    return metrics

def compute_all_metrics(self, music, target_instruments=None, style_model=None):
    """Compute all metrics for comprehensive evaluation"""
    metrics = {}

    # Layer 1: Structural Coherence
    metrics['ssm_score'] = self.self_similarity_matrix_score(music)

    # Layer 2: Rhythmic Consistency
    # Layer 2: Rhythmic Consistency
    metrics['ioi_variance'] = self.inter_onset_interval_variance(music)

    # Layer 3: Conditional Fidelity
    metrics['instrument_usage_ratio'] = self.instrument_usage_ratio(

```

```
        music, target_instruments
    )

    # Style Adherence
    metrics['style_score'] = self.style_perplexity_score(music, style_model)

    # Add muspy built-in metrics
    muspy_metrics = self.get_muspy_metrics(music)
    metrics.update(muspy_metrics)

    return metrics
```

Appendix – H (visualize_metrics.py)

```
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import numpy as np
import os

from generator import MusicGenerator
from metrics import MusicGenerationMetrics
import wav_composer as wc

class MusicVisualizer:
    def __init__(self, output_dir="assets/generation_plots", gen_method=1):
        self.output_dir = output_dir
        os.makedirs(output_dir, exist_ok=True)
        os.makedirs(f"{output_dir}/{gen_method}", exist_ok=True)

    def plot_music_timeseries(self, music, title="Generated Music", filename="music_timeseries.png"):
        """Plot music as a piano roll timeseries"""
        fig, ax = plt.subplots(figsize=(15, 10))

        colors = plt.cm.Set3(np.linspace(0, 1, len(music.tracks)))

        for i, track in enumerate(music.tracks):
            if len(track.notes) == 0:
                continue

            color = colors[i]
            instrument_name = f"Instr_{track.program}"

            for note in track.notes:
                # Create rectangle for each note
                rect = patches.Rectangle(
                    (note.time, note.pitch - 0.4), # x, y (time, pitch)
                    note.duration, 0.8, # width, height
                    linewidth=1, edgecolor='black',
                    facecolor=color, alpha=0.7,
                    label=instrument_name if i == 0 else ""
                )
                ax.add_patch(rect)

            # Customize plot
            ax.set_xlabel('Time (ticks)')
            ax.set_ylabel('Pitch')
            ax.set_title(f'{title}\nTotal Notes: {sum(len(track.notes) for track in music.tracks)}')
            ax.grid(True, alpha=0.3)
            ax.set_ylim(0, 128)

            # Add legend
            handles, labels = ax.get_legend_handles_labels()
            by_label = dict(zip(labels, handles))
            ax.legend(by_label.values(), by_label.values(), title='Instruments')

        plt.tight_layout()
        plt.savefig(os.path.join(self.output_dir, filename), dpi=300, bbox_inches='tight')
        plt.close()

    def plot_velocity_timeseries(self, music, title="Velocity Over Time",
                                filename="velocity_timeseries.png"):
        """Plot velocity as a timeseries"""
        fig, ax = plt.subplots(figsize=(15, 10))

        for i, track in enumerate(music.tracks):
```

```

        if len(track.notes) == 0:
            continue

        times = [note.time for note in track.notes]
        velocities = [note.velocity for note in track.notes]

        ax.scatter(times, velocities, alpha=0.7, label=f'Instr_{track.program}', s=30)

    ax.set_xlabel('Time (ticks)')
    ax.set_ylabel('Velocity')
    ax.set_title(title)
    ax.legend()
    ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig(os.path.join(self.output_dir, filename), dpi=300, bbox_inches='tight')
    plt.close()

class MetricsPlotter:
    def __init__(self, output_dir="assets/metrics_plots", gen_method=1):
        self.output_dir = output_dir
        os.makedirs(output_dir, exist_ok=True)
        os.makedirs(f"{output_dir}/{gen_method}", exist_ok=True)

    def plot_single_metrics(self, metrics, title="Generation Metrics", filename="single_metrics.png"):
        """Plot metrics for a single generation"""
        # Categorize metrics
        structural_metrics = {
            'ssm_score': 'Structural Coherence',
            'scale_consistency': 'Scale Consistency'
        }

        rhythmic_metrics = {
            'ioi_variance': 'Rhythmic Consistency',
            'gross_rhythm_consistency': 'Gross Rhythm Consistency'
        }

        conditional_metrics = {
            'instrument_usage_ratio': 'Instrument Usage Ratio',
            'pitch_range': 'Pitch Range'
        }

        style_metrics = {
            'style_score': 'Style Adherence',
            'pitch_entropy': 'Pitch Entropy'
        }

        # Create subplots
        fig, axes = plt.subplots(2, 2, figsize=(15, 10))
        axes = axes.flatten()

        # Plot each category
        categories = [
            (structural_metrics, 'Structural Metrics', 'blue', axes[0]),
            (rhythmic_metrics, 'Rhythmic Metrics', 'green', axes[1]),
            (conditional_metrics, 'Conditional Metrics', 'orange', axes[2]),
            (style_metrics, 'Style Metrics', 'red', axes[3])
        ]

        for metric_dict, category_title, color, ax in categories:
            available_metrics = {k: v for k, v in metric_dict.items() if k in metrics}
            if available_metrics:
                names = list(available_metrics.values())
                values = [metrics[k] for k in available_metrics.keys()]

                bars = ax.bar(names, values, color=color, alpha=0.7)
                ax.set_title(category_title)

```

```

        ax.set_ylabel('Score')

        # Add value labels on bars
        for bar, value in zip(bars, values):
            height = bar.get_height()
            ax.text(bar.get_x() + bar.get_width() / 2., height + 0.01,
                    f'{value:.3f}', ha='center', va='bottom', fontsize=9)

        ax.tick_params(axis='x', rotation=45)
        ax.grid(True, alpha=0.3)

    plt.suptitle(title, fontsize=16)
    plt.tight_layout()
    plt.savefig(os.path.join(self.output_dir, filename), dpi=300, bbox_inches='tight')
    plt.close()

def plot_comparison_metrics(self, metrics_list, labels, title="Metrics Comparison",
                           filename="comparison_metrics.png"):
    """Plot metrics comparison across multiple generations"""
    if not metrics_list:
        return

    # Get all available metrics
    all_metrics = set()
    for metrics in metrics_list:
        all_metrics.update(metrics.keys())
    all_metrics = sorted(all_metrics)

    # Create radar plot for comparison
    fig, ax = plt.subplots(figsize=(15, 10), subplot_kw=dict(projection='polar'))

    # Normalize metrics for radar plot
    normalized_metrics = []
    for metrics in metrics_list:
        normalized = {}
        for metric in all_metrics:
            if metric in metrics:
                # Simple normalization (you might want more sophisticated normalization)
                normalized[metric] = min(metrics[metric] / 1.0, 1.0) # Assuming max score of 1.0
            else:
                normalized[metric] = 0.0
        normalized_metrics.append(normalized)

    # Prepare angles
    angles = np.linspace(0, 2 * np.pi, len(all_metrics), endpoint=False).tolist()
    angles += angles[:1] # Complete the circle

    # Plot each generation
    colors = plt.cm.viridis(np.linspace(0, 1, len(metrics_list)))

    for i, (norm_metrics, label, color) in enumerate(zip(normalized_metrics, labels, colors)):
        values = [norm_metrics[metric] for metric in all_metrics]
        values += values[:1] # Complete the circle

        ax.plot(angles, values, 'o-', linewidth=2, label=label, color=color, alpha=0.7)
        ax.fill(angles, values, alpha=0.1, color=color)

    # Add metric names
    ax.set_xticks(angles[:-1])
    ax.set_xticklabels(all_metrics, fontsize=8)
    ax.set_ylim(0, 1)
    ax.set_title(title, size=16, y=1.08)
    ax.legend(loc='upper right', bbox_to_anchor=(1.3, 1.0))

    plt.tight_layout()
    plt.savefig(os.path.join(self.output_dir, filename), dpi=300, bbox_inches='tight')
    plt.close()

```

```

def plot_metrics_timeseries(self, metrics_history, title="Metrics Over Time",
filename="metrics_timeseries.png"):
    """Plot metrics evolution over multiple generations"""
    if not metrics_history:
        return

    # Extract metrics over time
    metrics_over_time = {}
    for step_metrics in metrics_history:
        for metric, value in step_metrics.items():
            if metric not in metrics_over_time:
                metrics_over_time[metric] = []
            metrics_over_time[metric].append(value)

    # Plot
    fig, axes = plt.subplots(2, 2, figsize=(15, 10))
    axes = axes.flatten()

    metrics_to_plot = ['ssm_score', 'ioi_variance', 'instrument_usage_ratio', 'style_score']

    for i, metric in enumerate(metrics_to_plot):
        if metric in metrics_over_time and len(metrics_over_time[metric]) > 1:
            ax = axes[i]
            steps = range(len(metrics_over_time[metric]))
            ax.plot(steps, metrics_over_time[metric], 'o-', linewidth=2, markersize=4)
            ax.set_title(f'{metric.replace("_", " ").title()}')
            ax.set_xlabel('Generation Step')
            ax.set_ylabel('Score')
            ax.grid(True, alpha=0.3)

    plt.suptitle(title, fontsize=16)
    plt.tight_layout()
    plt.savefig(os.path.join(self.output_dir, filename), dpi=300, bbox_inches='tight')
    plt.close()

class GeneratorMetrics:
    def __init__(self,
generator: MusicGenerator = None,
preset_name=None,
style=None,
instruments_used=None,
prompt_tokens=None,
num_generations=3,
num_steps=100,
control_context=[],
notes_per_chord=2,
temperature=0.7,
include_initial=False,
top_k=20,
top_p=0.9,
generation_method=0,
seq_length=128
):
        self.preset_name = preset_name
        self.style = style
        self.instruments_used = instruments_used
        self.prompt_tokens = prompt_tokens
        self.num_generations = num_generations
        self.generator = generator
        self.num_steps = num_steps
        self.control_context = control_context
        self.notes_per_chord = notes_per_chord
        self.temperature = temperature
        self.include_initial = include_initial
        self.top_k = top_k
        self.top_p = top_p

```

```

self.tokens = None
self.generation_method = generation_method
self.midi_music = None
self.seq_length = seq_length

def analyze_and_plot_generation(self, output_prefix="generation"):
    """Complete analysis and plotting for a generation"""
    # Initialize plotters
    music_viz = MusicVisualizer(gen_method=self.generation_method)
    metrics_plotter = MetricsPlotter(gen_method=self.generation_method)
    metrics_calculator = MusicGenerationMetrics()

    # Generate music
    print(f"Generating music and metrics for the method {self.generation_method}")
    if self.generation_method == 1:
        self.midi_music, self.tokens = self.generator.method_1(
            prompt_tokens=self.prompt_tokens,
            style=self.style,
            num_steps=self.num_steps,
            temperature=self.temperature,
            top_k=self.top_k,
            include_initial=self.include_initial,
            allowed_instruments=self.instruments_used
        )
    if self.generation_method == 2:
        self.midi_music, self.tokens = self.generator.method_2(
            prompt_tokens=self.prompt_tokens,
            style=self.style,
            num_steps=self.num_steps,
            temperature=self.temperature,
            top_k=self.top_k,
            include_initial=self.include_initial,
            allowed_instruments=self.instruments_used,
            seq_len=self.seq_length
        )
    if self.generation_method == 3:
        self.midi_music, self.tokens = self.generator.method_3(
            prompt_tokens=self.prompt_tokens,
            style=self.style,
            num_steps=self.num_steps,
            temperature=self.temperature,
            top_k=self.top_k,
            include_initial=self.include_initial,
            allowed_instruments=self.instruments_used,
            notes_per_chord=self.notes_per_chord,
            top_p=self.top_p,
            seq_len=self.seq_length
        )
    if self.generation_method == 4:
        self.midi_music, self.tokens = self.generator.method_4(
            preset_name=self.preset_name,
            prompt_tokens=self.prompt_tokens,
            style=self.style,
            num_steps=self.num_steps,
            temperature=self.temperature,
            top_k=self.top_k,
            include_initial=self.include_initial,
            allowed_instruments=self.instruments_used,
            notes_per_chord=self.notes_per_chord,
            seq_len=self.seq_length,
            top_p=self.top_p,
            control_context=np.array(self.control_context)
        )

    # Plot music timeseries
    print("Plotting music timeseries...")
    music_viz.plot_music_timeseries(
        self.midi_music,

```



```

        title=f"{self.generation_method}_{output_prefix} - Piano Roll",
        filename=f"{self.generation_method}/{output_prefix}_piano_roll.png"
    )

    music_viz.plot_velocity_timeseries(
        self.midi_music,
        title=f"{self.generation_method}_{output_prefix} - Velocity Distribution",
        filename=f"{self.generation_method}/{output_prefix}_velocity.png"
    )

    # Calculate metrics
    print("Calculating metrics...")
    metrics = metrics_calculator.compute_all_metrics(
        self.midi_music,
        target_instruments=self.instruments_used
    )

    # Plot metrics
    print("Plotting metrics...")
    metrics_plotter.plot_single_metrics(
        metrics,
        title=f"{self.generation_method}_{output_prefix} - Quality Metrics",
        filename=f"{self.generation_method}/{output_prefix}_metrics.png"
    )

    # Print summary
    print(f"\n {output_prefix} Metrics Summary:")
    print(f"    Structural Coherence: {metrics.get('ssm_score', 0):.3f}")
    print(f"    Rhythmic Consistency: {metrics.get('ioi_variance', 0):.3f}")
    print(f"    Instrument Usage Ratio: {metrics.get('instrument_usage_ratio', 0):.3f}")
    print(f"    Style Adherence: {metrics.get('style_score', 0):.3f}")
    print(f"    Generating assets that can be rendered in the HTML")
    os.makedirs(f"assets/mid/{self.generation_method}", exist_ok=True)
    os.makedirs(f"assets/wav/{self.generation_method}", exist_ok=True)
    wc.write_mid_and_wav(
        music=self.midi_music,
        mid_file_path=f"assets/mid/{self.generation_method}/{output_prefix}_song.mid",
        wav_file_path=f"assets/wav/{self.generation_method}/{output_prefix}_song.wav"
    )

    return self.midi_music, self.tokens, metrics

# Multiple generations comparison
def compare_multiple_generations(self):
    """Compare multiple generations"""
    all_metrics = []
    all_music = []

    for i in range(self.num_generations):
        print(f"\n Generating sample {i + 1}/{self.num_generations}")
        midi_music, tokens, metrics = self.analyze_and_plot_generation(
            f"sample_{i + 1}"
        )
        all_metrics.append(metrics)
        all_music.append(midi_music)

    # Plot comparison
    metrics_plotter = MetricsPlotter()
    labels = [f"Sample {i + 1}" for i in range(self.num_generations)]

    metrics_plotter.plot_comparison_metrics(
        all_metrics, labels,
        title="Multiple Generations Comparison",
        filename=f"{self.generation_method}/generations_comparison.png"
    )
    return all_music, all_metrics

```

Appendix – I (wav_composer.py)

```
import pretty_midi
import muspy
import numpy as np
from scipy.io import wavfile

SAMPLE_RATE = 44100

def write_mid_and_wav(
    music: muspy.Music = None,
    mid_file_path=None,
    wav_file_path=None, write_mid=True):
    if write_mid:
        print(f"Writing MIDI file {mid_file_path} started")

        music.write_midi(mid_file_path)

        print(f"Writing MIDI file {mid_file_path} completed")

    sf2_path = "fs2/FluidR3_GM.sf2"
    # The above can be downloaded from https://member.keymusician.com/Member/FluidR3_GM/

    midi = pretty_midi.PrettyMIDI(mid_file_path)

    # Generate audio samples using the SoundFont
    audio = midi.fluidsynth(sf2_path=sf2_path, fs=SAMPLE_RATE) if sf2_path else
midi.fluidsynth(fs=SAMPLE_RATE)

    audio = np.int16(audio / np.max(np.abs(audio)) * 32767)

    # Write audio to WAV
    wavfile.write(wav_file_path, SAMPLE_RATE, audio)

    print(f"Saved as {wav_file_path}")

# write_mid_and_wav(muspy.read_midi("20_FOX_DRUMS.mid"), mid_file_path="20_FOX_DRUMS.mid", write_mid=False,
#                    wav_file_path="assets/best_generation_classical.wav")
```